SES Labs

Engineering Solutions. Empowering Businesses.
Installation & Setup Guide
Requirements · Definitions · Every installation scenario · Modules · Devices · Daily running · Troubleshooting

Published by SyncLytics
Software version 0.34.3 (2026-10-14) · Guide generated 2026-10-17
This guide is updated with every release.

Contents

About this guide

- Who to contact
- How to read it
- Boxes used in this guide
- Typing commands
- What is ready today and what comes later

1. Words used in this guide

- The parts of the system
- Your business in the system
- How the software is organised
- Working offline and with several sites
- Technical words you will meet during installation

2. Choose your setup

- The five setups at a glance
- Which one should I choose?
- What each setup needs (and the chapters to follow)
- Moving between setups later

3. What you need

- Hardware
- Software
- Information to prepare

4. Installing the server on Windows (setups A, B and branch servers)

- 4.0 The quick way: the installer (recommended)
- 4.1 Prepare the computer
- 4.2 Install PostgreSQL (the database)
- 4.3 Install PHP
- 4.4 Install Composer and Node.js
- 4.5 Put the software in place
- 4.6 Configure the server
- 4.7 Build the till app (source code only)
- 4.8 Start the server automatically
- 4.9 Let tills and other PCs trust this server (servers without an internet name)
- 4.10 Next steps

5. Installing the server on Ubuntu Linux (setups B, C, D and branch servers)

- 5.0 The quick ways (recommended)
- 5.1 Get the server and connect to it
- 5.2 Update the system and set the time
- 5.3 Install PostgreSQL 17
- 5.4 Install PHP 8.4, Caddy, Composer and Node.js
- 5.5 Put the software in place
- 5.6 Configure the server
- 5.7 Configure the web server (Caddy)
- 5.8 Firewall
- 5.9 Background jobs
- 5.10 Next steps

6. Activating the licence
6.1 What the licence controls
6.2 Activate
6.3 Daily check-in
6.4 Licence states
6.5 Renewing, adding branches or modules
6.6 Moving to a new server

7. First-time set-up in the back office
Step 1 — Sign in for the first time
Step 2 — Set up two-factor sign-in
Step 3 — Activate the licence
Step 4 — The setup wizard
Step 4b — Industries (if you run more than one kind of business)
Step 5 — People and access
Step 6 — Settings
Step 7 — Tax set-up (with your accountant)
Step 8 — Price lists
Step 9 — Products
Step 10 — Opening stock
Step 11 — Customers and suppliers
Step 12 — Finance and HR (if licensed)
Step 13 — Set up the devices
Step 14 — Cash control: vaults, Cashier and Income Controller (optional)
Step 15 — Test before going live

8. Installing and setting up modules
8.0 Industries
8.1 Switching a module on or off
8.2 Available modules and their checklists
8.3 Company-wide features (licence options)
8.4 Planned modules

9. Setting up tills and other devices
9.1 How devices are set up
9.2 The till app address
9.3 Trust the server's address
9.4 Till PC (Windows)
9.5 Tablet (Android)
9.6 Choosing light or dark
9.7 Receipt printer and cash drawer
9.8 Barcode scanner
9.9 Handheld (warehouse)
9.10 Kitchen screen (Restaurant)
9.11 Counting stock with a handheld
9.12 Label scale (Grocery)
9.13 When a device is lost, stolen or replaced

10. Branch servers and several sites (setup E)
10.1 What flows where
10.2 Install the branch server
10.3 The Sync Center (System → Sync Center)
10.4 Keeping servers in step

11. Online stores, delivery apps and SMS
11.1 Requirements
11.2 Online stores (Online → Channels)
11.3 Delivery apps (Restaurant)
11.4 SMS, WhatsApp and e-mail
11.5 GRA E-VAT (electronic VAT invoicing)

12. Daily running, backups, updates and moving
12.1 Every day
12.2 Backups (automatic, encrypted)
12.3 Restoring from a backup
12.4 Updating the software
12.5 Moving to a new server

13. Troubleshooting
Installation
Back office
Tills
Branch servers and online stores
Health checks
Getting help from SyncLytics

14. Exporting and importing data, stock adjustments and stock takes
14.1 Exporting (downloading) anything you can see
14.2 Importing data from CSV files
14.3 Stock adjustments
14.4 Stock takes (counting stock)

15. Insights, alerts and the AI assistant
15.1 What works straight away (no AI needed)
15.2 The AI assistant — off unless you switch it on
15.3 Using the AI features safely
15.4 Troubleshooting

16. Messages, reports and connecting other systems
16.1 E-mail (any mail server)
16.2 SMS — choose your provider
16.3 WhatsApp Business (optional)
16.4 Which messages are sent, and their wording
16.5 Digital receipts at the till
16.6 Reports you build yourself
16.7 Exporting to QuickBooks, Xero or Sage
16.8 Connecting other systems (the API)
16.9 Importing from Excel

17. Going live
17.1 Make the plan
17.2 Bring the data over
17.3 Let staff practise: the practice company
17.4 Check the figures
17.5 The switch-over
17.6 During the pilot
17.7 Training material
17.8 The rollback plan: going back to the old system

17.9 The independent security test

18. Reference

18.1 Security levels
18.2 Roles provided
18.3 Settings in `.env`
18.4 Network ports
18.5 Useful commands (in the `server` folder)
18.6 Till keyboard shortcuts
18.7 Go-live checklist
18.8 About this guide

19. Case study: a cloud server from start to finish

19.1 What was decided first
19.2 Renting the server (Contabo)
19.3 Pointing the internet name at the server (Namecheap)
19.4 Getting the installation files
19.5 Copying the files to the server (from Windows)
19.6 Installing
19.7 Licence and first sign-in
19.8 After installing (the same day)
19.9 Time and cost (this installation)

About this guide

This guide takes you from an empty computer to a working shop, restaurant, pharmacy or chain of branches running **SES Labs**. It is written for someone who is **not** an IT professional. Every step says **what** to do, and where it helps, **why** it is needed.

Who to contact

SES Labs is made, licensed and supported by **SyncLytics**:

Hotline (support)	024 563 6889
Office	055 668 6975 · 050 693 8589
E-mail	info@synclyticsenterprisesystems.com
Website	synclyticsenterprisesystems.com
Address	Puff Cl, Race Course St, Tabora, Accra (GhanaPost GPS GC-027-0884)

How to read it

1. Read **Chapter 1 (Words used in this guide)** once. It explains terms such as *server*, *till*, *branch*, *module* and *licence*.
2. Use **Chapter 2 (Choose your setup)** to pick the installation that fits your business. Each setup lists the chapters you need, in order.
3. Check **Chapter 3 (What you need)** before buying or preparing anything.
4. Follow the installation chapter for your setup **in order**. Do not skip steps. Each step ends with a **Check** that tells you how to know it worked.
5. Keep the guide for daily running: backups, updates and troubleshooting are at the end.

Boxes used in this guide

Why: explains the reason for a step, so you understand what you are doing.

Note: extra information that is useful but not required.

Tip: a shortcut or a good habit.

Warning: something that can lose data, money or security if ignored. Read these carefully.

Check: how to confirm that a step worked before you move on.

Typing commands

Some steps ask you to type a **command**. A command is a line of text you type into a black window called a **terminal** and confirm with **Enter**.

- On **Windows** the terminal is **PowerShell**. Open it like this: click **Start**, type `PowerShell`, then click **Windows PowerShell**. When a step says *as administrator*, right-click it and choose **Run as administrator**, then click **Yes**.
- On **Ubuntu Linux** the terminal is called **Terminal**, or you reach the server through **SSH** (explained in Chapter 5).

Commands are shown in a dark box, like this:

```
php --version
```

Type them exactly as shown (you can copy and paste). Text in angle brackets, such as `<your-password>`, is a placeholder: replace the whole thing, brackets included, with your own value.

> **Tip:** In PowerShell you can paste with a right-click. Press the **Up arrow** key to bring back the last command.

What is ready today and what comes later

The software is built in phases. This guide always describes the software **as it is now**, and is updated with every release.

Area	Today	Later
Server installation	Step-by-step manual installation (this guide) on Windows or Ubuntu Linux	One-click installers for every setup (Phase 13)
Till app	Runs in the Chrome or Edge browser and can be installed as an app; works offline	Optional desktop app with printer and cash-drawer drivers (Phase 13)
Backups	Manual and scheduled database backups (Chapter 12)	Encrypted automatic backups with a tested restore (Phase 13)
Industry modules	Grocery, Pharmacy, Restaurant, Electronics, Fashion, Services, Sales & Distribution, Manufacturing, Online Storefront	Logistics, Assets, Projects and Helpdesk
Insights and AI	Forecasts, stock health, customers, pricing, cash flow, loss prevention and alerts (no AI needed); optional local AI assistant (Chapter 15)	Scheduled reports by e-mail (Phase 12)

> **Warning:** Use the software with test data until your accountant has checked the tax set-up (Chapter 7, step 7) and you have practised a backup and a restore (Chapter 12).

1. Words used in this guide

You do not need to remember these. Come back to this page whenever a word is unclear.

The parts of the system

Word	What it means
Server	The computer that keeps all the data (products, prices, stock, sales, customers, accounts) and runs the software. Every other device talks to it. It can be a PC in your shop, a computer in your office, or a rented computer on the internet (*cloud*).
Back office	The management screens, opened in a web browser at `http://<server>/admin`. Managers use it to set up products and prices, receive stock, see reports, run payroll and so on.
Till (POS, point of sale)	The selling screen used by cashiers. It runs in the Chrome or Edge browser at `http://<server>/pos/` on a PC, laptop or tablet, and keeps working when the server or the internet is down.
Device	Any till, handheld scanner, kitchen screen or self-service screen. Each one is *registered* in the back office and *enrolled* once with a one-time code, so only your own devices can sell.
Enrolment code	A one-time code (like `ABCD-EFGH-JKMN`) that the back office shows when you register a device. You type it on the device once.
Handheld	A phone or rugged scanner used in the warehouse to receive goods, put them away and count stock.
Kitchen screen (KDS)	A screen in the kitchen or bar that shows the orders to prepare (Restaurant module).
Online store / channel	A website that sells your products (the built-in shop, WooCommerce or Shopify), or a delivery app. It receives prices and stock automatically and sends its orders back.

Your business in the system

Word	What it means
Company	Your business. One installation normally holds one company.
Legal entity	The registered company, with its TIN and VAT number, printed on VAT invoices.
Branch	One shop, restaurant, pharmacy or warehouse location. One branch is the **head office** (HQ).
Warehouse	A place where stock is kept. Every branch has a main warehouse; you can add more (store room, cold room, van).
Till (in the set-up)	A selling point at a branch, for example *Till 1*. A device is assigned to one till.
Branch code	1–6 letters or digits, such as `HQ` or `KSI`. It appears in receipt numbers and cannot be changed later.
User	A person who signs in. Back-office users sign in with email, password and a second factor. Cashiers sign in on the till with a **username and PIN**.

Word	What it means
Role	A set of permissions, such as *Cashier*, *Supervisor*, *Branch Manager* or *Accountant*. You give people roles.
Scope	Where a role applies: the whole company, one branch, one warehouse or one till. A cashier with a till scope can sell only on that till.
Security level	A ceiling (L1 to L6) on what a person can ever be given. It stops anyone from handing out more power than they have.
Two-factor sign-in (2FA)	A second proof of identity after the password: a 6-digit code from an authenticator app on your phone, or a passkey. Required for managers.
Passkey	A sign-in using your device's fingerprint, face or PIN instead of a code. Optional.

How the software is organised

Word	What it means
Core	The part every business has: company, branches, users, products, prices, tax, stock, tills, customers, purchasing, sales, finance and HR (if licensed).
Module	An extra package for a type of business, switched on **per branch**: Grocery, Pharmacy, Restaurant, Electronics, Fashion, Services, Online Storefront, and more later. A branch can use several.
Licence	The key from SyncLytics that allows the software to run. It sets how many branches, devices and users you may have and which modules and features (Finance, HR & payroll) you bought.
Installation code	A code starting with `SYNREQ-` that your server shows on the Licence page. You send it to SyncLytics to receive your licence key (starting with `SYN1-`).
Read-only	What happens when a licence expires or is withdrawn: you can still see everything, but nothing can be changed or sold. Nothing is deleted.

Working offline and with several sites

Word	What it means
Offline	Without a connection to the server. Tills keep selling offline and send their sales later.
Offline sign-in (grant)	Permission for a cashier to sign in on a till without the server. It is given when they sign in on that till while it is online, and lasts 72 hours by default.
Queue	The list of sales a till made offline that are waiting to be sent. They are sent automatically.
Review queue	A list in the back office of things a manager should check, such as a sale made offline at an old price.
Node	One server running the software. A company can have several.
HQ server (head office)	The main server. It owns the master data (products, prices, users) and receives everything from the branches.
Branch server	An optional server **at** a branch. The branch keeps working fully (tills, back office, stock) when the internet to head office is down, and exchanges changes with HQ when it returns.
Standalone	A single server for a single site, with no other servers.

Word	What it means
Sync	Sending changes between servers (branch ⇄ HQ) and from tills to their server. It is automatic.

Technical words you will meet during installation

Word	What it means
Operating system	The basic software of a computer: Windows 10/11, or Ubuntu Linux on servers.
PHP	The programming language the server is written in. You install it so the server can run.
Composer	A tool that downloads the PHP building blocks (libraries) the server needs.
Node.js / npm	A tool used once during installation to build the till app.
PostgreSQL	The database: the program that stores all your data safely.
Database	The organised store of all your data inside PostgreSQL. It is called `erp`.
`.env` **file**	A small settings file on the server holding passwords and keys. It must stay private and be backed up.
APP_KEY / ERP_AUDIT_KEY	Secret keys in `.env`. APP_KEY protects sessions and stored secrets; ERP_AUDIT_KEY seals the audit log. Lose them and some data cannot be read or verified, so **back them up**.
Scheduler	A background timer that runs the software's routine jobs every minute: sending changes to online stores, syncing branches, posting to the accounts, reminders. It **must** run all the time.
Live updates (Reverb)	An optional service that pushes price and stock changes to tills and screens instantly. Without it, tills check every 60 seconds.
IP address	The number of a computer on the network, such as `192.168.1.10`. Tills use it to find the server.
LAN	Your shop's local network (cables or Wi-Fi through your router).
Port	A numbered "door" on a computer. The server uses the standard web ports: 443 for https, and 80 only to send visitors on to https (and, on the shop network, to hand out the server's certificate).
Certificate	The proof a web server shows a browser that it is really that server, which also encrypts everything sent between them (the padlock in the address bar).
HTTPS / SSL certificate	Encrypted web connection (the padlock in the browser). Needed when tills reach the server over the internet, and for some browser features.
Domain	A web address you own, such as `erp.yourshop.com`. Needed for a cloud server.
Firewall	Protection that blocks unwanted network connections. You open only the ports you need.
Service	A program that starts automatically with the computer and runs in the background.
Backup	A copy of the database and the `.env` file kept somewhere else, so you can recover after a theft, fire or disk failure.

2. Choose your setup

There are five ways to run SES Labs. They use the **same software**; only where the server sits changes. You can start small and grow later (for example, start with one shop and add a cloud head office when you open a second branch).

The five setups at a glance

	Setup	Best for	Server	Where tills connect
A	**One computer**	A small shop with one till	The till PC itself (Windows)	The same PC (`https://localhost/pos/`)
B	**One shop, several tills**	A shop, pharmacy or restaurant with 2–20 tills and devices	One PC or small server in the shop (Windows or Ubuntu)	Over the shop's network (LAN / Wi-Fi)
C	**Central server for several branches**	Branches joined by a private network (VPN) or in one building	A server at head office (Ubuntu recommended)	Over the private network
D	**Cloud head office**	Several branches in different towns, owner wants to see everything from anywhere	A rented cloud server with a domain name and HTTPS (Ubuntu)	Over the internet
E	**Cloud or central HQ + branch servers**	Branches with unreliable internet, busy restaurants, supermarkets	HQ as in C or D, plus a branch server at each such branch	Tills use their **branch** server over the LAN; branch servers sync with HQ

> **Why:** a till can sell offline in every setup, but the back office, kitchen screens, handhelds, shared restaurant bills and stock look-ups need the server. If the server is in the cloud and the internet is down, those stop until it returns. A branch server (setup E) keeps them working on the local network.

Which one should I choose?

Answer these questions in order:

1. **Do you have only one location?**
   - One till only → **A**.
   - Several tills, kitchen screens or handhelds → **B**.
2. **Several locations?**
   - Do all branches have reliable internet (or a private network)? → **D** (cloud) or **C** (central server with VPN).
   - Do some branches have unreliable internet, or need kitchen screens, handhelds or shared restaurant bills to keep working during outages? → **E** (add a branch server at those branches).

> **Tip:** Most businesses with 2–5 branches in Ghana do well with **D** for head office plus a branch server (**E**) only at branches with weak internet or a restaurant.

What each setup needs (and the chapters to follow)

A — One computer

- One Windows 10/11 PC (the till) with a receipt printer and barcode scanner.
- Chapters: 3 → **4 (Windows)** → 6 → 7 → 8 → 9 → 12.
- The till opens at `https://localhost/pos/` on the same PC, so no extra network set-up is needed.

B — One shop, several tills

- One PC or small server that stays switched on during trading hours, with a **fixed IP address** on the shop network.
- Till PCs/tablets on the same network (cable or Wi-Fi).
- Chapters: 3 → **4 (Windows)** or **5 (Ubuntu)** → 6 → 7 → 8 → 9 (including 9.3, *Trust the server's network address*, on every device) → 12.

> **Note:** With the server on Windows today, the web server built into PHP answers one request at a time. That is fine for about 1–4 tills. For more tills, kitchen screens and handhelds, use an Ubuntu server (Chapter 5) until the Phase 13 Windows installer arrives.

C — Central server for several branches

- One Ubuntu server at head office, reachable from the branches through a VPN (your network provider can set this up) or within one building.
- Chapters: 3 → **5 (Ubuntu)** → 6 → 7 → 8 → 9 → 10 (optional branch servers) → 12.

D — Cloud head office

- A cloud server (VPS) running Ubuntu 24.04, a **domain name** (for example `erp.yourshop.com`) and a free HTTPS certificate (set up in Chapter 5).
- Chapters: 3 → **5 (Ubuntu, including HTTPS)** → 6 → 7 → 8 → 9 → 11 (online stores) → 12.

E — HQ plus branch servers

- HQ as in C or D.
- At each chosen branch: a PC or small server installed like setup B, then **paired** with HQ.
- Chapters: first set up HQ (C or D). Then, for each branch server: 4 or 5 → **10 (Branch servers)** → 9 → 12.

Moving between setups later

From → To	What to do
A → B	Give the PC a fixed IP address and set up the other tills (Chapter 9). Nothing is reinstalled.
A/B → D	Install a cloud server, then move the database to it (Chapter 12, *Moving to a new server*). The licence must be re-activated on the new server.
D → E	Install a branch server at the branch and pair it (Chapter 10). Re-enrol that branch's tills on the branch server.

3. What you need

Hardware

The server

Setup	Minimum	Recommended
A (one PC, also the till)	Windows 10/11 64-bit, 4-core CPU, 8 GB RAM, 256 GB SSD	16 GB RAM
B (shop server)	Windows 10/11 Pro or Ubuntu 24.04, 4 cores, 8 GB RAM, 256 GB SSD	16 GB RAM, Ubuntu for more than 4 devices
C / D (HQ, central or cloud)	Ubuntu 24.04, 2 vCPU, 4 GB RAM, 80 GB SSD	4 vCPU, 8 GB RAM, 160 GB SSD; scale up with branches
E (branch server)	as B	as B

> **Why:** the database works best on an **SSD** (not a spinning hard disk). Memory lets the database keep frequently used data in RAM, which keeps the tills fast.

Sizing by business size (head office or shop server; the SSD matters more than the processor):

Business	Example	Memory	Processor	Disk
Small	1 shop, up to 3 tills, under 5,000 products, under 500 receipts a day	8 GB	4 cores	256 GB SSD
Medium	2–5 branches, up to 15 tills, under 20,000 products	16 GB	4–6 cores	512 GB SSD
Large	6+ branches or 30+ tills, or several industries with separate books	32 GB	8 cores	1 TB NVMe SSD

Branch servers (Setup E) need the *Small* size. If **System** → **Health** → **Slow pages** keeps filling up, add memory first.

Also needed for an on-site server:

- A **UPS** (battery backup) so a power cut does not switch the server off mid-write. At least 15 minutes, with enough time to shut down safely.
- The **correct date and time**, set automatically from the internet. Wrong clocks cause sync refusals.
- A place where it will not be unplugged by accident, and is not open to customers.

Tills and other devices

Device	Requirement
Till PC	Windows 10/11 with Google Chrome or Microsoft Edge (current version), 4 GB RAM, screen at least 1280 × 720 (touch screen optional)
Till tablet	Android 10 or later with Chrome, 10" screen recommended

Device	Requirement
Receipt printer	80 mm thermal receipt printer installed as a normal Windows/Android printer (USB, network or Bluetooth). Receipts are printed through the browser's print window. A4 printers also work.
Cash drawer	One that opens from the receipt printer (RJ11 cable to the printer), with the printer driver set to *open drawer* when printing
Barcode scanner	Any USB or Bluetooth scanner that acts as a **keyboard** (most do). It must send **Enter** after each code.
Handheld (warehouse)	Android phone or rugged scanner with Chrome and a built-in scanner or camera
Kitchen screen	Any PC, tablet or TV stick with Chrome; touch screen helpful
Label scale (grocery)	A scale that prints EAN-13 labels with prefix 21 (see Chapter 8, Grocery)

Network

- **Setup A:** no network needed for selling. Internet is needed for the daily licence check-in (it tolerates 30 days without).
- **Setup B/C/E:** a router with cable or reliable Wi-Fi. Give the server a **fixed (reserved) IP address** in the router so tills always find it.
- **Setup D:** a reliable internet connection at every branch (4G backup router recommended). Tills keep selling during outages.
- **Internet for head office:** needed once a day for the licence check-in, and for online stores, SMS and delivery apps.

Software

All software below is free. Download only from the official addresses listed.

Software	Version	Needed on	Official download
PostgreSQL	17	Server	`https://www.postgresql.org/download/` (Windows: the EDB installer)
PHP	8.4 (8.3 minimum), 64-bit, *Non Thread Safe* on Windows	Server	`https://windows.php.net/download/` (Windows); Ubuntu: Ondřej Surý's PPA, see Chapter 5
PHP extensions	`pdo_pgsql`, `pgsql`, `intl`, `bcmath`, `sodium`, `mbstring`, `openssl`, `curl`, `fileinfo`, `zip`, `gd`	Server	Included with PHP
Composer	2.x	Server (installation and updates)	`https://getcomposer.org/download/`

Software	Version	Needed on	Official download
Node.js (with npm)	22 LTS or later	Server (installation and updates only)	`https://nodejs.org/`
Git	any recent	Server (to download and update the software), optional	`https://git-scm.com/`
Google Chrome or Microsoft Edge	current	Every till, handheld, kitchen screen, and your own PC for the back office	`https://www.google.com/chrome/`
Authenticator app	any (Google Authenticator, Microsoft Authenticator, Authy…)	Phones of managers	Phone app store
Caddy (web server, https certificates)	from caddyserver.com	All servers	installed by the installer or the script (Chapters 4 and 5)
Ollama (optional)	current	Server, only for the local AI assistant	`https://ollama.com/download` — see Chapter 15 (needs 16 GB RAM or more)

Why Node.js on the server? It is used once during installation (and after each update) to build the till app. It does not run while you trade.

Information to prepare

Write these down before you start (keep the sheet in a safe place, not on the server):

Item	Example	Used in
Company trading name	Kofi Stores	Chapter 7
Registered name, TIN, VAT number	Kofi Stores Ltd, C0012345678	Chapter 7
Branches: code, name, number of tills, modules	HQ · Head Office · 2 tills · Grocery	Chapter 7
Owner: full name, username, email, password (12+ characters)		Chapter 4/5
Database password (long and random)		Chapter 4/5
Server IP address or domain	192.168.1.10 / erp.kofistores.com	Chapters 4, 5, 9
Your tax set-up, checked by your accountant	VAT 15 %, NHIL 2.5 %, GETFund 2.5 %	Chapter 7
Products list with prices, barcodes and opening stock		Chapter 7

Item	Example	Used in
Staff list with roles		Chapter 7

> **Warning:** Never use real customer data for practice or testing. Use made-up names and numbers until you go live.

4. Installing the server on Windows (setups A, B and branch servers)

Follow every step in order. Allow about two hours the first time. You need an internet connection during installation.

In this chapter the software goes in `C:\SESLabs` and PHP in `C:\php`. If you choose other folders, change them in every command.

4.0 The quick way: the installer (recommended)

SES Labs gives you `SESLabs-server-setup-<version>.exe` with your release. It does everything in 4.2–4.9 for you.

1. Do 4.1 (prepare the computer) first.
2. Double-click the setup file, click **Yes** when Windows asks for permission (it must install a database and services).
3. Choose **Setup A** (this PC is also the till) or **Setup B** (shop server for tills on the network). Leave the server id as **HQ** unless SyncLytics gave you a branch server id (Chapter 10).
4. **Reached from the internet?** Leave both boxes empty for a server used only in the shop (the usual case). Only if this server has an internet name pointing to it (for example `shop.example.com`, with ports 80 and 443 forwarded to it by your router), type that name and an e-mail address. **Why:** with an internet name, a free certificate from Let's Encrypt is fetched and renewed automatically. Without one, the server makes its own certificate for the shop network (4.9).
5. A blue window shows each step. It downloads PostgreSQL, PHP, Composer and the **Caddy** web server from their official sites (about 450 MB; keep the internet on) and checks each download against its published checksum. It then creates the database with its own random password, the settings, the keys, the tables, and the start-up tasks.
6. At the end it shows the **backup passphrase** — write it on paper and keep it away from the server (Chapter 12) — and asks for your company name and the owner's account.

> **Why:** the installer follows exactly the manual steps below, so you can still read them to understand what was done, or do them by hand on an unusual PC.

> **Note:** the passwords the installer created (database) are in `C:\SESLabs\installer-secrets.txt`, readable by administrators only. Running the installer again (a newer version) updates the software and keeps your data, settings and keys.

Advanced: the same steps without the setup file: `powershell -ExecutionPolicy Bypass -File install.ps1 -Release <release.zip> -Setup B` (as administrator).

The server is then reached over **https** only: `https://localhost/admin` (setup A), or `https://<server-address>/admin` (setup B, for example `https://192.168.1.10/admin`). For setup B, every till, tablet and other PC must trust the server once: see **4.9**.

If you used the installer, do 4.9 (setup B), then continue with Chapter 6 (licence). The rest of this chapter is the manual way.

4.1 Prepare the computer

1. **Install all Windows updates** (Start → Settings → Windows Update), restart, and repeat until none are left. **Why:** updates fix security holes and avoid restarts in the middle of trading later.
2. **Set the clock to update automatically:** Settings → Time & language → Date & time → switch on *Set time automatically* and *Set time zone automatically* (or choose *(UTC+00:00) Monrovia, Reykjavik* for Ghana). **Why:** receipts, shifts and sync depend on the correct time. Tills more than 10 minutes ahead of the server are refused.
3. **Stop the PC from sleeping:** Settings → System → Power → *Screen and sleep* → set *When plugged in, put my device to sleep after* to **Never**. **Why:** a sleeping server cannot answer the tills.
4. **Setup B only — give the PC a fixed address on the network.** In your router's settings, find *DHCP reservation* (sometimes *Address reservation* or *Static lease*) and reserve an address for this PC, for example `192.168.1.10`. Ask whoever installed your router if unsure. Write the address down. **Why:** tills find the server by its address. If the address changes after a restart, the tills cannot connect.

> **Check:** open PowerShell and type `ipconfig`. The *IPv4 Address* of your network adapter shows the address (for setup B it must be the reserved one).

4.2 Install PostgreSQL (the database)

1. Download the **PostgreSQL 17 Windows installer** from `https://www.postgresql.org/download/windows/` (the *Download the installer* link goes to EDB).
2. Run it and accept the defaults, with these choices:
   - **Components:** keep *PostgreSQL Server* and *Command Line Tools*. *pgAdmin 4* is optional (a graphical tool). Untick *Stack Builder*.
   - **Password:** type a strong password for the *postgres* administrator account. **Write it down** in your safe place.
   - **Port:** keep `5432`.
   - **Locale:** keep *Default locale*.
3. Finish. PostgreSQL now runs as a Windows service and starts with the computer.

> **Why:** PostgreSQL is where every sale, product and account is stored. It is designed never to lose committed data, even in a power cut.

Now create the database and its own user. Open **PowerShell** and type (replace `<db-password>` with a new long random password, different from the *postgres* one, and write it down):

```
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -c "CREATE ROLE erp LOGIN PASSWORD
'<db-password>';"
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -c "CREATE DATABASE erp OWNER erp EN
CODING 'UTF8';"
```

Each command asks for the *postgres* password you chose in step 2.

> **Why:** the software uses its own database user (`erp`) that can only reach its own database, not the administrator account.

> **Check:** both commands answer `CREATE ROLE` and `CREATE DATABASE` .

4.3 Install PHP

1. Install the **Microsoft Visual C++ Redistributable (x64)** for Visual Studio 2015–2022 from Microsoft's website (`https://aka.ms/vs/17/release/vc_redist.x64.exe`). **Why:** PHP for Windows is built with it and will not start without it.
2. Go to `https://windows.php.net/download/` and download **PHP 8.4, VS17 x64 Non Thread Safe, Zip**.
3. Create the folder `C:\php` and extract the ZIP **into** it, so that `C:\php\php.exe` exists.
4. In `C:\php` , copy `php.ini-production` and rename the copy to `php.ini` .
5. Open `php.ini` in Notepad and change these lines (search with Ctrl+F; remove the `;` at the start of a line to switch it on):

```
extension_dir = "ext"
extension=curl
extension=fileinfo
extension=gd
extension=intl
extension=mbstring
extension=openssl
extension=pdo_pgsql
extension=pgsql
extension=sodium
extension=zip
memory_limit = 512M
upload_max_filesize = 20M
post_max_size = 25M
date.timezone = Africa/Accra
upload_tmp_dir = "C:\SESLabs\tmp"
sys_temp_dir = "C:\SESLabs\tmp"
```

Create the folder `C:\SESLabs\tmp` (it holds files while they are being uploaded, e.g. CSV imports).

Why: these extensions let PHP talk to PostgreSQL (`pdo_pgsql`), handle money and text in every language (`bcmath` , which is built in, `intl` , `mbstring`), and protect secrets (`sodium` , `openssl`). 6. Add PHP to the system path, so Windows finds `php` from any folder: Start → type `environment` → **Edit the system environment variables** → **Environment Variables…** → under *System variables* select **Path** → **Edit** → **New** → type `C:\php` → **OK** three times. 7. Close PowerShell and open it again (it reads the new path only when it starts).

> **Check:** type `php --version`. It shows `PHP 8.4…`. Then type `php -m` and make sure the list contains `bcmath`, `intl`, `pdo_pgsql`, `pgsql` and `sodium`.

4.4 Install Composer and Node.js

1. Download and run **Composer-Setup.exe** from `https://getcomposer.org/download/`. When it asks for PHP, choose `C:\php\php.exe`. Leave the proxy empty.
2. Download and run the **Node.js LTS** Windows installer (`.msi`) from `https://nodejs.org/`. Accept the defaults. You do not need the *Tools for native modules* option.
3. Close PowerShell and open it again.

> **Check:** `composer --version` shows `Composer version 2…`, and `node --version` shows `v22…` or later.

> **Why:** Composer downloads the server's building blocks; Node.js builds the till app. Both are only used during installation and updates.

4.5 Put the software in place

SES Labs gives you the software as a ZIP file named like `ses-labs-<your-company>-<build>.zip`.

1. Create the folder `C:\SESLabs`.
2. Extract the ZIP into it, so that `C:\SESLabs\server\artisan` exists.

> **Note:** If you were given the **source code** (for example through Git) instead of a release ZIP, place it so that `C:\SESLabs\server` and `C:\SESLabs\pos` exist, and also do step 4.7 (*Build the till app*).

4.6 Configure the server

Open PowerShell and go to the server folder:

```
cd C:\SESLabs\server
```

1. **Download the server's libraries:**

   ```
   composer install --no-dev --optimize-autoloader
   ```

   > **Why:** the release contains only SES Labs' own code; this downloads the open-source libraries it builds on, at the exact versions it was tested with. `--no-dev` leaves out testing tools.

2. **Create the settings file** from the example:

   ```
   copy .env.example .env
   notepad .env
   ```

Change these lines and save:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://<server-address>
SESSION_SECURE_COOKIE=true
DB_PASSWORD=<db-password>
ERP_NODE_ROLE=hq
ERP_NODE_ID=HQ
```

- `<server-address>`: `localhost` for setup A; the fixed address from 4.1 (e.g. `192.168.1.10`) for setup B.
- For a **branch server**, use `ERP_NODE_ROLE=branch` and the branch server's id (e.g. `B01`) as given in Chapter 10.

> **Why:** `.env` tells the software where its database is, what address it is reached on, and that it is a live system (`APP_DEBUG=false` hides technical error details from users). `SESSION_SECURE_COOKIE=true` makes browsers send the sign-in cookie only over https.

> **Warning:** `.env` will hold your secret keys. Never e-mail it, never put it on a shared drive, and include it in your backups (Chapter 12).

3. **Create the secret keys:**

```
php artisan key:generate
php artisan erp:audit-key
```

> **Why:** `key:generate` creates `APP_KEY`, which encrypts sessions and stored secrets. `erp:audit-key` creates `ERP_AUDIT_KEY`, which seals the audit log so changes to history can be detected. **Never change either key after this.**

4. **Create the database tables and the standard data** (roles, permissions, units, tax templates, chart of accounts):

```
php artisan migrate --seed --force
```

> **Why:** this builds the empty structure of the database. `--force` confirms you are on a live system.

> **Check:** it ends with lines saying `DONE` and no red errors.

5. **Create your company and the owner account:**

```
php artisan erp:install
```

It asks for the company name, the owner's full name, a username (for till sign-in), email and a password of at least 12 characters.

> **Why:** the owner is the first person who can sign in; they create everyone else.

> **Note:** Skip this step on a **branch server**: its company and people come from head office (Chapter 10).

6. **Build the files for the online shop's live updates** (optional, but harmless):

```
npm install
npm run build
```

7. **Speed up the server** (run again after every change to `.env`):

```
php artisan optimize
```

4.7 Build the till app (source code only)

Skip this if your release ZIP already contains `C:\SESLabs\server\public\pos\index.html`.

```
cd C:\SESLabs\pos
npm install
npm run build:server
```

> **Why:** this turns the till's source code into the app the server hands out at `/pos/`, including the offline files that let tills keep selling without the server.

> **Check:** the file `C:\SESLabs\server\public\pos\index.html` now exists.

4.8 Start the server automatically

The server needs these programs running all the time:

- the **web server (Caddy)**: the only program the tills and browsers talk to. It serves everything over **https**, so passwords, PINs and sales cannot be read by anyone else on the network;
- **PHP workers**: they run SES Labs itself, behind the web server. Each one serves one request at a time, so there are four;
- the **scheduler**: it runs the routine jobs every minute (sending to online stores, syncing branches, posting to the accounts, reminders, the daily licence check-in, the nightly insights and alerts);
- **live updates (Reverb)**: they tell tills about price and stock changes within seconds. They are reached through the web server, so no extra port is open.

We register them with Windows **Task Scheduler**, so they start with the computer even when nobody is signed in, and restart if they stop.

Step 1: get Caddy. Download `caddy_2.10.0_windows_amd64.zip` from the official releases page `https://github.com/caddyserver/caddy/releases` and unzip it to `C:\SESLabs\caddy`.

> **Why:** Caddy is a widely used web server that gets and renews https certificates by itself.

Step 2: protect its data folder. In PowerShell as administrator:

```
New-Item -ItemType Directory -Force C:\SESLabs\caddy\data
icacls C:\SESLabs\caddy\data /inheritance:r /grant:r "Administrators:(OI)(CI)F" "SYSTEM:(OI)(CI)F"
```

> **Why:** on the shop network Caddy acts as its own small certificate authority, and its secret key lives in this folder. Only administrators may read it.

Step 3: the settings file. Create `C:\SESLabs\Caddyfile` with Notepad. This example is for setup B with address `192.168.1.10` and computer name `shopserver`; change both. For setup A, use `localhost` as the address, remove the whole `http://…` block, and add the line `default_bind 127.0.0.1` under `admin off`.

```
{
	admin off
	skip_install_trust
	storage file_system "C:/SESLabs/caddy/data"
	pki {
		ca local {
			name "SES Labs on shopserver"
			root_cn "SES Labs shopserver Root"
			intermediate_cn "SES Labs shopserver Intermediate"
		}
	}
}

http://192.168.1.10, http://shopserver {
	handle /ca.crt {
		root * "C:/SESLabs/caddy/data/pki/authorities/local"
		rewrite * /root.crt
		header Content-Type application/x-x509-ca-cert
		file_server
	}
	handle {
		redir https://{host}{uri} permanent
	}
}

192.168.1.10, shopserver {
	tls internal
	root * "C:/SESLabs/server/public"
	encode zstd gzip
	request_body {
		max_size 25MB
	}
	header -Server
	@hidden {
		path_regexp /\.
		not path /.well-known/*
	}
	handle @hidden {
		respond 404
	}
	@live path /app/* /apps/*
	handle @live {
		reverse_proxy 127.0.0.1:8080
	}
	handle /pos/* {
		header /pos/sw.js Cache-Control no-cache
```

```
                try_files {path} /pos/index.html
                file_server
        }
        handle {
                php_fastcgi 127.0.0.1:9001 127.0.0.1:9002 127.0.0.1:9003 127.0.0.1:9004 {
                        lb_policy least_conn
                        lb_try_duration 10s
                        read_timeout 300s
                }
                file_server
        }
}
```

> **Why:**

- *Plain http* only hands out the server's certificate (4.9) and sends everything else to https.
- *Hidden files* such as `.env` are never served.
- *Live updates* use the same https address.
- *The till app* falls back to its own start page, so it works offline.
- *Everything else* goes to the PHP workers.

For a server with an **internet name**:

- remove the `http://…` block and the `tls internal` line;
- write the name instead of the address;
- add `email you@example.com` under `admin off`.

Caddy then gets a Let's Encrypt certificate by itself.

> **Check:** `C:\SESLabs\caddy\caddy.exe validate --config C:\SESLabs\Caddyfile --adapter caddyfile` ends with *Valid configuration*.

Step 4: the PHP worker. Create `C:\SESLabs\php-worker.cmd`:

```
@echo off
set PHP_FCGI_MAX_REQUESTS=0
:run
"C:\php\php-cgi.exe" -b 127.0.0.1:%1
ping -n 3 127.0.0.1 >nul
goto run
```

> **Why:** PHP on Windows stops a worker after 500 requests unless told otherwise. The loop restarts it if it ever stops.

Step 5: in `.env`, add or change these lines (with your address), then run `php artisan optimize`:

```
REVERB_APP_ID=<random-1>
REVERB_APP_KEY=<random-2>
REVERB_APP_SECRET=<random-3>
REVERB_HOST=192.168.1.10
REVERB_PORT=443
REVERB_SCHEME=https
REVERB_PUBLISH_HOST=127.0.0.1
REVERB_PUBLISH_PORT=8080
REVERB_PUBLISH_SCHEME=http
```

Use three random values of your own (letters and digits, at least 20 characters each).

> **Why:** tills connect to live updates through the https address. The server itself sends to them directly on the same PC.

Step 6: the start-up tasks. Open **PowerShell as administrator** and paste this whole block:

```
$app = "C:\SESLabs\server"; $php = "C:\php\php.exe"
$settings = New-ScheduledTaskSettingsSet -ExecutionTimeLimit ([TimeSpan]::Zero) -RestartCount 99
9 -RestartInterval (New-TimeSpan -Minutes 1) -AllowStartIfOnBatteries -DontStopIfGoingOnBatterie
s
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel Hig
hest
$trigger = New-ScheduledTaskTrigger -AtStartup
Register-ScheduledTask -TaskName "SES Labs Scheduler" -Trigger $trigger -Settings $settings -Pri
ncipal $principal -Action (New-ScheduledTaskAction -Execute $php -Argument "artisan schedule:wor
k" -WorkingDirectory $app)
Register-ScheduledTask -TaskName "SES Labs Live" -Trigger $trigger -Settings $settings -Principa
l $principal -Action (New-ScheduledTaskAction -Execute $php -Argument "artisan reverb:start --ho
st=127.0.0.1 --port=8080" -WorkingDirectory $app)
foreach ($i in 1..4) { Register-ScheduledTask -TaskName "SES Labs PHP $i" -Trigger $trigger -Set
tings $settings -Principal $principal -Action (New-ScheduledTaskAction -Execute "cmd.exe" -Argum
ent "/c `"`"C:\SESLabs\php-worker.cmd`" $(9000 + $i)`"" -WorkingDirectory $app) }
Register-ScheduledTask -TaskName "SES Labs Web" -Trigger $trigger -Settings $settings -Principal
$principal -Action (New-ScheduledTaskAction -Execute "C:\SESLabs\caddy\caddy.exe" -Argument "run
--config C:\SESLabs\Caddyfile --adapter caddyfile" -WorkingDirectory "C:\SESLabs\caddy")
Get-ScheduledTask -TaskName "SES Labs *" | Start-ScheduledTask
```

> **Why:** `ExecutionTimeLimit 0` stops Windows from ending the tasks after three days (its default). `SYSTEM` runs them without anyone signed in.

Step 7: trust the server on this PC. Wait 10 seconds, then:

```
Import-Certificate -FilePath C:\SESLabs\caddy\data\pki\authorities\local\root.crt -CertStoreLoca
tion Cert:\LocalMachine\Root
```

> **Why:** the server's own browser then accepts its certificate (other devices: 4.9). Skip this step for a server with an internet name.

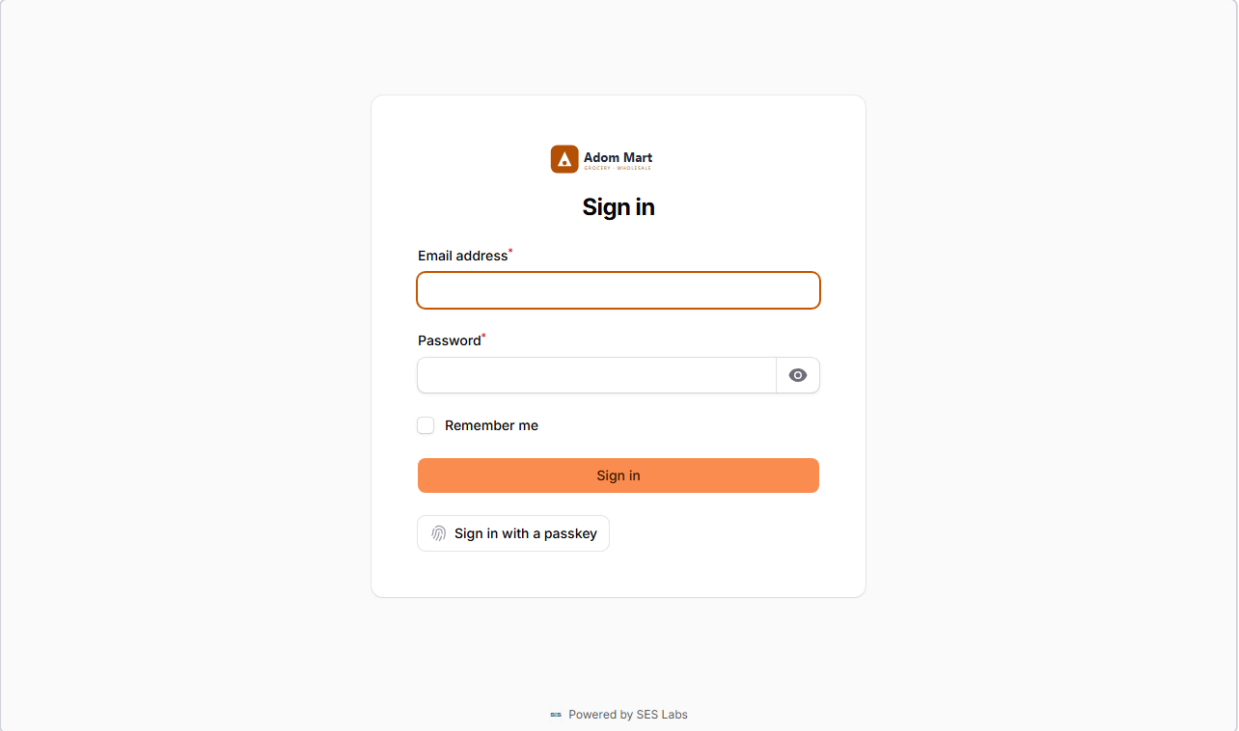
Step 8 (setup B only): let the tills through the Windows firewall:

```
New-NetFirewallRule -DisplayName "SES Labs server" -Direction Inbound -Protocol TCP -LocalPort 8
0,443 -Action Allow -Profile Private,Domain
```

> **Why:** Windows blocks incoming connections by default. This opens only the web server's two ports, and only on private (shop) networks, not public Wi-Fi. The database, the PHP workers and live updates stay reachable from this PC only.

> **Check:**

1. In a browser on the server, open `https://localhost/up` (setup A) or `https://<server-address>/up` (setup B). It shows a page saying *Application up*, with no certificate warning.
2. Open `…/admin`: the sign-in page appears:



3. Setup B: on another PC that trusts the server (4.9), open `https://<server-address>/admin`. The same page appears.

> **Tip:** to restart the server, open **Task Scheduler**, click *Task Scheduler Library*, right-click *SES Labs Web* (or a *PHP*, *Scheduler* or *Live* task), and choose **End**, then **Run**.

4.9 Let tills and other PCs trust this server (servers without an internet name)

On the shop network (and on an Ubuntu server reached by its address, Chapter 5) the server uses **its own certificate**. A certificate is what proves to a browser that it is really talking to your server and that nobody can read along. Each till, tablet, handheld, kitchen screen and office PC must be told **once** to trust it. Without this, browsers show a warning, and the till refuses to enrol: it only uses its security functions on a trusted address.

On a Windows PC (also for the SES Labs Till desktop app): open PowerShell **as administrator** and type, with your server's address:

```
Invoke-WebRequest http://192.168.1.10/ca.crt -OutFile $env:TEMP\seslabs.crt
Import-Certificate -FilePath $env:TEMP\seslabs.crt -CertStoreLocation Cert:\LocalMachine\Root
```

On an Android tablet or phone:

1. In Chrome, open `http://<server-address>/ca.crt`. It downloads a file.
2. Open **Settings** → **Security (and privacy)** → **More security settings** → **Encryption & credentials** → **Install a certificate** → **CA certificate**, confirm with **Install anyway**, and choose the downloaded file. The wording differs a little between makes.

If the installer said port 80 is in use:

- Another program on the server, such as IIS, already uses port 80, so `http://…/ca.crt` is not available. The server still works normally over https.
- Copy the file `C:\SESLabs\SESLabs-certificate.crt` from the server to each device with a USB stick or a shared folder.
- On Windows, import it with `Import-Certificate -FilePath <the file> -CertStoreLocation Cert:\LocalMachine\Root` (PowerShell as administrator).
- On Android, choose the copied file in the *Install a certificate* step.

> **Check:** open `https://<server-address>/pos/`. It opens with a padlock and no warning.

Why this is safe: the certificate is made by your own server, for your own server. Its secret key never leaves the server and is readable by administrators only.

> **Note:**

- The certificate is valid for the server's address and computer name at the time of installation. If you ever give the server a different address, run the installer again, then repeat this step on each device.
- A server with an internet name uses a public Let's Encrypt certificate instead, which every device already trusts. Skip this section.
- Live updates (instant price and stock changes on tills) are set up by the installer and need nothing more.

4.10 Next steps

- Activate the licence: **Chapter 6**.
- First-time set-up in the back office: **Chapter 7**.
- Set up the tills: **Chapter 9**.
- Set up backups **before** you go live: **Chapter 12**.

5. Installing the server on Ubuntu Linux (setups B, C, D and branch servers)

Ubuntu is the recommended system for a head-office, central or cloud server, and for any shop server with more than four devices. It runs for months without restarts and handles many tills at once.

This chapter assumes **Ubuntu Server 24.04 LTS**, freshly installed, on a cloud server (VPS) or a computer at your premises. You run every command as a user that may use `sudo` (the cloud provider's default user, often `ubuntu` or `root`).

5.0 The quick ways (recommended)

A. The install script — does everything in 5.2–5.9 on a plain Ubuntu server:

1. Do 5.1 (get the server, connect to it). Copy the release ZIP and `install.sh` (both from SyncLytics) to the server, e.g. with WinSCP.
2. Run it. Add your own domain and e-mail if the server has an internet name (setup D); leave both out for a server used only inside your network:

   ```
   sudo bash install.sh --zip ~/ses-labs-<customer>-<build>.zip --domain erp.yourcompany.com
   --email you@yourcompany.com
   ```

   For a branch server add `--role branch --node-id B01`.
3. Write down the **backup passphrase** it shows, then answer its questions to create your company and the owner.

The server is then reached over **https** only:

- **With a domain:** a free Let's Encrypt certificate is fetched and renewed automatically.
- **Without one:** the server uses its own certificate for its network address and name. Each till, tablet and PC trusts it once, as in **4.9**, from `http://<server-address>/ca.crt`.

The firewall is switched on with only SSH, 80 and 443 open (443 also for HTTP/3).

Running it again with a newer ZIP updates the software and keeps your data, `.env` and keys. An older installation that used nginx is moved to the new web server (Caddy) automatically.

A shared server for small clients (SyncLytics staff). The standard is **one server per client**. For small businesses that are just starting, several clients can share one server, each as its **own separate copy** of the system on its own internet name:

```
sudo bash install.sh --zip <release>.zip --instance kofi-pharmacy --domain kofi.synclytics.examp
le --email you@example.com
```

- `--instance` is a short name for the client: lowercase letters, digits and dashes.
- Each copy gets its own:

    - folder (`/var/www/seslabs-kofi-pharmacy`) and database (`erp_kofi_pharmacy`);
    - system user, so one copy cannot read another's files, backups or passwords;
    - PHP workers, which stop when idle so a quiet client uses little memory;
    - live-updates port, backups, licence (its own installation code) and update timing.
- Give each copy its own **DNS A record** pointing to the same server (5.1), and its own `--domain`.
- **Update one client:** run the same command with the new ZIP and the same `--instance`. It backs that copy up first and leaves the others alone.
- **How many:** a server with 12 GB of memory (for example Contabo Cloud VPS 6) holds about **3 small clients** (one or two shops each). Move a client to a server of its own when it grows, or when *System → Health → Slow pages* starts filling up: back it up, install it on the new server, restore the backup there (Chapter 12.5), and point its internet name to the new server.
- **Remove a client's copy** (after a final backup and when the contract has ended):

  ```
  sudo systemctl disable --now seslabs-<name>-reverb
  sudo rm /etc/cron.d/seslabs-<name> /etc/caddy/sites/seslabs-<name>.caddy /etc/php/8.4/fpm/pool.d/seslabs-<name>.conf
  sudo systemctl restart php8.4-fpm caddy
  ```

  Then delete its folder and database (`sudo -u postgres dropdb erp_<name>`) only once you are sure the backup is safe elsewhere.

Why separate copies, not one copy holding several companies: one copy shares one licence, one backup and one update for everyone in it. Restoring one client's backup would roll back the others, and a fault in one would reach them all.

B. Docker (if your host prefers containers):

1. Install Docker (`curl -fsSL https://get.docker.com | sudo sh`).
2. In the release folder: `cp installer/docker/.env.docker.example installer/docker/.env` and edit it: `APP_URL`, a long random `DB_PASSWORD`, a `BACKUP_PASSPHRASE` (12+ characters, written down), and e-mail settings (Chapter 16).
3. `sudo docker compose -f installer/docker/docker-compose.yml up -d --build`
4. The first start prints an `APP_KEY` and `ERP_AUDIT_KEY` in `docker compose logs app`: copy both into `installer/docker/.env` straight away — they must never change.
5. Create your company: `sudo docker compose -f installer/docker/docker-compose.yml exec app php artisan erp:install`.
6. Put HTTPS in front (your host's load balancer, or Caddy / Traefik).

Why Docker: the whole server (PHP, web server, scheduler, live updates) is one image that runs the same everywhere; the database and your files live in Docker volumes that survive updates (`up -d --build` with a new release).

The rest of this chapter is the manual way.

5.1 Get the server and connect to it

Cloud server (setup D)

1. Rent a VPS from any provider (for example a provider with a data centre in or near West Africa or Europe; with Contabo, the *Cloud VPS 6* with 100 GB NVMe storage in the *European Union* region, with Auto Backup). Choose **Ubuntu 24.04**, at least 2 vCPU, 4 GB RAM, 80 GB SSD; see the sizing table in Chapter 3. Note the server's **public IP address** and the password or key you are given.
2. Buy or use a **domain name** and create a **DNS A record** such as `erp.yourshop.com` pointing to that IP address (in your domain provider's control panel). It can take up to an hour to work. **Why:** tills, browsers and the HTTPS certificate need a name, not just a number.

Server at your premises (setups B, C, branch servers)

1. Install Ubuntu Server 24.04 from a USB stick (`https://ubuntu.com/download/server`). During installation choose **Install OpenSSH server**.
2. Reserve a fixed IP address for it in your router (see 4.1, step 4).

Connect from your Windows PC

Open PowerShell on your own PC and type (replace the user and address):

```
ssh ubuntu@<server-ip-address>
```

Type `yes` the first time, then the password. You are now typing commands **on the server**.

> **Tip:** to copy files to the server with a mouse, install **WinSCP** (`https://winscp.net`) on your PC and connect with the same address, user and password.

5.2 Update the system and set the time

```
sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone Africa/Accra
sudo timedatectl set-ntp true
```

> **Why:** security fixes first; and the correct time is essential for receipts, shifts and sync.

> **Check:** `timedatectl` shows *System clock synchronized: yes*.

5.3 Install PostgreSQL 17

```
sudo apt install -y postgresql-common
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh -y
sudo apt install -y postgresql-17
```

> **Why:** Ubuntu 24.04 ships an older PostgreSQL; the official PostgreSQL repository provides version 17, which the software is tested with.

Create the database and its user (replace `<db-password>` with a long random password and write it down):

```
sudo -u postgres psql -c "CREATE ROLE erp LOGIN PASSWORD '<db-password>';"
sudo -u postgres psql -c "CREATE DATABASE erp OWNER erp ENCODING 'UTF8';"
```

> **Check:** the commands answer `CREATE ROLE` and `CREATE DATABASE`.

5.4 Install PHP 8.4, Caddy, Composer and Node.js

```
sudo add-apt-repository -y ppa:ondrej/php
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/gpg.key | sudo gpg --dearmor -o /usr/sha
re/keyrings/caddy-stable-archive-keyring.gpg
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt | sudo tee /etc/apt/sourc
es.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy unzip git php8.4-fpm php8.4-cli php8.4-pgsql php8.4-intl php8.4-bcmath
php8.4-mbstring php8.4-xml php8.4-curl php8.4-zip php8.4-gd
curl -sS https://getcomposer.org/installer | sudo php -- --install-dir=/usr/local/bin --filename
=composer
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
```

> **Why:**

- **PHP** runs the software, and **PHP-FPM** lets several requests run at the same time.
- **Caddy** is the web server that receives the tills' and browsers' requests. It gets and renews https certificates by itself. It comes from its makers' own signed package source.
- **Composer** and **Node.js** are used during installation.

> **Check:** `php --version` shows 8.4, `caddy version` shows v2, `composer --version` shows 2.x, `node --version` shows v22 or later, and `php -m` lists `pdo_pgsql`, `intl`, `bcmath` and `sodium`.

5.5 Put the software in place

1. Copy your release ZIP to the server (with WinSCP, into your home folder).
2. Unpack it into `/var/www/seslabs`:

```
sudo mkdir -p /var/www/seslabs
sudo unzip ~/ses-labs-*.zip -d /var/www/seslabs
sudo chown -R $USER:www-data /var/www/seslabs
cd /var/www/seslabs/server
```

> **Check:** `ls` shows `artisan`, `app`, `public` and other folders.

5.6 Configure the server

```
composer install --no-dev --optimize-autoloader
cp .env.example .env
nano .env
```

In the editor, change these lines (move with the arrow keys; save with **Ctrl+O**, **Enter**, exit with **Ctrl+X**):

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://erp.yourshop.com
SESSION_SECURE_COOKIE=true
DB_PASSWORD=<db-password>
ERP_NODE_ROLE=hq
ERP_NODE_ID=HQ
```

- For a server reached by its address on your network, use `APP_URL=https://<server-ip-address>`. It is still https: the server makes its own certificate (5.7).
- For a **branch server**: `ERP_NODE_ROLE=branch` and its id (e.g. `B01`), see Chapter 10.

Then:

```
php artisan key:generate
php artisan erp:audit-key
php artisan migrate --seed --force
php artisan erp:install
npm install && npm run build
php artisan optimize
sudo chown -R www-data:www-data storage bootstrap/cache
sudo chmod 640 .env && sudo chown $USER:www-data .env
```

The reasons for each command are the same as in Chapter 4, section 4.6. The last two lines let the web server write its logs and cache, and stop other users of the machine from reading `.env`.

> **Note:** Skip `php artisan erp:install` on a branch server (Chapter 10).

If you received the source code instead of a release, also build the till app:

```
cd /var/www/seslabs/pos && npm install && npm run build:server && cd ../server
```

5.7 Configure the web server (Caddy)

First let Caddy reach PHP:

```
sudo usermod -aG www-data caddy
```

> **Why:** PHP-FPM listens on a socket that only the `www-data` group may use.

Replace Caddy's settings file (the install script instead writes a short main file that imports one file per copy from `/etc/caddy/sites/`; the manual way keeps everything in one file):

```
sudo nano /etc/caddy/Caddyfile
```

Delete what is in it and paste the text below. This example is for a server on your network with address `192.168.1.20` and name `shopserver`; change both everywhere.

For a server with a domain (setup D):

- write the domain instead of `192.168.1.20, shopserver`;
- remove the whole `http://…` block and the `tls internal` line;
- add `email you@yourshop.com` under `admin off`.

```
{
	admin off
	skip_install_trust
	pki {
		ca local {
			name "SES Labs on shopserver"
			root_cn "SES Labs shopserver Root"
			intermediate_cn "SES Labs shopserver Intermediate"
		}
	}
}

http://192.168.1.20, http://shopserver {
	handle /ca.crt {
		root * /var/lib/caddy/.local/share/caddy/pki/authorities/local
		rewrite * /root.crt
		header Content-Type application/x-x509-ca-cert
		file_server
	}
	handle {
		redir https://{host}{uri} permanent
	}
}

192.168.1.20, shopserver {
	tls internal
	root * /var/www/seslabs/server/public
	encode zstd gzip
	request_body {
		max_size 25MB
	}
	header -Server
	@hidden {
		path_regexp /\.
		not path /.well-known/*
	}
	handle @hidden {
		respond 404
	}
	@live path /app/* /apps/*
	handle @live {
		reverse_proxy 127.0.0.1:8080
	}
	handle /pos/* {
		header /pos/sw.js Cache-Control no-cache
		try_files {path} /pos/index.html
```

```
                file_server
        }
        handle {
                php_fastcgi unix//run/php/php8.4-fpm.sock {
                        read_timeout 300s
                }
                file_server
        }
}
```

Check and start it:

```
sudo -u caddy caddy validate --config /etc/caddy/Caddyfile --adapter caddyfile
sudo systemctl restart php8.4-fpm caddy
```

> **Why:**

- *Plain http* only hands out the server's own certificate (so devices can trust it, 4.9) and sends everything else to https.
- *Hidden files* such as `.env` are never served.
- *Live updates* use the same https address.
- *The till app* falls back to its own start page, so it works offline.
- *Everything else* goes to PHP.
- With a domain, Caddy fetches a Let's Encrypt certificate and renews it by itself.
- `admin off` closes Caddy's local control port. After a change to this file, use `systemctl restart caddy` (a *reload* needs that port).

Server on your network: trust the certificate on the server itself, and keep a copy for devices:

```
sudo cp /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt /usr/local/share/ca-cer
tificates/seslabs.crt
sudo update-ca-certificates
```

> **Check:** `validate` ends with *Valid configuration*. `curl https://192.168.1.20/up` (or your domain) answers without a certificate error. On another device that trusts the server (4.9), `https://192.168.1.20/admin` shows the sign-in page.

5.8 Firewall

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 443/udp
sudo ufw enable
```

> **Why:** the firewall blocks everything except remote administration (SSH) and the web server's ports. `443/udp` is HTTP/3, which browsers use when they can: it needs fewer round trips and copes better with a slow or unsteady internet line, which matters for a server far from the shop. The database, PHP and live updates are reachable from the server itself only. If SSH on your server uses another port than 22, allow that port instead of `OpenSSH` **before** `ufw enable`, or you lock yourself out.

Why HTTPS always:

- HTTPS encrypts everything between tills and the server: PINs, sales and customer details.
- Browsers only let tills enrol, install as apps and work offline from https addresses (or from `localhost`).

> **Warning:** Never run a server that is reachable from the internet without HTTPS.

5.9 Background jobs

The scheduler runs the routine jobs. Add it to the system's timer (cron):

```
echo "* * * * * www-data cd /var/www/seslabs/server && php artisan schedule:run >> /dev/null 2>&1" | sudo tee /etc/cron.d/seslabs
```

> **Why:** every minute it sends changes to online stores, syncs branch servers, posts to the accounts, sends reminders and, once a day, checks the licence.

Live updates: prices and stock reach tills within seconds, instead of within 60 seconds.

1. In `.env` set (three random values of your own, letters and digits, at least 20 characters each; your address or domain as `REVERB_HOST`):

   ```
   REVERB_APP_ID=<random-1>
   REVERB_APP_KEY=<random-2>
   REVERB_APP_SECRET=<random-3>
   REVERB_HOST=192.168.1.20
   REVERB_PORT=443
   REVERB_SCHEME=https
   REVERB_PUBLISH_HOST=127.0.0.1
   REVERB_PUBLISH_PORT=8080
   REVERB_PUBLISH_SCHEME=http
   ```

   > **Why:** tills connect through the https address. The server itself sends to the live-updates service directly on the same machine.

2. Create the service:

```
sudo nano /etc/systemd/system/seslabs-reverb.service
```

```
[Unit]
Description=SES Labs live updates
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/seslabs/server
ExecStart=/usr/bin/php artisan reverb:start --host=127.0.0.1 --port=8080
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

3. Start it:

```
cd /var/www/seslabs/server && php artisan optimize
sudo systemctl daemon-reload
sudo systemctl enable --now seslabs-reverb
```

> **Check:** `systemctl status seslabs-reverb` shows *active (running)*.

5.10 Next steps

Continue with the licence (**Chapter 6**), the first-time set-up (**Chapter 7**), devices (**Chapter 9**) and backups (**Chapter 12**).

6. Activating the licence

Every installation needs a **licence key from SyncLytics** (hotline 024 563 6889, info@synclyticsenterprisesystems.com). Until it is activated, only the Licence page works.

6.1 What the licence controls

- How many **branches**, **tills and devices** and **active users** you may have.
- Which **industry modules** you may switch on (Grocery, Pharmacy, Restaurant, Electronics, Fashion, Services…).
- Whether **Finance** (accounts) and **HR & payroll** are included.
- Until when it is valid.

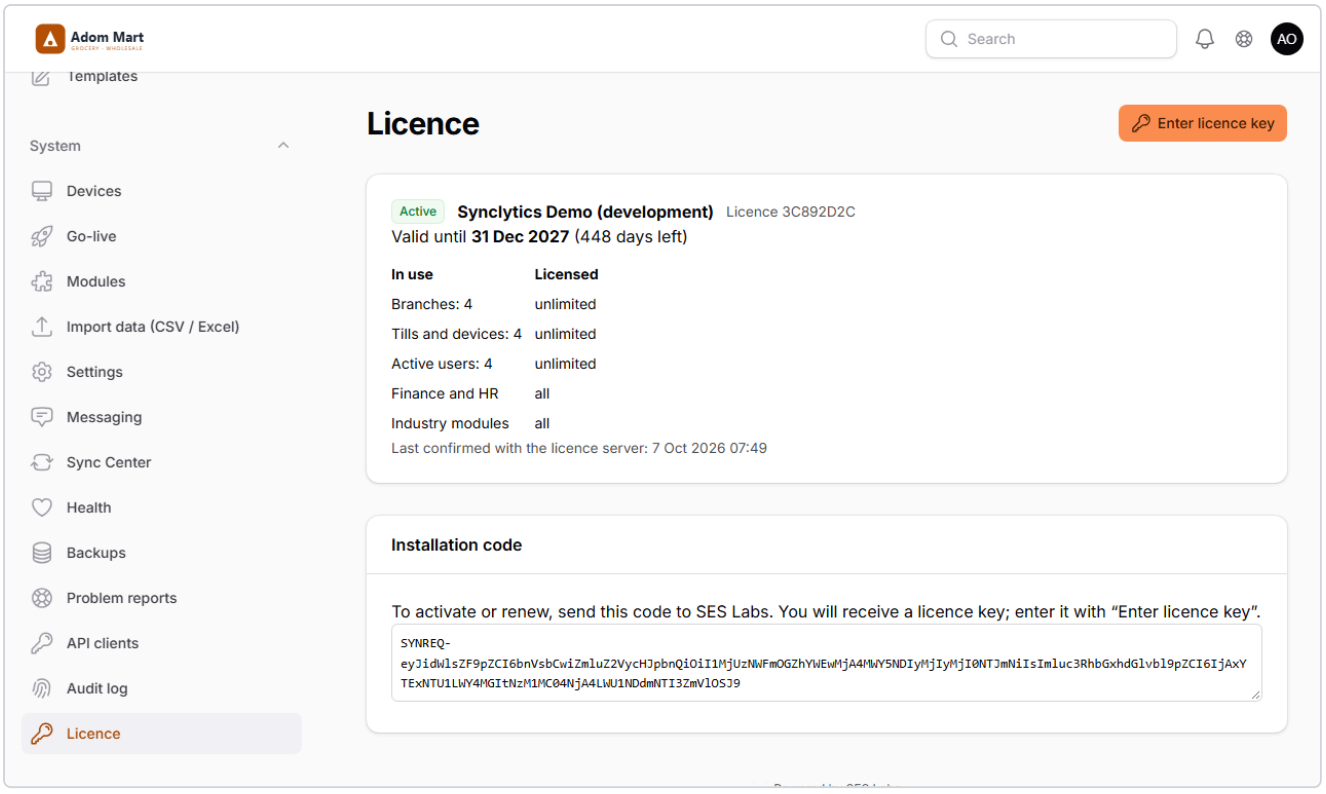
Parts that are not licensed are hidden from the menus.

6.2 Activate

1. Open the back office at `https://<server-address>/admin` (Windows) or `https://<your-domain>/admin` (Ubuntu) and sign in as the **owner** (Chapter 7, steps 1–2 explain the first sign-in and two-factor set-up).
2. You are taken to **System** → **Licence**. It shows the **installation code**, which starts with `SYNREQ-`.
3. Send that code to SyncLytics: copy and paste it into an e-mail to info@synclyticsenterprisesystems.com, or a WhatsApp message to the hotline 024 563 6889. The Licence page shows these contacts too. Tell them which modules and how many branches, devices and users you need.
4. You receive a **licence key** starting with `SYN1-`. On the Licence page click **Enter licence key**, paste it and confirm.

> **Why:** the key is signed by SyncLytics and checked on your server without the internet. It is tied to this installation, so a copied server or database asks to be activated again.

> **Check:** the Licence page shows **Active**, your company name, the expiry date and the limits:



6.3 Daily check-in

Head office contacts the SyncLytics licence server once a day (03:15, by the scheduler). It collects renewals automatically, so you normally never paste a key again.

- If the server cannot reach the internet for **30 days**, a yellow banner appears; after another **14 days** the software becomes read-only until it checks in.
- **Branch servers** receive the licence from head office; they do not check in themselves.

6.4 Licence states

State	When	What works
Not activated	New installation	Only the Licence page
Active	Normal	Everything you bought
Attention (yellow banner)	30 days before expiry; no check-in for 30 days; software moved to another server	Everything
Grace	Up to 14 days after expiry	Everything, with a banner
Read-only	After the grace period; licence withdrawn; no check-in for 44 days; moved and not re-activated within 14 days; software modified	You can **view** everything, but nothing can be changed, sold or printed. Tills keep their unsent sales until the licence is renewed. **Nothing is deleted.**

6.5 Renewing, adding branches or modules

Ask SyncLytics for a new key. It reaches head office at the next daily check-in, or paste it on the Licence page straight away.

6.6 Moving to a new server

The licence is tied to the server it was activated on. After moving (Chapter 12), the Licence page shows *Attention: moved*. Send the new installation code to SyncLytics and enter the new key within **14 days**.

> **Warning:** Do not try to copy or alter the software to get around the licence. Modified copies stop working and are reported at check-in; printed documents carry hidden references to the licence they were made with.

7. First-time set-up in the back office

Do these steps once, at head office, in this order. Allow a day for a single shop, longer for a chain (most of the time goes into products and prices).

> **Tip:** Practise first on a **test installation** with made-up products: `php artisan erp:demo-catalog --as=<owner-email>` fills it with sample products and stock. Never run that command on your live system.

Step 1 — Sign in for the first time

1. Open the back office: `https://<server-address>/admin` (Windows server) or `https://<your-domain>/admin` (Ubuntu).
2. Sign in with the owner's **email and password** (from `erp:install`).

Step 2 — Set up two-factor sign-in

You are asked to set up an **authenticator app**:

1. Install an authenticator app on your phone (Google Authenticator, Microsoft Authenticator or similar).
2. In the app choose *Add account* → *Scan a QR code* and scan the code on the screen.
3. Type the 6-digit code the app shows.
4. **Save the recovery codes** shown next: print them or write them down and keep them somewhere safe, **not** on the same phone.

> **Why:** a stolen password alone is then not enough to get in. Everyone from security level L3 (supervisors and above) must use two-factor sign-in.

> **Tip:** the back office can be **Light**, **Dark** or follow your device (**System**): open your profile menu (top right) and choose the sun, moon or screen icon. Each person's choice is kept in their browser.

> **Note:** Later you can add a **passkey** (fingerprint or face) under your profile. Passkeys work only on an HTTPS address or `localhost`.

Step 3 — Activate the licence

See **Chapter 6**.

Step 4 — The setup wizard

The wizard opens by itself the first time.

1. **Deployment** — how the system runs: *Standalone single PC* (setup A/B), *Central on-premise server* (setup C), or *Cloud HQ* (setups D/E). This records your starting layout; you can grow later.

2. **Company** — trading name, country, base currency and time zone. Optionally the **legal entity**: registered name, **TIN** and **VAT number**, needed for VAT invoices (you can add it later under *Company*).
3. **Your brand** (optional) — the company **logo**, a **square icon** and the **brand colour** (leave the colour empty and it is taken from the logo). You can do this later under *Company → Branding* (below).
4. **Branches & modules** — one row per branch:
    - **Code** (1–6 letters/digits, e.g. `HQ`, `KSI`). It appears in every receipt number and **cannot be changed later**.
    - **Name**, the number of **tills**, and whether it is the **head office** (exactly one).
    - **Modules** for that branch (Grocery, Pharmacy, Restaurant, Electronics, Fashion, Services, Online Storefront…). You can change these later (Chapter 8).
5. Click **Finish setup**.

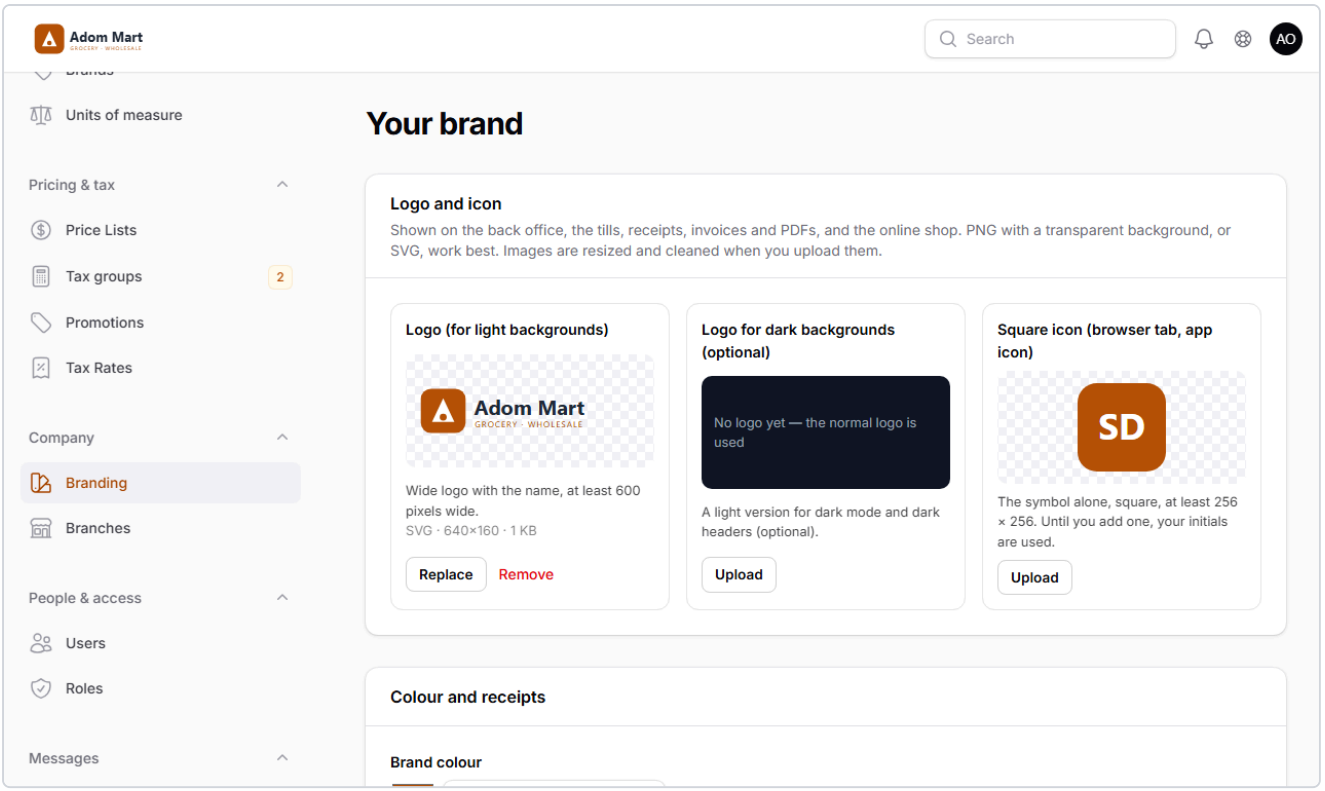
Why: each branch automatically gets a main warehouse and the tills you asked for. Receipt numbers look like `HQ-T01A-000123`: branch, till, device series and number.

Check: *Company → Branches* lists your branches with their warehouses and tills.

Your brand: logo, icon and colour

Company → Branding. Your brand is what staff and customers see: the back office's top bar, the till, printed receipts, invoices, statements, PDF exports and your online shop.

1. **Logo** — your logo with the name, on a transparent background (PNG) or as SVG, at least 600 pixels wide. Choose **Upload**.
2. **Logo for dark backgrounds** (optional) — a light version for dark mode and dark headers. Without it, the normal logo is used.
3. **Square icon** — the symbol alone, at least 256 × 256 pixels. It is the browser-tab icon and the small icon on the till. Until you add one, your **initials on your colour** are used.
4. **Brand colour** — pick it, type it (`#0F766E`), or press **Use … from your logo**. If the colour is too light to read on white, the page says so: choose a darker shade.
5. **Print the logo on till receipts** — receipt printers print it in black and white (it is converted for them).
6. Press **Save colour and receipts**, and look at the **Preview**: the till accent, a receipt and an invoice.



Why: customers trust a receipt or invoice that carries the shop's own logo, and staff recognise their own system. The images are checked and made smaller when you upload them, and they reach branch servers and backups like the rest of your data.

Note: a small line *"Powered by SES Labs"* stays at the bottom of the back office, receipts and documents. A **white-label licence** removes it (ask SyncLytics).

Step 4b — Industries (if you run more than one kind of business)

Each branch runs one industry (Pharmacy, Grocery, …), switched on per branch in the wizard or under Company → Modules. Head office can stay company-level. See **Chapter 8.0** for the industry switcher, separate catalogues and industry managers.

Books: by default each industry gets **its own company and books**. Fill in each company's registered name and TIN under **Company** → **Legal entities** before the first sale, so receipts print them. For one set of books instead, switch off **System** → **Settings** → **Finance** → **Separate books for each industry** now, before anything is posted (Chapter 8.0, *Accounts by industry*).

Step 5 — People and access

For each staff member:

1. **People & access** → **Users** → **New user**: name, username, email (for back-office users), and a **Security level (ceiling)** (the highest level of what they can ever be given — see Chapter 18).

2. On the next screen, **Grant access**: choose a **role** (Cashier, Supervisor, Branch Manager, Storekeeper, Accountant, Pharmacist, Waiter, Stylist / Therapist…) and **where it applies** (a till, a branch, a warehouse or the whole company).
3. For anyone who sells or clocks in on a till: **Set POS PIN** (4–8 digits; simple PINs like 1234 are refused).

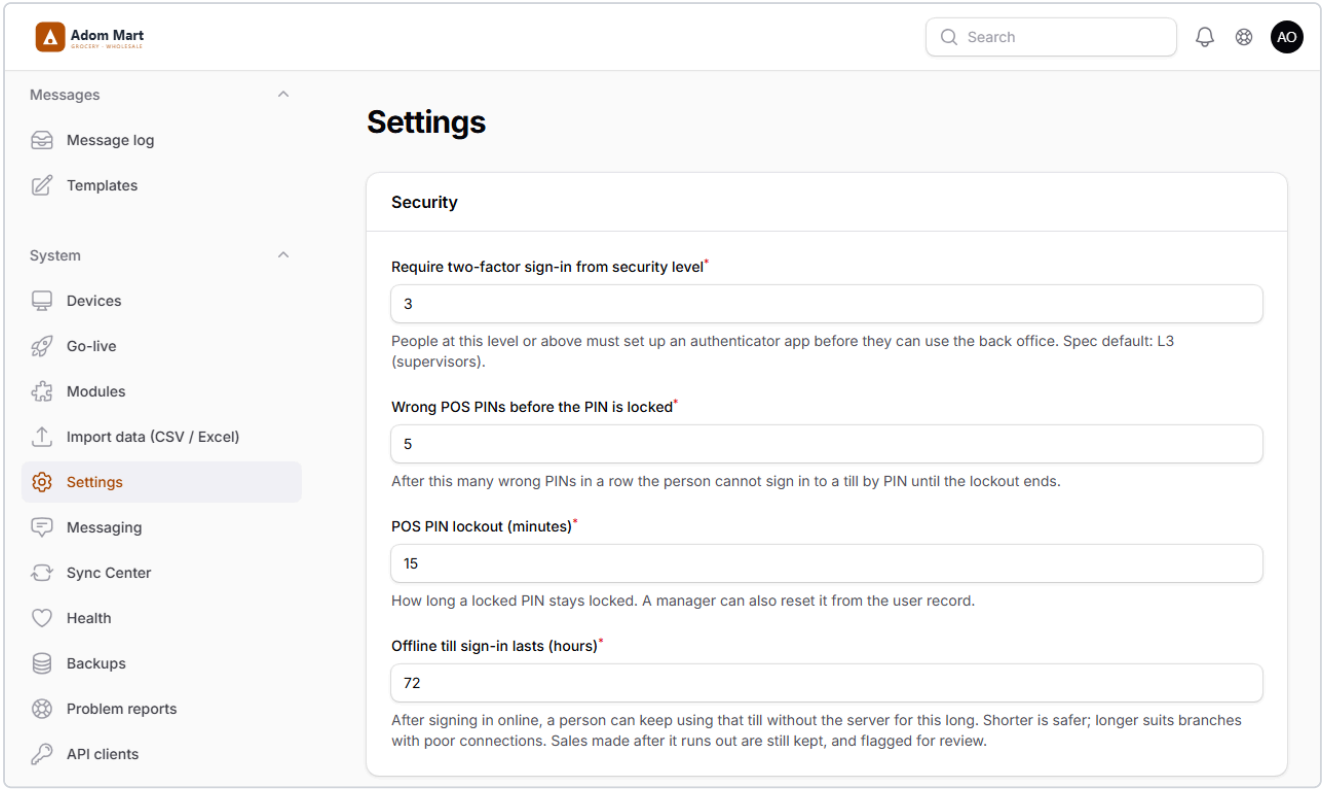
> **Why:** people see and do only what their role allows, only where it applies. A cashier with access to *Till 1* cannot sell on *Till 2* or open the back office.

> **Tip:** For temporary cover (e.g. a supervisor on leave), grant access **with an end date and a reason**: it stops by itself.

> **Warning:** Do not share one account between several people. Every sale, refund and change is recorded under the person who signed in.

Step 6 — Settings

System → **Settings** — review each group. The most important:



Setting	Why it matters
Require two-factor sign-in from security level	Default L3. Lower it for more security.
Wrong POS PINs before the PIN is locked / POS PIN lockout (minutes)	Stops PIN guessing at the till.

Setting	Why it matters
Offline till sign-in lasts (hours)	How long a cashier can sign in on a till without the server (default 72). Shorter is safer, longer helps during long outages.
Stop sales when the system shows no stock	Off by default (tills can sell items the system thinks are out of stock, and you count them later).
Show stock left on the till while selling	Off by default. On: each item in the sale, and in the till's search list, shows how many are left at that till's store (e.g. *14 TAB left*), counting what is already in the sale. Useful where cashiers should tell customers what is available. Tills pick it up at their next sign-in.
Message at the bottom of receipts	Your returns policy or a thank-you line.
Returns accepted within (days, with receipt)	How long after a sale a return with the receipt is accepted.
Cash control with vaults	Off: whoever closes a till counts the drawer. On: tellers, a Cashier and an Income Controller check the cash at every step (Step 14).

Module settings (Grocery, Pharmacy, Restaurant, Fashion, Services) are explained in Chapter 8.

Step 7 — Tax set-up (with your accountant)

Pricing & tax → **Tax groups** and **Tax rates.** The starter Ghana set-up (VAT 15 %, NHIL 2.5 %, GETFund 2.5 %, all on the price before tax) is marked **Not verified**.

1. Ask a qualified accountant to confirm the rates and the method against current GRA rules.
2. They (or you, on their advice) adjust the rates, then use **Record accountant check** on each group.

> **Warning:** Do not go live until the tax groups show the accountant's check. Wrong tax set-up means wrong receipts and wrong tax returns.

Step 8 — Price lists

Pricing & tax → **Price lists.** You start with **Retail** (default) and **Wholesale**, both including tax. Add lists for special customers if needed. Tick *Cashier can switch to it at the till* only for lists cashiers may use.

> **Note:** whether a list's prices include tax cannot change once it has prices.

Step 9 — Products

1. **Catalog** → **Categories** and **Brands**: create the groups you want for reports.
2. **Catalog** → **Units of measure**: check the units (PC, CTN, KG with 3 decimals…).
3. **Catalog** → **Products** → **New product** for each item:
    - **Name**, category, brand, **tax group**, **base unit** (the unit stock is counted in, usually PC). The **SKU** (product code) is given by the system when you save, so no two products can share one: an industry's products get its code and a number (`PHARMA-000123`), products shared by every

industry a plain number (`100123`). It does not change, except when the product is moved to another industry (it then takes that industry's series).
    - **Barcodes** — scan them into the field.
    - **Other units** — e.g. a carton of 24 with its own barcode.
    - **Prices** — one row per list and unit. Add rows with *From qty* for bulk prices or with dates for promotions.
    - Industry details (medicine, serial numbers, sizes and colours, services) appear when the module is on — see Chapter 8.

> **Why:** products are never deleted (old receipts refer to them). Untick *For sale* instead.

> **Tip:** with many products, use **Import products (CSV)** instead of typing them one by one: download the template, fill it in Excel (name, category, barcodes, a price column per list; leave SKU empty for new products) and import it. Chapter 14.2 explains how.

Step 10 — Opening stock

Stock → **Stock on hand** → **Opening stock / adjustment**, choosing *Opening balance*, for each product and warehouse: the quantity you counted and the **cost per unit**.

> **Why:** the cost is used for the value of your stock and the cost of goods sold in the accounts.

> **Tip:** count on a day you are closed, enter the counts, then start selling. For many products use **System** → **Import data (CSV / Excel)** → **Opening stock** (warehouse, SKU, quantity, unit cost per row).

Step 11 — Customers and suppliers

- **Sales** → **Customers**: customers who buy on credit or collect loyalty points. Credit limits and payment terms are set at head office. Customers who owe you money get an **opening balance**.
- **Purchasing** → **Suppliers**: suppliers you buy from, with an **opening balance** if you owe them.
- Both can be imported from CSV files (**Import customers / suppliers (CSV)** on their lists).

Step 12 — Finance and HR (if licensed)

- **Finance** → **Chart of accounts:** the Ghana SME template is already installed. Your accountant can rename and add accounts.
- **Opening balances** on the go-live date: customers and suppliers (step 11), stock (step 10), and everything else (bank, cash, MoMo, assets, loans, capital) under **Finance** → **Journals** → **Opening balances**. The account *Opening balances (to clear)* must end at zero.
- **HR & payroll** → **Employees:** add staff with their pay and link each to their sign-in. Check **Payroll profile** (PAYE and SSNIT rules) with your accountant.

Step 13 — Set up the devices

Register and enrol the tills, handhelds and kitchen screens: **Chapter 9**.

Step 14 — Cash control: vaults, Cashier and Income Controller (optional)

A shop with several tellers can check its cash at every step, as banks do.

- **Teller:** sells at a till and counts the tray at the end of the shift.
- **Cashier:** gives each teller a cash tray with a float (a **vault**), receives it back and counts it again.
- **Income Controller** (accounts): receives the day from the Cashier, counts it again, decides any difference and records the bank deposit.

Nobody sees the figure they are checking against before they have counted: these are **blind counts**.

> **Why:** a difference is found at the step where it happened and is tied to the person responsible. Cash never moves without a second person checking it.

1. **Switch it on: System → Settings → Tills and cash → Cash control with vaults**.
2. **Shifts: Cash → Shifts and vaults** lists the shifts, *Morning* (07:00–14:00) and *Afternoon* (14:00–21:00). Change the times, or **Add a shift** (e.g. *Evening*). Each till gets one vault per shift, e.g. *T01 · Morning*.
3. **Standard floats:** on the same page, **Edit** each vault and enter the float the Cashier usually puts in it. It is suggested every day, and the Cashier can change it.
4. **People** (**People & access → Users**, Step 5):
    - give each till person the **Teller** role, at their till or branch;
    - give one or more people the **Cashier** role, at their branch;
    - give the **Income Controller** role (security level 4) to someone in accounts, usually company-wide.

    The Cashier and the Income Controller do not sell, so they must not also have the Teller role.
5. **Every day:**
    1. The Cashier opens **Cash → Cash desk** and clicks **Issue float** on each vault, choosing the teller.
    2. The teller signs in at that till, ticks *I have counted the float* and opens the shift.
    3. At the end of the shift the teller counts the tray note by note and closes the till, then hands the tray to the Cashier.
    4. The Cashier clicks **Receive and count** and counts again. If the two counts differ, they are asked to count once more; the second count stands.
    5. At the end of the day the Cashier clicks **Hand over to the income controller**.
    6. The Income Controller opens **Cash → Income control**:
        - **Receive and count** the bundle;
        - **Decide** each difference: *cash over/short*, or *charged to the staff member*, recovered from pay through HR & payroll;
        - **Record deposit** with the bank slip number.

> **Check:** on a test day, issue a float, sell, close, receive, hand over and receive again. *Income control* then shows the day's reconciliation (system, teller, cashier, income controller), and *Finance → Trial balance* shows *Cash on hand – tills* and *Cash with cashiers* back at zero.

The full guides for each person are in the training material (Chapter 17.7).

Step 15 — Test before going live

Do a full test day with made-up sales:

1. Open a shift on a till, sell with cash, MoMo and card, print receipts, do a return, close the shift.
2. Unplug the till's network cable (or switch off Wi-Fi), sell two items, reconnect, and watch the sales arrive (*Sales → Till sales*).
3. Check *Sales → Review queue*, *Stock → Stock on hand* and, if licensed, *Finance → Reports → Trial balance*.
4. Take a backup and **restore it on another PC** (Chapter 12).

> **Check:** when all four work, remove the test data the easy way — make a fresh installation for go-live, or ask SyncLytics for help — and enter your real opening figures.

8. Installing and setting up modules

Modules add the features of a type of business. They are **already part of the software** — "installing" a module means three things:

1. your **licence** includes it (Chapter 6);
2. it is **switched on for each branch** that needs it;
3. its **set-up checklist** below is done.

A branch runs **one industry module** (Pharmacy, Grocery, Restaurant, …), plus any company-wide features and add-ons such as Sales & Distribution. A supermarket with a pharmacy counter is set up as two branches at the same address, so each keeps its own stock, staff and figures.

8.0 Industries

When a company runs more than one kind of business, each kind is an **industry**: for example *Pharmacy* with the branches Pharmacy Osu and Pharmacy Tema, and *Grocery* with Grocery Spintex.

> **Why:** each industry has its own staff, products and figures. Separating them keeps a pharmacist's screens free of grocery items, and gives the owners each industry's results on its own as well as the whole company's.

- **An industry appears by itself** when its module is switched on for a branch (8.1). The branch then belongs to it. *Company → Industries* lists the industries, and new ones can be added there.
- **Head office or a central warehouse** can belong to no industry (*company-level*). Only people with company-wide access see it.
- **The industry switcher** at the top of the back office (*All industries*, *Pharmacy*, *Grocery*, …) is shown to people who can see more than one industry.
    - With an industry chosen, every screen shows only that industry: branches, stock, prices, purchasing, sales, reports, and its own dashboard. The pharmacy dashboard shows prescriptions and expiring batches.
    - With *All industries*, the dashboard shows each industry's sales side by side.
- **Each industry has its own catalogue.**
    - When you create a product or price list, choose its industry; the industry chosen at the top is filled in for you.
    - That industry's tills sell only its products.
    - Products created before industries are *shared* (sold everywhere) until you give them an industry (Catalog → Products → edit → Industry).
- **Staff:**
    - give a person a role **at their branch** (People & access → Users → Grant access → Applies to: *Branch*). A pharmacist at Pharmacy Osu then sees that pharmacy only;
    - an **industry manager** gets the role with *Applies to: Industry*, and sees every branch of that industry, and nothing of the others.

Only what the business runs is offered. Roles (such as *Waiter* or *Tailor*), settings sections, message templates and device types that belong to an industry appear only where that industry is switched on. A

pharmacy, for example, is never offered a kitchen display.

Menus follow the choice. With an industry chosen, only its own module menus show (for example *Pharmacy* but not *Restaurant*). With *All industries*, every installed industry's menus show. Grocery's menu has **Markdowns** (near-expiry stickers and expired write-offs at the grocery stores) and **Scale items** (the weighed products to load into the scales).

> **Check:** choose *Pharmacy* at the top. *Stock → Stock on hand* shows only the pharmacy branches, and *Catalog → Products* shows only pharmacy (and shared) products. Choose *All industries* to go back.

Stock between branches and industries

- **Sister branches** (Pharmacy Osu and Pharmacy Tema), or head office and a branch: use a normal transfer (*Stock → Transfers*), or let the branch that needs stock ask for it (*Stock → Stock requests → Ask for stock*). The branch asked clicks **Send** (entering 0 for anything it cannot spare), or **Reject** with a reason. Sending creates the transfer; the asking branch receives it as usual.
- **Between two industries** (a grocery branch asking a pharmacy for bottled water) only a stock request works, and it needs two approvals before anything moves:

  1. a manager of the asking branch, other than the person who asked;
  2. a manager of the supplying branch.

  The supplier then clicks **Send** and, for each item, chooses **its own product** that matches (the two catalogues are separate). The goods leave at the supplier's cost and arrive as the asking branch's product, with the same batch numbers and expiry dates.

> **Why:** each industry keeps its own stock and figures. Two approvals mean neither side's stock leaves or arrives without a manager knowing, and moving at cost means neither side makes a profit on the other.

Who: storekeepers, supervisors, pharmacists and managers can ask (*Ask another branch for stock*). Branch, warehouse and general managers can approve (*Approve stock requests between industries*). Sending needs the transfer permission at the supplying branch.

Accounts by industry

Two ways, chosen per installation under **System → Settings → Finance → Separate books for each industry**:

Setting	What it means	Suits
On (the default)	Each industry is its own company with its own books, TIN and VAT return. See *Separate books* below.	Groups whose industries are registered separately, or will be.
Off	**One set of books** (one TIN, one VAT return); each industry is a profit centre.	A single registered company running several kinds of shop.

Change it only at the start of a financial year, because switching moves the branches' future postings to other books. Companies already running when this option arrived were left on *Off*.

With either choice, everything a branch posts is labelled with its industry:

- **Finance → Reports → Profit by industry** shows each industry's income, costs and profit side by side. A *Company* column holds head office and shared costs.
- The **Industry** filter on the other reports (trial balance, profit and loss, balance sheet, account ledger, budget) shows one industry, or *Company* for head office lines. It starts on the industry chosen at the top of the screen.
- **Stock moved between industries** leaves one industry's stock and joins the other's at cost, so neither makes a profit on it.

> **Why:** the owners see what each line of business earns, without the cost and paperwork of separate companies.

Sharing head office costs (optional). By default head office costs (rent, admin salaries, the accountant) stay in the *Company* column.

1. To share them, switch on **System → Settings → Finance → Share head office costs between industries by sales**.
2. At month end open **Profit by industry** with the *To* date in that month, and click **Share head office costs**. The window shows each industry's share of that month's sales.
3. Click **Post the journal**.

One journal moves each cost from *Company* to the industries in proportion to their sales, to the pesewa; the company's total profit does not change. A month can be shared only once. To redo it, reverse that journal (*Finance → Journals*) and share again.

> **Note:** an industry's balance sheet on its own does not balance, because bank accounts, money owed and stock in transit sit partly at company level. Use *All industries* for the company's balance sheet.

Separate books (the default)

When an industry is first switched on, the system creates its company under **Company → Legal entities**, named after it (for example *Demo Ltd – Pharmacy*), and a **head office company** if there is none.

1. **Company → Legal entities:** open each company and enter its **registered name, registration number, TIN, VAT number and address**.
    - Its branches' receipts and invoices print that name and TIN.
    - Their sales go to GRA E-VAT under that TIN.
2. **Head office company:** head office lines, and branches without an industry, are booked there. Only one company can be head office at a time; ticking *Head office company* on another moves it.
3. **Two industries under one registered company:** choose the same company for both under **Company → Industries → Books**. To give an industry no company of its own, choose the head office company.
4. Changing an industry's company is a **start-of-year** step. Balances from before (stock, money owed) stay in the old books; ask your accountant to move them with a journal on the first day.

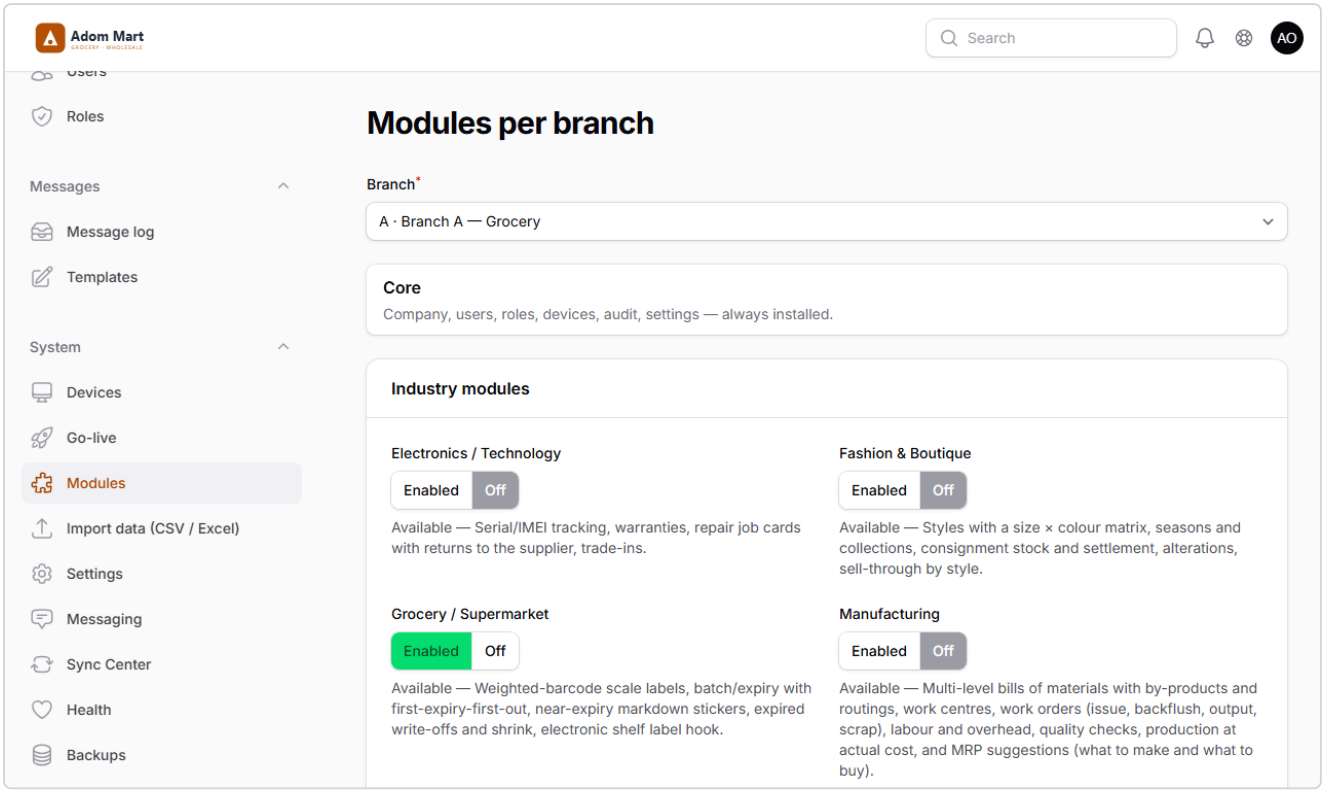
What changes:

- **Every entry balances inside each company.** When one company pays for another, for example head office paying the pharmacy's electricity, the system records what the pharmacy now owes head office on **Due from / to group companies**, in both sets of books. Stock sent between the companies at cost is recorded the same way.
- **Finance → Reports → Books:** choose a company for its own trial balance, profit and loss, balance sheet and ledger. *Whole group (consolidated)* adds them together, and what they owe each other cancels out.
- **VAT return:** each company files its own. Choose it under *Books*.
- **Settling up:** when one company pays the other, enter a journal with each company's bank line (*Company* on lines without a branch). The amount owed clears by itself.
- **Ledger check** (*Finance → Reports → Ledger check*) also proves that each entry balances in every company's books, and that both sides agree on what they owe each other.

> **Why:** a separately registered company must keep its own books and file its own returns. Recording what they owe each other automatically means no entry is ever missed or one-sided.

8.1 Switching a module on or off

1. **System → Modules.**
2. Choose the **Branch**, then for each module choose **Enabled** (or **Off**; a planned module offers **Reserved**).
3. Type the **Reason for change** and click **Save modules**.



> **Why:** modules are per branch so that, for example, only the pharmacy branch shows medicine screens. Every change is recorded in the audit log.

Status shown	Meaning
Available	Released: can be switched on now.
Planned · Phase N	Not built yet. You may **reserve** it for a branch; it switches on by itself when that phase is released. Nothing is shown as working before it exists.

Only modules you have bought are shown. The set-up wizard, this screen and the industry choices list only the modules in your licence, and the industry switcher and dashboards list only industries switched on at a branch.

> **Why:** staff never see, or try to set up, a business type the company has not bought. To add one, ask SyncLytics for a new licence key (Chapter 6); the module then appears here, ready to switch on. If a licence renewal drops a module, its screens are hidden but its records are kept, and they come back if it is added again.

> **Note:** switching a module **off** hides it and stops new work. It **never deletes** data; switching it on again brings everything back.

Who can do it: the owner and the IT Admin (permission *Manage modules*).

> **Check:** after switching a module on, the module's menu appears in the back office (you may need to refresh the page), and its buttons appear on that branch's tills after their next catalog refresh (within 60 seconds, or press *Refresh* on the till).

8.2 Available modules and their checklists

Online Storefront (built-in online shop)

1. Switch it on for the branch that will fulfil online orders.
2. **Online** → **Channels** → **New channel**, type *Built-in online shop*: choose the fulfilling branch, the stock warehouse, the price list and the shop's web address (`/shop/<address>`).
3. Set **safety stock** on products that sell fast in the shop too.
4. Click **Activate**.

> **Why:** safety stock is held back from the shop so walk-in and offline sales do not oversell online.

> **Check:** open `https://<your-domain>/shop/<address>` — your products appear with prices and availability.

Grocery / Supermarket

1. Products with use-by dates: tick **Batches and expiry dates** (works in any shop, but essential for grocery).
2. **Weighed items:** give each its **scale code (PLU)** and sell it in **KG** (3 decimals). Set the label scale to print EAN-13 labels: `21` · item code (5 digits) · weight in grams (5 digits) · check digit.

3. **System → Settings → Grocery:** check the scale label prefixes, and the markdown rules (default 30 % off at 7 days left, 50 % at 3 days).
4. Electronic shelf labels (optional): enter their gateway address in System → Settings → Grocery.
5. **Grocery → Scale items** lists every weighed product with its scale code, unit and retail price. **Export** it to load the scales, and again after price changes.
6. **Grocery → Markdowns** (daily): apply the suggested markdowns, print the stickers, and write off what has expired.

Why: the till reads the weight and item from the scale label, and sells the batch expiring first, so old stock does not go to waste.

Check: print a test label on the scale and scan it at a till: the item appears with its weight and price.

Pharmacy

1. On each medicine (Catalog → Products → **Medicine**): **schedule** (OTC, Prescription only, Controlled), generic name, strength, form, FDA registration number, **active ingredients**. Switch on **Batches and expiry dates** (required for medicines).
2. **Pharmacy → Drug interactions:** add the interactions you want the till to warn about.
3. **System → Settings → Pharmacy:** the Pharmacy Council premises licence number and the superintendent pharmacist, printed on the controlled-drug register.
4. Give pharmacists the **Pharmacist** role (and technicians **Pharmacy Technician**) at the branch.

Why: every prescription-only or controlled medicine sale needs a pharmacist's approval and records the prescription; controlled drugs go into the Controlled Drug Register.

Check: sell a POM medicine on a till: the till asks for the patient, the prescription and the pharmacist's PIN.

Restaurant / Café / Bar

1. **Restaurant → Kitchen stations:** e.g. `KITCHEN`, `BAR`. Tick *Print tickets* for a station that has a printer instead of a screen.
2. **Restaurant → Floor plan:** each table with its number, area, seats and position.
3. **Restaurant → Modifiers:** groups of choices (e.g. *Spice: Mild / Hot*). A choice can be charged as a product.
4. On each **dish** (Catalog → Products → **Restaurant**): the station, the choices offered, and the **recipe** (ingredients per portion). Switch **Track stock** off for the dish itself; the ingredients are stocked products.
5. **System → Settings → Restaurant:** service charge rate and its product (create a *Service charge* product with *Track stock* off, and copy the SKU the system gives it into this setting), tips on/off, booking reminders and the SMS gateway address (only with your own SMS provider's details).
6. Give staff the **Waiter / Bartender** or **Kitchen Staff** role.
7. Set up the **kitchen screens** (Chapter 9).

Why: bills live on the server so every till, waiter tablet and kitchen screen shares the same tables. For a restaurant with unreliable internet, use a **branch server** (Chapter 10) so this keeps working during outages.

Check: open a table on a till, send a dish, and see it on the kitchen screen.

Electronics / Technology

1. On serial-numbered products (Catalog → Products → **Serial numbers and warranty**): tick **Serial / IMEI numbers**, enter the **warranty** in months.
2. Stock already on hand: **Electronics** → **Serial numbers** → **Register numbers for stock on hand**.
3. Create a **labour** product (no stock) for repair charges.
4. Give technicians the **Repair Technician** role.

Why: every unit is then traced from receipt to sale, warranties are checked by number, and stolen or wrong units cannot be sold.

Check: add a serial-numbered product at a till: it asks for the number and refuses one that is not in stock there.

Fashion & Boutique

1. **Fashion** → **Styles** → **New style:** code, name, tax, season, collection, sizes, colours and price. Then **Create sizes and colours** to make one product per size and colour, each with SKU and barcode.
2. **Consignment** (goods owned by a supplier): on the product, choose the consignor and their share.
3. **Alterations:** create an *Alteration* product with *Track stock* off, and copy the SKU the system gives it into System → Settings → Fashion.
4. Give tailors the **Tailor** role.

Check: pick any variant of a style at the till: the size × colour grid opens with the stock at that till.

Sales & Distribution (vans, routes, reps)

1. **Distribution** → **Vans** → **New van:** branch, van code (e.g. `V01`), name, registration, rep and the warehouse it is loaded from. The van is created as a warehouse **with its own till**.
2. Register the rep's phone as a **POS till** device on that van's till and enrol it (Chapter 9.4/9.5). It works **fully offline**.
3. Give the rep the **Van Sales Rep** role at the branch and a PIN; in **HR & payroll** → **Employees** set their **Commission %** if they earn commission.
4. **Routes:** Distribution → Routes → New route — the van, rep, weekdays and customers in visiting order.
5. **Targets:** Distribution → Rep targets — monthly sales target per rep.
6. **Returnable containers** (crates, bottles, gas cylinders): mark the container products as *returnable containers* (their price is the deposit) and choose the container on each product sold in them (Product → Returnable containers).

7. Daily: **Load van** in the morning; in the evening **Start end-of-day settlement**, count, submit, and a supervisor approves (Chapter 14 and `docs/distribution.md`).

> **Why:** a van as a warehouse with its own till means every sale, receipt, shift and cash count works exactly as in a shop — offline too — and the daily settlement catches shortages the same day.

> **Check:** load a van, sell from the phone, then settle: the van's stock in *Stock on hand* matches what is left on it.

Manufacturing (factories, bakeries, workshops, food processing)

1. **Switch it on** for the branch that produces (System → Modules → Manufacturing). *Why:* the screens and work orders only work where the module is on.
2. **Work centres:** Manufacturing → Work centres → New — each machine or bench (e.g. `MIX` Mixer, `OVEN` Oven), its **cost per hour** (wages of the people there plus running costs such as power) and the hours a day it runs. *Why:* every minute worked there is charged to the product, so its cost is real; the hours show how full each machine is.
3. **Bills of materials:** Manufacturing → Bills of materials → New — the product made, how many one batch makes, the **materials** per batch (with any expected loss %), **by-products** (e.g. bran from milling), and the **operations** in order with their work centre and minutes. Then **Put in use**. *Why:* a BOM in use is locked so open orders and history stay true; to change it, start a **New revision**.
    - A material that has its own BOM is a **sub-assembly** (e.g. dough for bread): it is made on its own work order.
    - **Backflush** (default) means the material or time is charged automatically when output is reported; switch it off for items you issue by hand or time you record.
    - Tick **Quality check before stock** if output must be inspected first.
4. **Roles:** *Production Manager* (sets up, runs orders, checks quality), *Machine Operator* (runs orders), *Quality Inspector* (passes or fails output). *Why:* the person who makes the goods is not the one who passes them.
5. **Daily:** Manufacturing → **Production planning (MRP)** shows what to make and what to buy; **Raise work order**, then on the work order **Release** → **Report output** → **(Quality check)** → **Finish**. Print the **Job card** for the shop floor.

> **Why:** production at actual cost (the owner's choice) means each product enters stock at what its materials, labour and machines really cost, and the accounts show work in progress until it is finished.

> **Check:** make a small order of a product, then look at *Stock on hand*: the materials went down, the product went up, and the work order's *Actual cost* section shows the cost per unit.

Services & Packages (salons, spas, gyms, clinics, consultancies)

1. **Services:** on each service (Catalog → Products → **Services**) enter the **duration** in minutes and switch **Track stock** off. Optionally a **commission** rate of its own.
2. **Staff:** in **HR & payroll** → **Employees**, switch on **Takes appointments** for each person who does services, and set their **Commission %**. Give them the **Stylist / Therapist** role (or another role with *appointments*).

3. **Prepaid packs** (e.g. *5 massages for GHS 800*): a product with *Track stock* off and, under **Services → Prepaid pack**, the services it covers and the number of sessions, and how many days it is valid.
4. **Memberships and subscriptions:** create a product to invoice them as (e.g. *Gym membership*, *Track stock* off), then **Services → Membership plans → New plan**: price per period, how often it is billed (monthly, every 3 or 6 months, yearly) and any sessions included each period.
5. **System → Settings → Services:** the calendar's start and end times and time steps, appointment reminders (hours before), and the SMS gateway address (leave empty to use the restaurant one; only with your own SMS provider).
6. **Software licence keys** (optional): on the product tick **Hands out a licence key** (*Track stock* off), then **Services → Licence keys → Load keys** for each branch that sells it.

> **Why:** the duration fills the appointment book; the person who did each service earns their commission, which payroll pays each month; packs and memberships let customers prepay and use their sessions at the till.

> **Check:** book an appointment in **Services → Staff calendar**, check it out at a till with **Appointments (F10)**, and see the commission under **Services → Commissions**.

> **Note:** memberships are invoiced by **head office** each day (01:30) for every period that has started; the invoices go on the customer's account (Sales → Invoices) and are paid like any invoice.

8.3 Company-wide features (licence options)

Feature	What it adds	Set-up
Finance	Chart of accounts, automatic postings from sales, purchases, stock and payroll; supplier bills; bank reconciliation; month-end; reports	Chapter 7, step 12; your accountant reviews the chart and opening balances
HR & payroll	Employees, clock-in at tills, leave, loans, monthly payroll with PAYE and SSNIT, payslips	Chapter 7, step 12; payroll profile checked with your accountant

8.4 Planned modules

These appear as **Planned** and can be reserved: **Manufacturing** (bills of materials, production at actual cost), **Logistics & Delivery**, **Assets & Maintenance**, **Projects & Tasks**, **Helpdesk**. This guide gets a checklist for each as it is released.

9. Setting up tills and other devices

9.1 How devices are set up

Every till, handheld and kitchen screen is set up the same way:

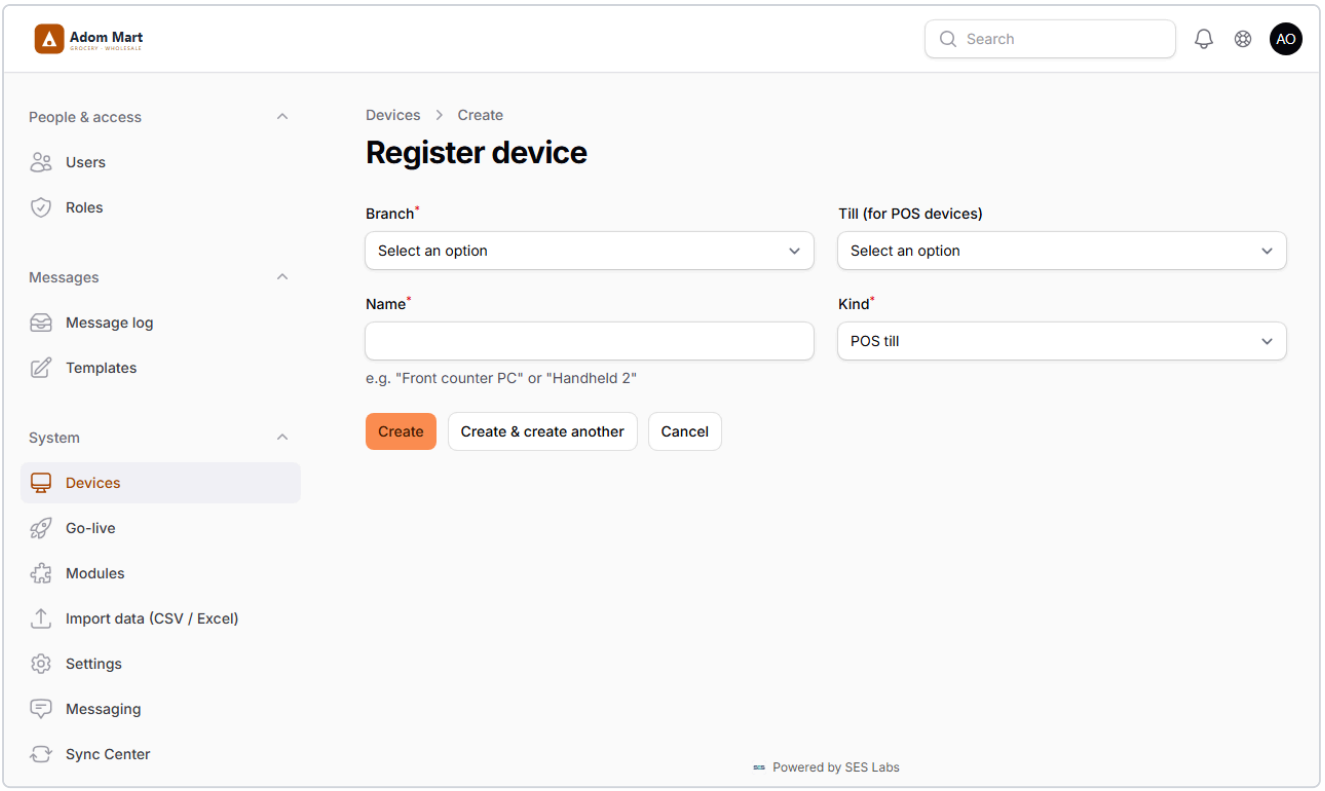
1. **Register** it in the back office: **System** → **Devices** → **New device**. You get a **one-time enrolment code** (valid 24 hours by default).
2. On the device, open the till app address in Chrome or Edge and **enter the code**.
3. People **sign in** on it (cashiers with username and PIN; kitchen screens need no sign-in).

> **Why:** only devices you registered can sell or see your data. Each device creates its own security key when it is enrolled, so a copied browser cannot pretend to be your till. A lost device is switched off with one click (**Revoke**).

Register a device

System → **Devices** → **New device**:

Field	What to enter
Branch	Where the device is used
Till (for POS devices)	The till it sells on (e.g. *Till 1*). Each selling device needs a till.
Name	Something you will recognise: *Front counter PC*, *Waiter tablet 2*, *Handheld 1*
Kind	**POS till** (tills and waiter tablets) or **Handheld scanner** (warehouse). **Kitchen display** (kitchen or bar screen) is offered only at a branch running the Restaurant module.



Click **Create**. The code is shown **once**: write it down or keep the page open while you set up the device. If it expires, use **New code** on the device's row.

9.2 The till app address

Setup	Address to open on the device
A (on the server PC itself)	`https://localhost/pos/`
B (Windows server on the shop network)	`https://<server-address>/pos/` (e.g. `https://192.168.1.10/pos/`). Trust the server first (4.9).
B/C (Ubuntu server on the network, no domain)	`https://<server-address>/pos/`. Trust the server first (4.9).
D (cloud, with HTTPS)	`https://<your-domain>/pos/`
E (branch server)	the **branch server's** address, as in B

9.3 Trust the server's address

Browsers allow a web app to use its **security functions** (the till's own signing key, the offline PIN check), to **work offline** and to **install as an app** only when it comes from a secure address: `https://…` or `localhost`. **This is required**: without it the till cannot even enrol.

- **A server without an internet name** (Windows or Ubuntu): it is reached over `https` with its own certificate. Each device trusts it once, as in **4.9**. Nothing else is needed.
- **Setup D and servers with an internet name:** nothing to do. The certificate is a public one.

- **Only for a server you set up by hand without https** (not recommended): each till, tablet, handheld and kitchen screen must be told once that the plain `http://` address is trusted:

1. In Chrome, open `chrome://flags/#unsafely-treat-insecure-origin-as-secure` (in Edge: `edge://flags/#unsafely-treat-insecure-origin-as-secure`).
2. In the box, type the server address **without** `/pos/`, e.g. `http://192.168.1.10`.
3. Set the option to **Enabled** and click **Relaunch**.

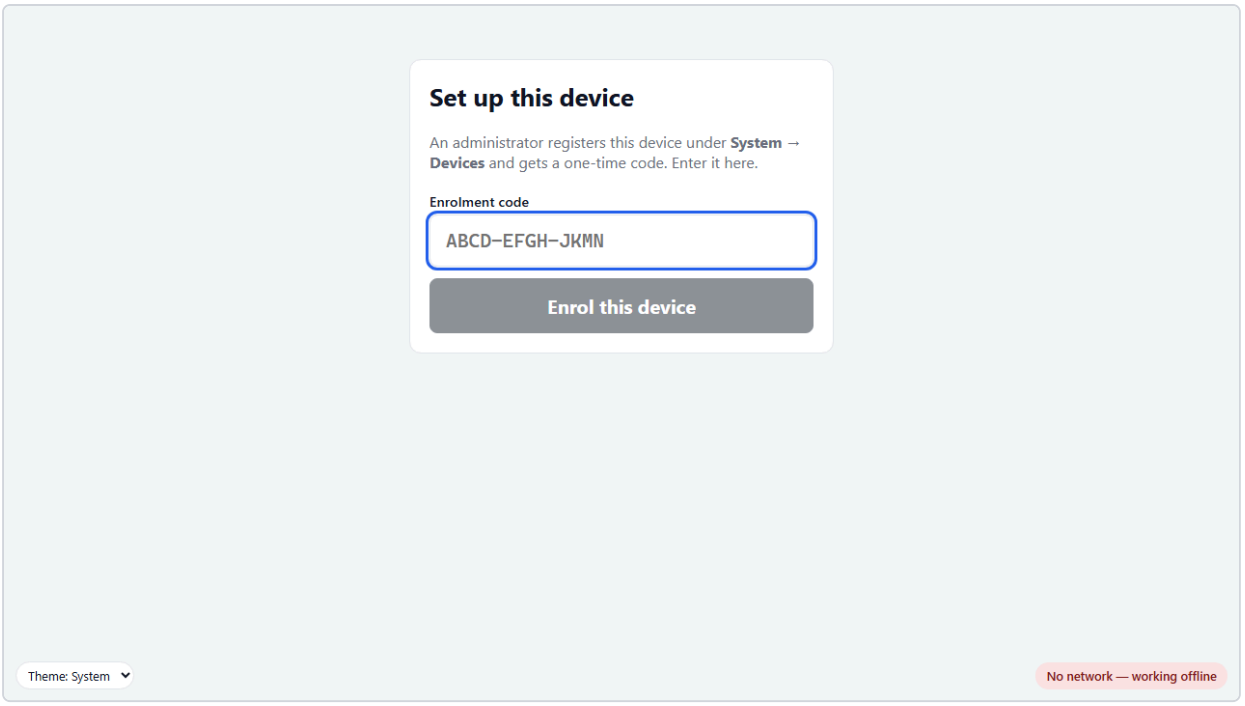
Why: the till proves it is your device by signing with a key it creates and keeps itself, and checks PINs offline; browsers only offer these functions to trusted addresses.

Check: the till app shows *Set up this device* and enrolling with the code works.

Note: This is safe on your own shop network. Never use plain `http://` over the internet; use HTTPS (Chapter 5).

9.4 Till PC (Windows)

1. Install **Google Chrome** or use **Microsoft Edge**. Set the PC's clock to update automatically (as in 4.1, step 2).
2. Do 9.3 if your setup needs it.
3. Open the till address (9.2).
4. **Install it as an app:** in Chrome, menu ⋮ → **Cast, save and share** → **Install page as app…**; in Edge, menu **…** → **Apps** → **Install this site as an app**. A *POS* icon appears on the desktop and Start menu; use it from now on. **Why:** the app opens full-screen without the address bar and starts even when the server is off.
5. Enter the **enrolment code** and click **Enrol this device**:



6. A cashier signs in with **username and PIN**, counts the drawer and **opens the shift**.

> **Check:** sell an item and print the receipt. Then pull out the network cable (or switch off Wi-Fi): the top shows *working offline*, and you can still sell. Plug it back in: the yellow banner disappears as the sale is sent.

9.5 Tablet (Android)

1. Update Chrome from the Play Store. Do 9.3 if needed (Chrome on Android has the same flags page).
2. Open the till address, then Chrome menu ⋮ → **Add to Home screen** → **Install**.
3. Enrol and sign in as on a PC.

> **Tip:** waiter tablets in a restaurant are registered as **POS till** devices on a till of their branch.

Van reps: register the rep's Android phone as a **POS till** on the **van's till** (each van has one, created with the van). Allow the app to use the phone's **location** when asked, so visits record where they happened. On the phone the till shows **Route (F3)**, **Take order** and **Empties returned**. Install it as an app (step 2) so it opens without signal.

9.6 Choosing light or dark

Every till, handheld and kitchen screen has a **Theme** switch at the bottom left: **System** (follows the device's own setting), **Light** or **Dark**. The choice is remembered on that device. Dark is easier on the eyes in a dim bar or kitchen and saves battery on phones.

9.7 Receipt printer and cash drawer

1. Install the printer's **Windows (or Android) driver** from the manufacturer, and print a Windows test page.
2. In the printer's **Printing preferences**, choose paper **80 mm** (or 72 mm) receipt roll. For a cash drawer connected to the printer, switch on the driver's **cash drawer / open drawer before printing** option.
3. Make it the **default printer** on that till PC.
4. The first time you click **Print receipt**, in the browser's print window choose the printer, set **Margins: None**, **Scale: 100**, and untick **Headers and footers**. The browser remembers it.
5. **Print without the print window** (recommended): edit the desktop shortcut of the till app (right-click → **Properties**), and at the end of **Target** add a space and `--kiosk-printing`. Start the till from that shortcut.

> **Why:** `--kiosk-printing` sends receipts straight to the default printer, which is faster and avoids mistakes at busy times.

> **Check:** a sale prints a full-width receipt and the drawer opens.

The desktop till app (silent printing, cash drawer, scale)

For a Windows till PC with a receipt printer, a cash drawer or a weighing scale, install **SES Labs Till** (`SES Labs Till_<version>_x64-setup.exe`, from SyncLytics) instead of using the browser:

1. Install the receipt printer's Windows driver (or note the IP address of a network receipt printer).
2. Trust the server on this PC (4.9). Then run the SES Labs Till setup. On first start it asks for the **server address**: exactly what you type in a browser to open the back office, without `/admin`. For a cloud server that is its name, e.g. `https://erp.clientdomain.com`, **never its IP address** (the security certificate is for the name, so the IP is refused). For a server in the shop it is the address used in 4.9, e.g. `https://192.168.1.10`.
3. **Receipt printer:** choose it from the list (or type `tcp://<printer-ip>:9100` for a network printer); tick **Open the cash drawer** if the drawer is plugged into the printer; **Print a test receipt**.
4. **Scale:** choose its COM port and speed (usually 9600), then **Read the scale** with something on it.
5. **Save and open the till.** The app first checks that the server answers. If it does not, it says so and stays on this page: correct the address and save again. (**Use this address anyway** keeps an address that cannot be reached yet, for example when the server is not switched on.) Then enrol and sign in as with any till (9.1).

The app also tells the server **which computer it is** (its name, network card and address on the shop network), so the audit log and **System** → **Devices** (*Last used from*) show which till did what (12.1). **Open the back office** on its set-up page opens the back office in the app, so changes made there are recorded with the computer's details too.

Receipts then print straight away without a print window, the drawer opens after cash sales, and items sold by weight show a **Weigh** button. **Printer and scale set-up** on the receipt screen opens these settings again.

> **Why:** the app sends the receipt to the printer in the printer's own language (ESC/POS), which is faster and more reliable than a browser print window, and can talk to the drawer and scale, which a browser cannot.

> **Note:** the app only opens your SES Labs server — no other website — so the printer and drawer cannot be reached from anywhere else.

When the till starts: it opens the server straight away if it answers. If the server has worked on this PC before but cannot be reached now (internet down), the till still opens and sells offline. If the saved address has **never** worked, the set-up page comes back with a message, so a wrong address can always be corrected.

Starting the desktop till again from scratch

Uninstalling the app keeps its settings (the server address, printer and scale), so a reinstall comes back with the same settings. To clear them:

1. Close SES Labs Till (also from the system tray, if it shows there).
2. Open **PowerShell** (Start → type *PowerShell*) and run these two lines:

```
Remove-Item "$env:APPDATA\app.seslabs.till" -Recurse -Force -ErrorAction SilentlyContinue
Remove-Item "$env:LOCALAPPDATA\app.seslabs.till" -Recurse -Force -ErrorAction SilentlyCont
inue
```

3. Open SES Labs Till again (or reinstall it). It starts at the set-up page with empty fields.

> **Why:** the first folder holds the till's settings; the second holds what the till app saved for working offline. Clearing both makes the PC behave like a new till. Then delete the old entry under **System** → **Devices** in the back office and enrol the till again (9.1).

> **Note:** sales not yet sent to the server are in the second folder. Clear it only when the till shows nothing waiting to send, or when the till never worked.

9.8 Barcode scanner

1. Plug it in (USB) or pair it (Bluetooth).
2. Open Notepad and scan a product: the number appears and the cursor moves to a **new line**.
3. If it does not move to a new line, scan the **"Add Enter / CR suffix"** setting barcode from the scanner's manual.

> **Why:** the till adds the product when the scanner "presses Enter" after the code.

9.9 Handheld (warehouse)

1. Register a device of kind **Handheld scanner** for the branch.
2. On the phone: Chrome, do 9.3 if needed, open the till address (9.2), enrol with the code.
3. Storekeepers sign in with **username and PIN**. They need a warehouse role (Storekeeper, Warehouse Manager…), not till access.

> **Note:** the handheld needs its branch's server to be reachable (it does not queue work offline). For warehouses with weak internet, use a branch server (Chapter 10).

9.10 Kitchen screen (Restaurant)

1. Register a device of kind **Kitchen display** for the branch.
2. On the screen's PC or tablet: Chrome, do 9.3 if needed, open the till address, enrol with the code.
3. Choose the **station** it shows (e.g. *KITCHEN* or *BAR*). No one signs in.
4. Press **F11** (PC) for full screen.

> **Check:** send a dish from a till: it appears on the screen within a few seconds.

9.11 Counting stock with a handheld

During a stock count (Chapter 14.4) storekeepers scan the bin, then each product, and type the quantity. Counts from several handhelds go into the same count.

9.12 Label scale (Grocery)

Set the scale to print **EAN-13** labels starting with `21` (see Chapter 8, Grocery). The scale's own software holds each item's **PLU** (5-digit code) and price per kg; keep its PLU codes the same as the products' **scale codes**.

9.13 When a device is lost, stolen or replaced

- **Lost or stolen: System → Devices → Revoke** at once. Its access and every sign-in on it end immediately.
- **Replacing a till that still has unsent sales:** on the old till press **Export unsent sales** (in the yellow banner). In the back office: **System → Sync Center → Import till file**. Then register and enrol the new device.
- **Re-installing the same device:** register it as a new device (it gets the next receipt series letter, so receipt numbers never repeat).

10. Branch servers and several sites (setup E)

A **branch server** is a second copy of the server at a branch. The branch's tills, back office, kitchen screens and handhelds use it over the local network, so the branch keeps working fully when the internet to head office is down. It exchanges changes with head office every minute when the link is up.

> **Why:** in setup D, tills sell offline during an outage, but kitchen screens, shared restaurant bills, handhelds and stock look-ups need the server. A branch server puts a server inside the branch.

10.1 What flows where

Information	Direction	Who decides
Company, branches, tills, modules, settings, licence	HQ → branches	Head office
Users, roles, access, PINs	HQ → branches	Head office
Products, units, barcodes, prices, tax, promotions	HQ → branches	Head office
Sales, shifts, devices, review items	Branches → HQ	The branch
Stock movements, transfers, counts, purchase orders	Both ways	Where it happened
Customers, balances, bookings, passes, memberships	Both ways	Field by field / where it happened

> **Note:** on a branch server, master data (products, prices, users) is **read-only**: change it at head office and it arrives within a minute. Nothing is ever overwritten — sales and stock movements are only added.

10.2 Install the branch server

1. Head office must already be installed and activated (setup C or D).
2. At **head office**: **System** → **Sync Center** → **Add branch server**. Enter the branch server's **id** (e.g. `B01`, unique per server) and a name. Note the **command** and the **secret** that are shown **once**.
   Why: the secret proves to head office that the branch server is yours; every exchange is signed with it.
3. Install the branch server on its PC or small server with **Chapter 4** (Windows) or **Chapter 5** (Ubuntu), with these differences:
   - In `.env`: `ERP_NODE_ROLE=branch`, `ERP_NODE_ID=B01` (the id from step 2), and `APP_URL` = the branch server's own network address.
   - **Do not run** `php artisan erp:install` (the company comes from head office).
   - `php artisan migrate --seed --force` on a branch server only installs the permissions.
4. Join head office (on the branch server):

```
php artisan erp:node:join https://<hq-address> <HQ node id> <company id>
```

Paste the secret when asked. The exact command, with the head office id and company id filled in, is the one shown in step 2. **Why:** this downloads everything the branch needs: company, people, products, prices and stock.

5. Make sure the scheduler runs on the branch server (Chapter 4.8 or 5.9). It runs the sync every minute.

6. Register and enrol **that branch's devices on the branch server's own back office** (`http://<branch-server>/admin` → System → Devices), and point the tills at the branch server's address (Chapter 9).

> **Check:**

- Branch server back office → **System** → **Sync Center**: head office shows **OK** and a recent *Last sync*.
- Sell on a branch till; within a minute the sale appears at head office under **Sales** → **Till sales**.
- Change a price at head office; within a minute it appears on the branch till.

> **Note:** two-factor sign-in secrets and passkeys are not copied to branch servers (each server has its own encryption key). Managers who use the branch server's back office set up two-factor sign-in there once.

10.3 The Sync Center (System → Sync Center)

What you see / do	Use it when
Status per server: last sync, waiting, failed, last error	Daily glance
Sync now	You want changes across immediately
Choose what syncs per group: Automatic, Manual (preview first), Off	Normally leave everything on *Automatic*
Review incoming	A group is on *Manual*
Export / Import sync file (USB)	The link is down for days: carry the changes on a USB stick
Full resync from this peer	A branch seems out of step; safe to repeat
Import till file	A till was replaced before it could send its sales

10.4 Keeping servers in step

- **Upgrade head office first**, then branch servers. Changes a branch server cannot apply yet wait as *failed — update this node* and apply by themselves after the upgrade.
- Server clocks must agree within 5 minutes (automatic time on every server).
- To stop a branch server (closed branch, replaced PC): at head office **Sync Center** → **(row menu)** → **Disable this peer**. Nothing is deleted. Keep a final backup of the branch server.

11. Online stores, delivery apps and SMS

These connect your system to other services. Each one is used **only with your own account and keys** at that service. SES Labs never creates accounts for you.

> **Warning:** treat every key and secret you paste here like a password. They are stored encrypted and never shown again.

11.1 Requirements

- A server reachable from the internet with **HTTPS** (setup D, or head office in setup E). Online stores send orders to your server; they cannot reach a PC inside a shop.
- The **scheduler** running (it sends changes every minute and checks the stores every 15 minutes).

11.2 Online stores (Online → Channels)

Type	Set-up at the store	In SES Labs
Built-in online shop	none	Online Storefront module on (Chapter 8); channel type *Built-in online shop*; choose the address
WooCommerce	WooCommerce → Settings → Advanced → REST API: create a key with **Read/Write**. Webhooks: *Order created* and *Order updated* to the address SES Labs shows, with your webhook secret	Channel type *WooCommerce*: store address, key, secret, webhook secret
Shopify	Create a custom app with Admin API access to products, inventory and orders; note the access token and your location id. Webhook *Order creation* to the address SES Labs shows	Channel type *Shopify*: shop address, token, location id, webhook secret
Other shops (generic)	Must accept JSON product updates and send signed order webhooks	Channel type *Generic REST*

For every channel:

1. **New channel**: type, name, the **fulfilling branch**, **stock warehouse** and **price list**.
2. Enter the secrets (they stay hidden; leave blank to keep the current one).
3. Copy the **webhook address** shown in the channel list into the store's webhook settings.
4. Set **safety stock** on products where needed.
5. Click **Activate**. Every product is sent once; afterwards only changes are sent.

> **Why:** the store is offered *on hand − reserved − safety stock*, so the shop and the till do not sell the same last item.

> **Check:** change a price; within a minute it is on the store. Place a test order on the store; it appears under **Online** → **Online orders** with its stock reserved.

> **Tip: Compare with store** lists differences between the store and SES Labs and fixes them; it also runs automatically every 15 minutes.

11.3 Delivery apps (Restaurant)

Bolt Food, Glovo and Chowdeck orders can arrive by webhook (with the keys from **your** merchant account) or be entered by hand at the till. Set them up under **Online** → **Channels** with the type *Bolt Food (delivery app)*, *Glovo* or *Chowdeck*, then copy the webhook address into the app's merchant portal.

11.4 SMS, WhatsApp and e-mail

Booking and appointment messages, digital receipts, invoice e-mails and scheduled reports go through **your own** SMS, WhatsApp and e-mail accounts. Setting them up is explained in **Chapter 16**.

> **Note:** if you set an *SMS gateway address* under Settings → Restaurant or Services in an earlier version, it keeps working for those messages until you choose an SMS provider in Chapter 16.

11.5 GRA E-VAT (electronic VAT invoicing)

Once GRA has onboarded your business for E-VAT, it gives you the E-VAT address and your API key.

1. Put them in the server's `.env` (head office): `GRA_EVAT_URL=…`, `GRA_EVAT_KEY=…`, `GRA_EVAT_TIN=<your TIN>`, then `php artisan optimize`. Use GRA's **sandbox** address first.
2. **System** → **Settings** → **GRA E-VAT** → **Report to GRA**, choose one:

Mode	What happens	Receipts
Each sale and invoice as it happens	Every till sale and invoice is sent at once. If GRA or the internet is down, it is sent automatically later.	Carry GRA's code and QR code when GRA answers within a few seconds
Till sales in a daily batch	Till sales are sent together after the batch time (e.g. 23:30); invoices still go one by one	No GRA code
Invoices and credit notes only	Only invoices to business customers are reported	—

3. **Finance** → **GRA E-VAT** lists every report: *Accepted by GRA* (with GRA's reference), *Waiting to send*, or *Refused by GRA* (with GRA's reason — fix it, e.g. the customer's TIN, then **Send again**).

> **Why:** GRA requires VAT-registered businesses to report invoices electronically. The mode lets you start with invoices only, and move to real-time when your tills are reliably online.

> **Note:** test with GRA's sandbox and compare a few documents in Finance → GRA E-VAT with what GRA shows before switching to the production address. `docs/evat.md` describes the fields sent.

12. Daily running, backups, updates and moving

12.1 Every day

Who	What	Where
Cashiers	Open the shift with the float; close it with the cash count	Till
Supervisor	Look at the **Review queue** and resolve items	Sales → Review queue
Manager	Check no till shows a red banner (*refused*) or a long yellow queue	Tills; System → Devices (*Last seen*)
IT / owner	Check **System** → **Health** is all green (backup made last night, scheduler running)	System → Health
Head office	Check branch servers synced recently (setup E)	System → Sync Center

Every month: verify the audit log (**System** → **Audit log** → **Verify chain**), check stock (`php artisan erp:stock-check`), take the USB backup drive home if you use one, and every three months practise a full restore on a spare PC (12.3).

Who did what, and from which computer

System → **Audit log** lists every change and every sign-in: who, when, what changed, and where from.

Column	What it shows
IP address	The address the server saw. For a cloud server it is the shop's internet router, the same for every PC in that shop. For a server in the shop it is the PC's own address.
Computer	The name you gave the computer (see below), the browser and system (e.g. *Edge 141 on Windows*), and for the **desktop till app** (version 0.25 or later) also the **computer name**, its **MAC address** (network card) and its **address on the shop network**.

Open a row for the details; click the IP address to see roughly where it is (town and internet provider, on ipinfo.io). The **Computer** filter shows everything one computer did; the **Sign-ins** filter shows back-office sign-ins, sign-outs and failed attempts, and till sign-ins.

Name your computers: System → Computers. Every computer is listed the first time someone signs in to the back office from it, with who used it last, when and from which address. Click **Name it** and type a name you will recognise, such as *Front desk PC* or *Manager's laptop*. From then on the audit log shows that name. **What it did** opens its audit log. A computer you do not recognise means someone signed in from a computer you did not expect: change that person's password.

Renamed accounts: records keep the name a person had when they did something, so renaming an account never rewrites history. Where the account has been renamed since, a small ⇄ icon shows beside the name; hover over it for today's name. **People & access → Users → the person → Name history** lists every change of name and username, when and by whom. Give every person their own account; do not rename one person's account for someone else, or their work will look like the new person's.

Why browsers show less: for your security, a web browser never tells a website the computer's name or MAC address. So the system gives each browser its own lasting ID instead (kept like a cookie). Clearing the browser's data or using another browser makes it a new, unnamed computer in the list. The desktop till app does report the computer's name and MAC address, also when the back office is opened in it (on its set-up page: **Open the back office**). These are what the computer says about itself: good for telling computers apart, not proof that nobody pretended.

12.2 Backups (automatic, encrypted)

> **Why:** a theft, fire, failed disk or ransomware attack can destroy the server. A backup kept **somewhere else** is the only way to get your sales, stock and accounts back.

The server makes a backup **every night at 02:00** by itself. Each backup holds everything needed to start again: the **database**, the `.env` **settings** (with the keys that unlock stored secrets and seal the audit log) and the **uploaded files**. It is **encrypted** with your backup passphrase, so a lost USB drive or a leaked cloud folder reveals nothing. Every Sunday the newest backup is **checked**: decrypted, read, and — where the database user may create a scratch database — fully restored into it and compared, without touching your live data.

The backup passphrase

The installers create it and show it once. Otherwise, in the `server` folder run `php artisan erp:backup:key`.

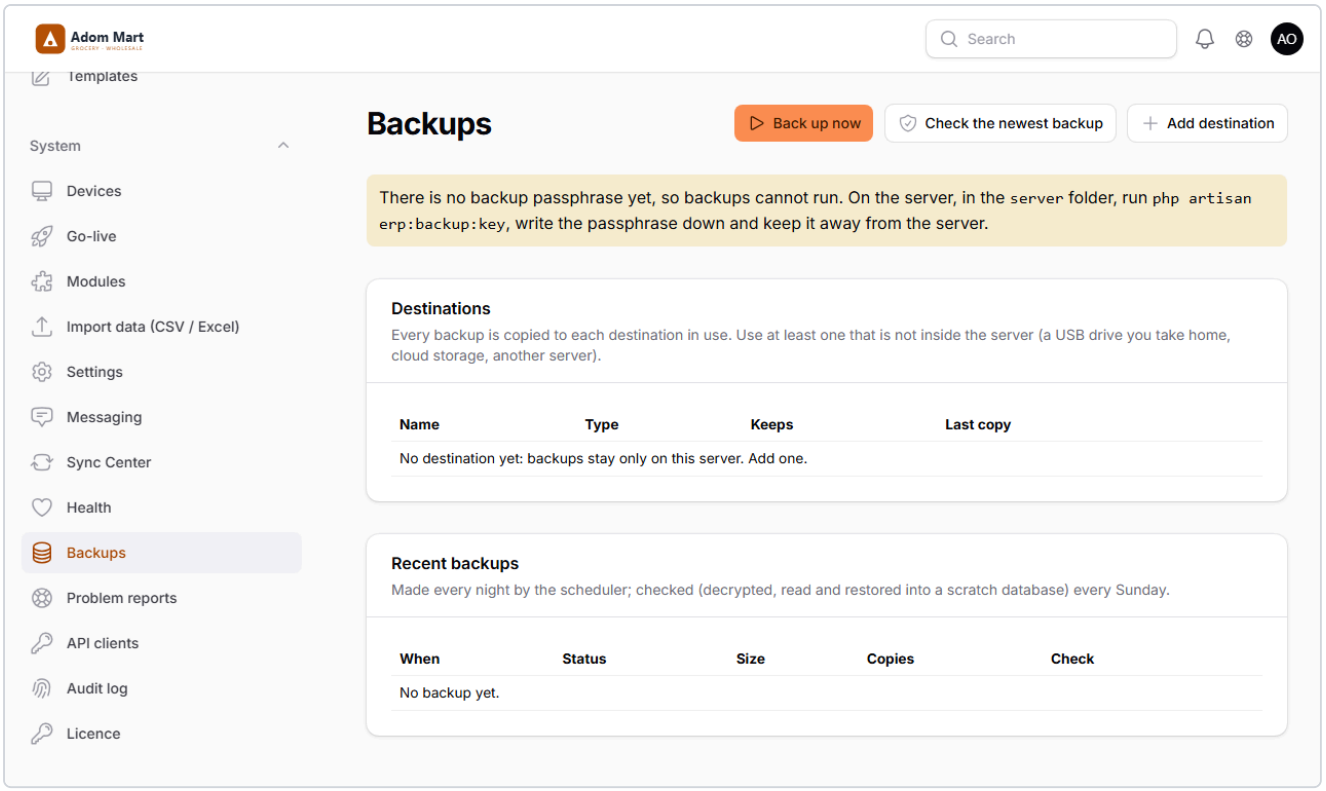
> **Warning:** write the passphrase on paper and keep it **away from the server** (with the owner, or in a safe). Without it no backup can be restored.

Where backups go

The server keeps the last 7 itself. Add at least one place **outside the server** under **System** → **Backups** → **Add destination** (a test file is written and removed before it is saved):

Destination	What to fill in	Good for
Folder — USB drive, second disk, network share	The folder, e.g. `E:\Backups` (Windows) or `/mnt/usb/backups`	Shops: rotate two USB drives, one always at home
S3-compatible storage — Backblaze B2, Wasabi, AWS	Endpoint, region, bucket, access key and secret (from your storage account; give the key access to that bucket only)	Cloud copies at low cost
Google Drive	OAuth client ID and secret, a refresh token, and the Drive folder ID (from a Google Cloud project in **your** Google account — your IT person sets this up once)	Businesses already on Google Workspace
Another server (SFTP)	Address, port, user, password or private key, folder	Head office or a hosting account

Keep the newest … says how many backups each destination keeps; older ones there are deleted automatically.



> **Check: Back up now**, then the backup shows ✓ for each destination, and **Check the newest backup** reports *Readable* or *Restore drill passed*. **System** → **Health** then shows *Backups OK*.

> **Note:** on Windows set `BACKUP_PG_BIN=C:\SESLabs\pgsql\bin` (the installer does this). For the full restore drill the database user needs permission to create a scratch database: the installers give it; on an existing server an administrator runs `ALTER ROLE erp CREATEDB;` in PostgreSQL.

12.3 Restoring from a backup

Use this after a disaster, or to practise on a spare PC.

1. Install the server software on the new machine with the installer or script (Chapter 4 or 5) — it creates an empty company you will replace.
2. Copy the newest `.synbak` backup file to it (from the USB drive, cloud storage or **System** → **Backups** → **Download** on the old server if it still works).
3. Put your **backup passphrase** in the new server's `.env` as `BACKUP_PASSPHRASE=…`, then in the `server` folder:

   ```
   php artisan erp:backup:restore C:\path\ses-labs-hq-20261029-020000.synbak
   ```

   It checks the backup, asks for confirmation, and replaces the data.
4. The backup's settings are saved next to `.env` as `.env.from-backup`. Copy `APP_KEY` and `ERP_AUDIT_KEY` from it into `.env` (they must match the backup), then `php artisan optimize`.
5. Restart the server (Task Scheduler on Windows, or reboot).

> **Check:** `php artisan erp:audit-verify` reports the chain intact, `php artisan erp:stock-check` passes, and yesterday's sales are in **Sales** → **Till sales**. If this is a new machine, re-activate the licence (Chapter 6.6).

> **Note:** tills keep any sales made since the backup in their queue and send them when the server is back, so they are not lost.

12.4 Updating the software

SES Labs sends you a new release. Update outside trading hours.

The simple way (recommended):

- **Windows:** run the new `SESLabs-server-setup-<version>.exe` on the server and choose the same folder.
- **Ubuntu:** `sudo bash install.sh --zip <new release>.zip` with the same options as the first time.

Either one **takes a backup first** with the version still installed. If that backup fails, it stops and changes nothing. Then it replaces the software, keeps your `.env`, uploads, backups and keys, updates the database, prepares the caches, applies the speed settings and restarts the services. Tills update by themselves (step 6 below).

Why the backup first: if anything goes wrong during the update, that backup puts the business back exactly as it was.

By hand (only if SyncLytics support asks you to):

1. **Take a backup** (run the backup task now) and check it.
2. Stop the server: Windows — Task Scheduler → *End* both SES Labs tasks; Ubuntu — `sudo systemctl stop caddy`.
3. Extract the new ZIP **over** the existing folder (`C:\SESLabs` or `/var/www/seslabs`). Your `.env` and `storage` are kept.
4. In the `server` folder:

   ```
   composer install --no-dev --optimize-autoloader
   php artisan migrate --seed --force
   php artisan optimize
   php artisan filament:optimize
   ```

   Why the last two: they prepare the settings, routes, screens and icons once, so pages do not rebuild them on every request.

   (Ubuntu: then `sudo chown -R www-data:www-data storage bootstrap/cache`.) If you install from source code, also rebuild the till: in `pos`, `npm install` and `npm run build:server`.

5. Start the server again (Windows — *Run* both tasks; Ubuntu — `sudo systemctl start caddy` and `sudo systemctl restart php8.4-fpm`).

6. **Tills** download the new version by themselves. It takes effect when the till app has been **closed completely and opened again** (do this at the end of the day; sometimes it needs two restarts).

Updating to 0.34.2 or later from an older version: use the simple way above. It switches on PHP's code cache (OPcache) and other speed settings, which older Windows installations did not have, and pages then open several times faster. *System → Health* shows *PHP code cache: On* afterwards. The first update to 0.34.2 also adds database indexes; on a large database this takes a few minutes, so let it finish.

Optional on older installations: in `server\.env` set `LOG_LEVEL=warning` (the log no longer records every detail) and `CACHE_STORE=file` (one less database round-trip for each till request), then run `php artisan optimize` again. New installations have both already.

Why `migrate --seed`: it adds new tables and new permissions safely; it never removes your data or changes your own settings.

With branch servers: update **head office first**, then each branch server.

12.5 Moving to a new server

1. Install the new server (Chapter 4 or 5) without creating keys, tables or the company, as in 12.3 step 1.
2. Take a fresh backup on the old server **after closing trading** and stop the old server.
3. Restore it on the new server (12.3).
4. Point the devices to the new address: if the address changed, open the new till address on each device and enrol it again (System → Devices → New code), after its queue has been sent.
5. Re-activate the licence within 14 days (Chapter 6.6).

13. Troubleshooting

Find the symptom, then try the fixes in order.

Installation

Symptom	Cause and fix
`php` is not recognised	PHP is not on the path. Repeat 4.3 step 6, then **close and reopen** PowerShell.
`could not find driver` or `pdo_pgsql` missing	The extension is not switched on. Check `php.ini` (4.3 step 5) and that `extension_dir = "ext"`. Run `php -m` to confirm.
PHP does not start: *VCRUNTIME140.dll missing*	Install the Visual C++ Redistributable (4.3 step 1).
`SQLSTATE[08006] … password authentication failed for user "erp"`	`DB_PASSWORD` in `.env` differs from the one in 4.2/5.3. Fix `.env`, then `php artisan optimize`.
`SQLSTATE[08006] … Connection refused`	PostgreSQL is not running. Windows: Services → *postgresql-x64-17* → Start. Ubuntu: `sudo systemctl start postgresql`.
`composer install` fails with a PHP extension message	Switch on the extension it names in `php.ini` (Windows) or install `php8.4-<name>` (Ubuntu).
`ERP_AUDIT_KEY is missing`	Run `php artisan erp:audit-key` (only on a new installation — never replace an existing key).
The page shows *500 Server Error*	Look at the end of `server/storage/logs/laravel.log` for the reason. After changing `.env`, always run `php artisan optimize`.
Ubuntu: *Permission denied* in the log	`sudo chown -R www-data:www-data storage bootstrap/cache`
Uploading a CSV file does nothing, or *failed to upload*	PHP cannot write its temporary upload file. Windows: check `upload_tmp_dir` in `php.ini` points to an existing folder (4.3 step 5) and restart the server task.
`/pos/` says the till app is not installed	Build it: in `pos`, `npm install` then `npm run build:server` (4.7).

Back office

Symptom	Cause and fix
Lost the authenticator phone	Use a **recovery code**. If those are lost too, another owner/IT Admin resets the person's two-factor sign-in.
A menu is missing	The module is off for your branches (Chapter 8), your licence does not include it (Chapter 6), or your role does not allow it (ask the owner).
This is managed at head office	You are on a branch server; change master data at head office (Chapter 10).

Symptom	Cause and fix
Everything is read-only, with a red banner	Licence expired, withdrawn or not checked in for 44 days (Chapter 6.4). Renew; nothing is lost.
Passkey button does nothing	Passkeys need HTTPS or `localhost`. Use the authenticator app.
Pages are slow	Open **System** → **Health**. *PHP code cache* must say *On* (if not, update with the simple way, Chapter 12.4). *Slow pages* lists what was slow, how often and how many database reads it made: send it to SyncLytics support. Run `php artisan erp:load-test --force` outside trading hours for a full speed check (it leaves nothing behind).

Tills

Symptom	Cause and fix
Till shows *working offline* but the server is up	Network or Wi-Fi between till and server. Open `https://<server-address>/up` on the till's browser. On Windows servers, check the firewall rule (4.8, step 8).
The browser warns *Your connection is not private*	The device does not trust the server's certificate yet (4.9), or the server's address changed since installation (run the installer again, then 4.9 on each device).
The till does not work at all when the server is off	Offline was never enabled on that device: setups B/C/E need 9.3; then open the till once while the server is on.
Enrolling a till fails on a network address (`http://192.168…`)	The browser does not trust the address yet: do 9.3 on that device.
Cashier cannot sign in offline	They must have signed in on **this** till while it was online within the offline period (System → Settings → Offline till sign-in lasts).
PIN locked	Too many wrong PINs. A manager resets it: People & access → Users → the person → **Set POS PIN**.
Red banner: items refused	A supervisor opens **Sales** → **Review queue** (*sync failed*) to see why, fixes it, then presses **Retry refused** on the till.
The price … changed. Refresh the till	The price changed at head office while the item was in the cart. Remove the line and add it again.
Open a shift on this till before selling	Open the shift (count the float).
The till clock is ahead of the server	Set the till's clock to automatic (Windows Settings → Time).
Receipt is tiny or cut	Printer paper size 80 mm, margins *None*, scale 100 %, headers off (9.7).
Scanner types the code but nothing happens	The scanner does not send Enter (9.8).
Desktop till: *Could not reach …* on the set-up page, or a blank window	The server address is wrong. Use the server's name exactly as in the browser (`https://erp.clientdomain.com`), not its IP address. Correct it and click **Save** (9.7). Old app versions open the saved address with no way back: clear the settings as in 9.7, *Starting the desktop till again from scratch*.

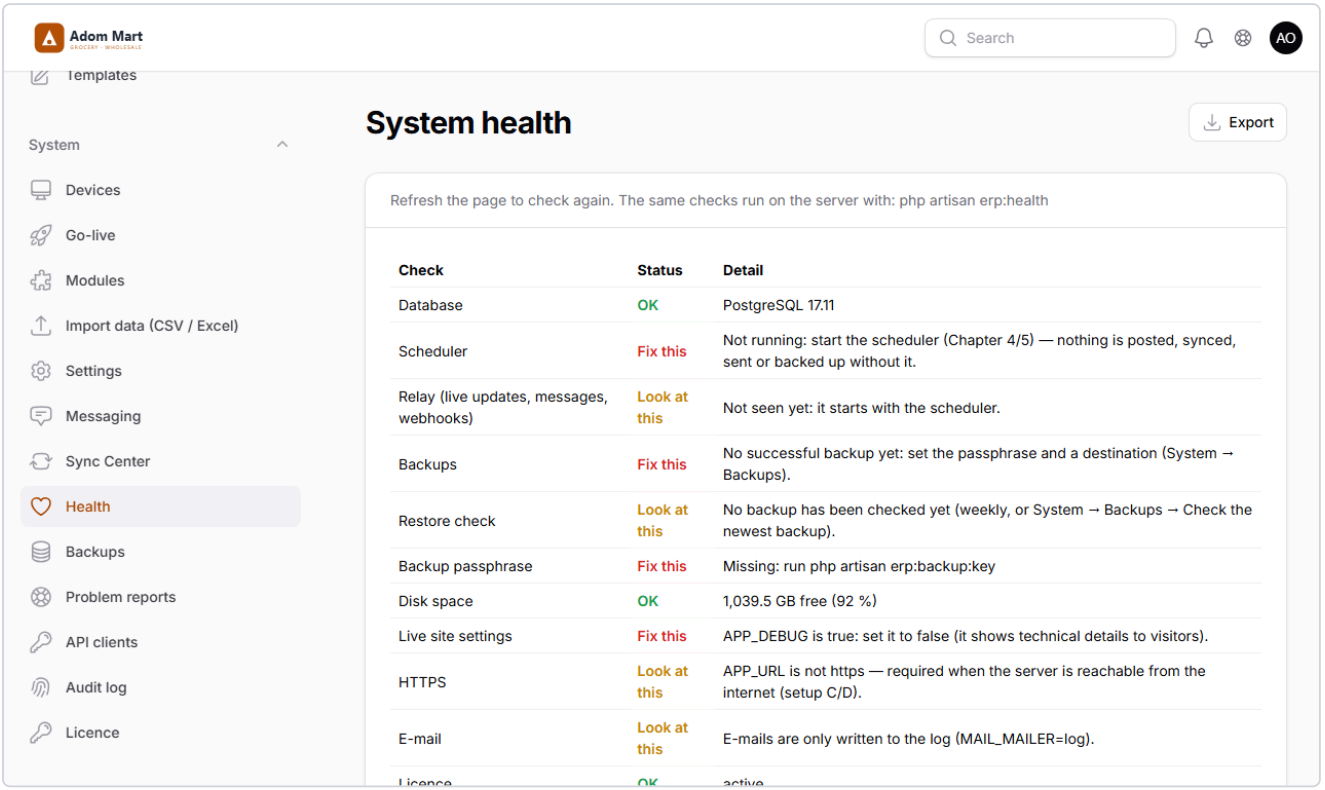
Symptom	Cause and fix
Till did not pick up a new version	Close the till app completely and open it again (twice if needed).

Branch servers and online stores

Symptom	Cause and fix
Sync Center: *Sync request time … off*	Server clocks differ by more than 5 minutes; set automatic time on both.
Changes *failed — update this node*	Update the branch server to the same version as head office (12.4).
A branch seems out of step	Sync Center → **Full resync from this peer**, then `php artisan erp:stock-check`.
Online store not updating	Is the scheduler running (4.8 / 5.9)? Is the channel **Active**? Look at the channel's failed updates and **Retry failed updates**.
No SMS arrive	Check the SMS gateway address in Settings and that the phone number is on the booking. Test your provider separately.

Health checks

System → **Health** shows the state of the server at a glance: database, scheduler, live updates, last backup and its check, disk space, live-site settings, HTTPS, e-mail, licence, PHP's code cache, slow pages, and anything stuck (messages, webhooks, GRA E-VAT, sync). Each line says what to do. Below the checks, **Slow pages** lists every page and till call that took over a second in the last 7 days, worst first.



The same checks run with `php artisan erp:health` (useful for monitoring tools: it fails when something needs fixing).

Individual checks (run in the `server` folder)

Command	What it checks
`php artisan about`	PHP, database and settings in use
`php artisan erp:audit-verify`	The audit log is complete and untouched
`php artisan erp:stock-check`	Stock levels match the stock ledger
`php artisan erp:gl-check`	The accounts agree with sales, stock, customers and suppliers (Finance)
`php artisan license:status`	Licence state, limits and last check-in
`php artisan erp:load-test --force`	Speed: posts test sales and times the heavy reports, searches and the till catalogue on your real data, then undoes everything. Lines over a second are marked *SLOW*

Getting help from SyncLytics

Hotline (support)	024 563 6889
Office	055 668 6975 · 050 693 8589
E-mail	info@synclyticsenterprisesystems.com
Website	synclyticsenterprisesystems.com
Address	Puff Cl, Race Course St, Tabora, Accra (GhanaPost GPS GC-027-0884)

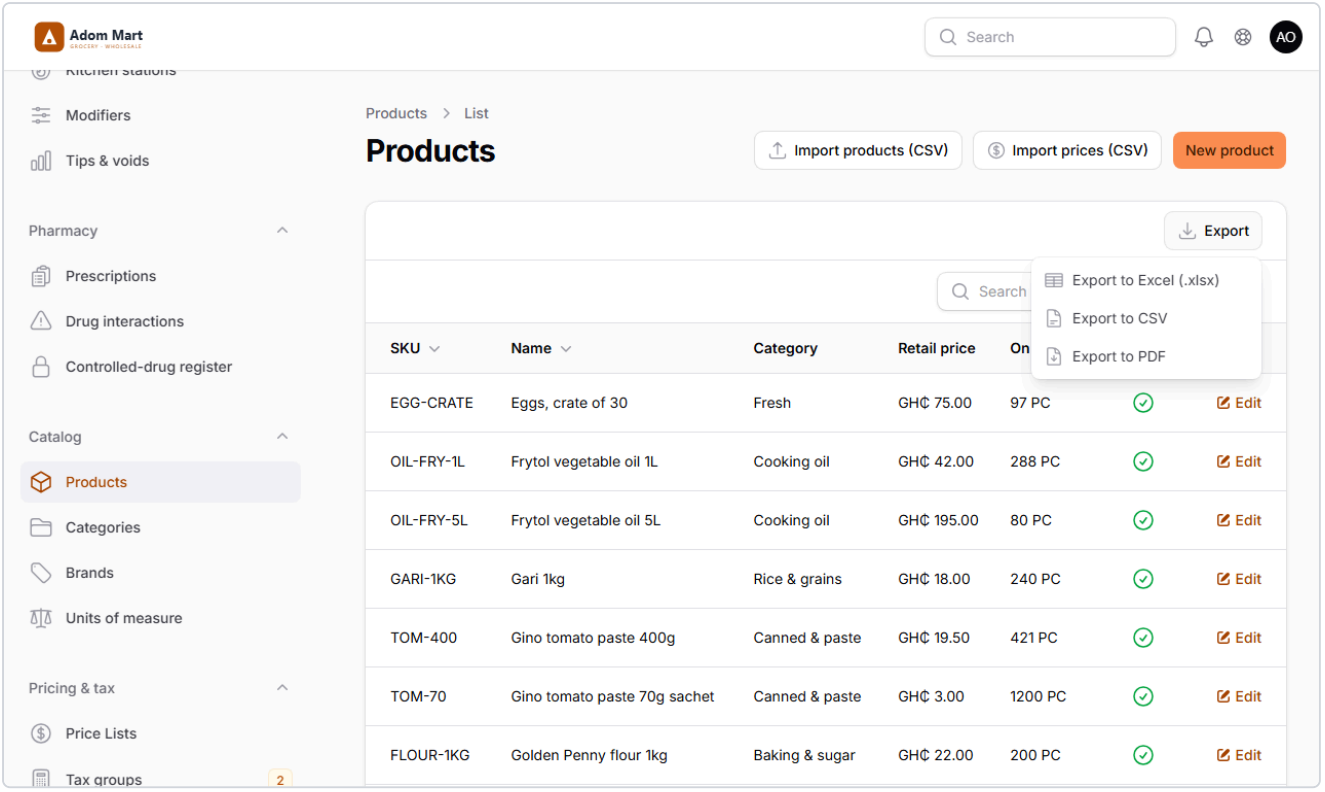
The quickest way is **Report a problem** in the back office (top right) or on the till: it attaches the technical details for you.

Otherwise send: what you were doing, the exact message (a screenshot helps), the time it happened, and the last lines of `server/storage/logs/laravel.log`. **Never** send your `.env` file, passwords, PINs or recovery codes.

14. Exporting and importing data, stock adjustments and stock takes

14.1 Exporting (downloading) anything you can see

Every list in the back office (products, customers, invoices, stock on hand, sales, audit log…) and every report page (financial reports, aged debts, shrink, commissions, timesheets, the staff calendar…) has an **Export** button at the top right:



Choice	What you get	Use it for
Export to Excel (.xlsx)	An Excel workbook (numbers stay numbers; opens in Excel, Google Sheets, LibreOffice, Numbers)	Working with the figures: sorting, totals, sending to your accountant
Export to CSV	A plain `.csv` file	Loading into another program
Export to PDF	An A4 PDF with your company name, the title, the filters in use, who printed it, the date and page numbers	Printing, filing, e-mailing

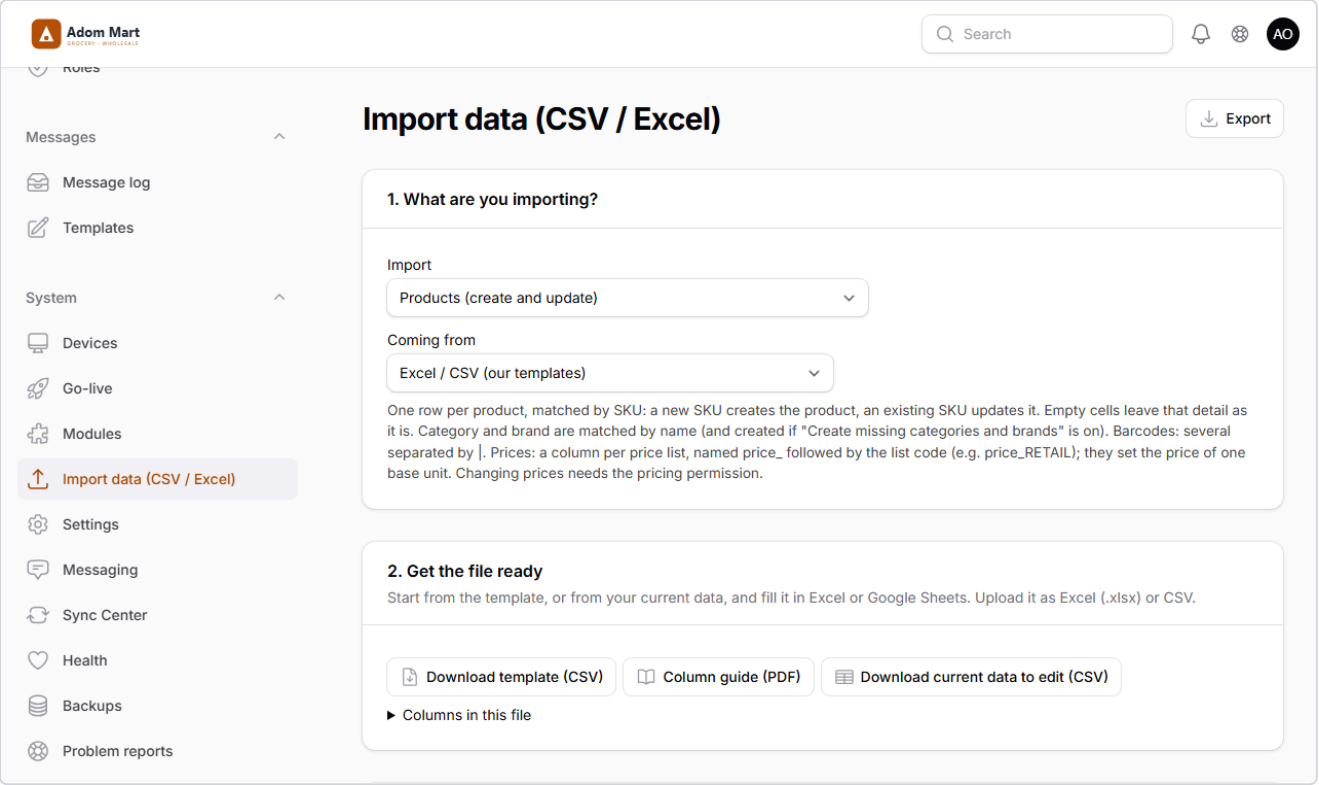
- The file contains **what is on screen**: the same columns, search, filters and sort — but **all pages**, not only the page you are looking at. Filter first (e.g. one branch, one month), then export.
- Amounts appear as on screen (with the currency); dates in your local time.
- A PDF holds up to 5,000 rows per table. For more, use CSV.

Why: every export is recorded in the audit log (who, what, how many rows), because exported files leave the system. When the licence is read-only, exports are blocked like printing.

Note: A CSV opens in Excel by double-clicking it. Accents and the cedi sign show correctly. A cell starting with `=`, `+`, `-` or `@` that is not a number gets a leading apostrophe so that Excel never runs it as a formula — this protects you from "CSV injection".

14.2 Importing data from CSV files

System → Import data (CSV / Excel) — or the **Import … (CSV)** buttons on Products, Customers, Suppliers, Stock adjustments and stock counts.



What you can import

Import	What a row does	Matched by
Products (create and update)	Creates a product (leave SKU empty: the system gives it one), or updates an existing one by its SKU. Prices per list in columns `price_RETAIL`, `price_WHOLESALE` …	SKU
Prices (price list update)	Sets one price (list, unit, from-quantity, optional dates for a promotion)	SKU + list + unit + quantity + dates
Customers	Creates or updates a customer, optionally with credit limit and terms	Code, else phone number
Suppliers	Creates or updates a supplier	Code
Opening stock	The go-live quantity and cost of a product in a warehouse	Warehouse + SKU

Import	What a row does	Matched by
Stock adjustment lines	Increase, decrease or "set to" lines in a stock adjustment	SKU or barcode
Stock count results	Counted quantities in an open stock count	SKU or barcode (+ bin)
Chart of accounts (migration)	Adds an account from the old books (QuickBooks types accepted)	Code, else name
Opening trial balance (migration)	One line of the opening journal on the go-live date	Account code or name
Customer / supplier balances owed (migration)	What a customer owed you / you owed a supplier on the go-live date	Code, phone or name
Loyalty points / gift cards (migration)	Points and unspent cards brought over from the old system	Customer; card number

The migration imports are explained in Chapter 17.2. **Coming from** (QuickBooks, another till system, or our templates) matches the old system's column names for you.

Step by step

1. **Choose what you are importing.** The page explains what a row does and lists every column (**Columns in this file**), with an example. For **products**, a company with several industries also chooses **For which industry?** (for example *Pharmacy*, or *Shared by all industries*). New products go into that industry's catalogue, and **Download current data** gives that industry's products only. Check and Import stay refused until you choose. An `industry` column in the file can name a different industry for single rows.

   > **Why:** each industry's tills and screens show only its own products. A product list imported to the wrong industry would show at the wrong shops.

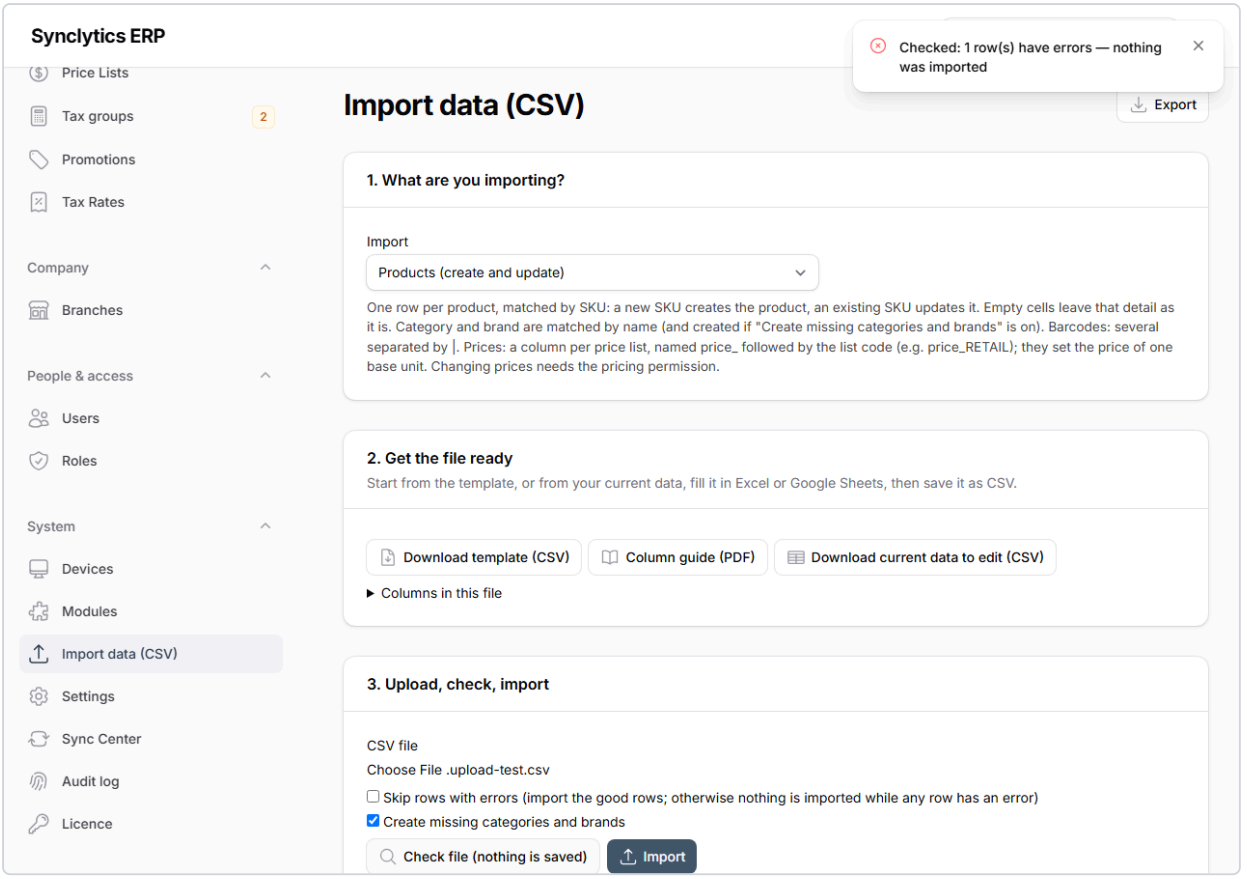
   Products already in the wrong industry: Catalog → Products, tick them (the tick at the top selects the page; *Select all* selects every product in the list), then **Move to industry**. Stock and prices stay; SKUs given by the system take the new industry's series (e.g. `100123` becomes `PHARMA-000124`), while SKUs from your old system stay as they are.

2. **Get the file ready:**
   - **Download template (Excel)** — the headings and one example row (also as **CSV**); or
   - **Download current data to edit (Excel)** — your existing products, prices, customers or suppliers in exactly the import layout (also as **CSV**). Change what you need in Excel and import it back;
   - **Column guide (PDF)** — what to put in each column, to print.

3. Fill it in Excel or Google Sheets and save it, as Excel (.xlsx) or as CSV (Excel: *File → Save As → CSV UTF-8 (Comma delimited)*).

Barcodes: before typing or pasting barcodes in Excel, select the barcode column and choose *Format Cells → Text*. **Why:** otherwise Excel treats a barcode as a number. It drops the zeros in front (`0012345678905` becomes `12345678905`) and shortens long ones (`6.00107E+12`), so the scanner would no longer find the product. The Excel downloads already have codes as text. On import, zeros Excel dropped are put back when the barcode's check digit proves them, and a shortened barcode is refused with a message.

4. **Choose the file**, then **Check file (nothing is saved)**. Every row goes through exactly the same checks as the screens — then everything is undone. You see each row's result and a list of errors.



5. Fix the errors (**Download error report (CSV)** gives the failed rows with the reason in the first column), and check again.

6. **Import.** By default a file is imported **all or nothing**: if any row has an error, nothing is imported. Tick **Skip rows with errors** to import the good rows and get the others in the error report.

Why the check first: a mistake in a file can change hundreds of products at once. The check shows you exactly what would happen, and what would be refused, before anything changes.

Rules worth knowing

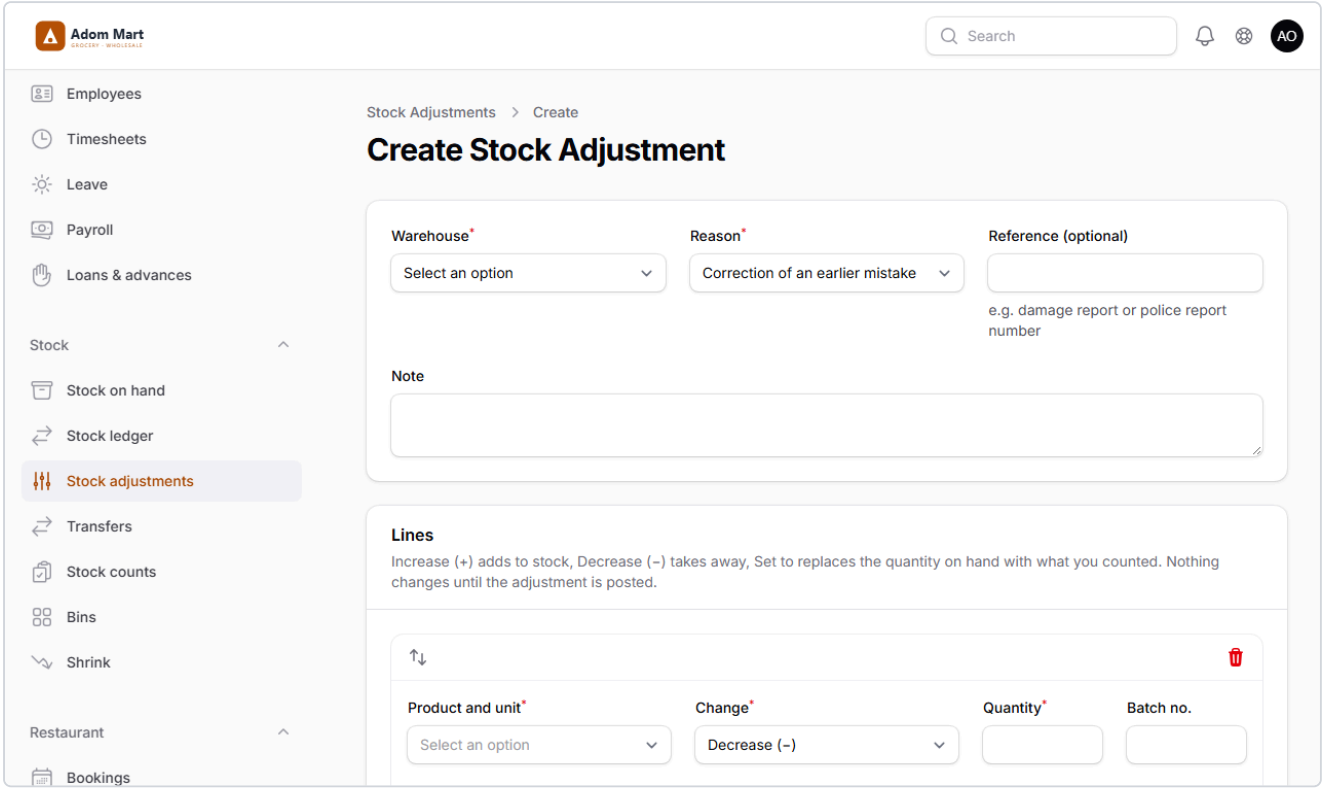
- **Empty cells leave a detail unchanged** when updating. To change only prices, a file with just `sku` and `price_RETAIL` is enough.
- **Headings** are matched loosely: `Unit cost`, `unit_cost` and `UNIT COST*` are the same column. Columns the import does not know are listed and ignored.
- **Units** (`base_unit`, `unit`) can be the code or the name: `TAB`, `Tablet`, `Strips` and `Cont.` all work.

- **Yes/no columns** accept yes, no, y, n, true, false, 1, 0.
- **Numbers** may have thousands separators (`1,250.50`). **Dates**: `2026-10-31` or `31/10/2026`.
- **Location** (products file): where the product is kept, e.g. `Shelf B2`. It shows on the product, the stock sheet and the till's search list.
- **Barcodes and serial numbers**: several in one cell, separated by `|`. In CSV downloads a code starting with 0 appears as `="0123…"`, which is how Excel is told to keep the zeros; leave it like that.
- **Large files** (thousands of products) take a few minutes to check and import. Wait for the result; do not click again. The tills and the rest of the back office keep working meanwhile: the file goes in 50 rows at a time. *All or nothing* still holds: the whole file is checked first and imported only if no row has an error. (Opening balances and stock adjustments go in as one piece, since their rows make one document.)
- Each row is checked with **your permissions**: changing prices needs the pricing permission, credit limits need the credit permission.
- Every import is listed under **Import history** on the same page and recorded in the audit log under your name.

> **Warning:** Only import customers who agreed to be on your records, and never use real customer data in a test system.

14.3 Stock adjustments

Stock → Stock adjustments. A stock adjustment changes stock outside sales and purchases — breakage, theft, expiry, samples, items found, or correcting a mistake. One adjustment covers one warehouse and has as many lines as you need.



1. **New stock adjustment:** the warehouse, the **reason** (damaged, expired, theft / missing, found / surplus, own use / sample, correction…) and an optional **reference** (e.g. a damage report number).
2. **Lines** — for each product, one of:
    - **Increase (+)** — adds to stock (give the unit cost if you know it; otherwise the average cost is used);
    - **Decrease (−)** — takes away from stock;
    - **Set to counted quantity** — the new quantity on hand; the system works out the difference when you post. Batch-tracked products need the batch number; serial-numbered products need one serial number per unit.
3. Save. The adjustment is a **Draft**: you can still change it, or **Add lines from CSV**.
4. **Post** — the changes go into the stock ledger.

Approval: an adjustment worth more than the limit in **System → Settings → Stock** (default GHS 500 at average cost) must be **submitted for approval** and **approved and posted by a manager other than the person who prepared it**.

> **Why:** stock adjustments are where stock losses are hidden. A second person checking large adjustments, a reason on every line and a permanent record of who did what are the standard controls.

The adjustment page shows each line's stock **before**, the **change** and **after**, and the value. **Export → Adjustment voucher (PDF)** prints it with a signature line for the approver.

> **Note:** a posted adjustment cannot be changed or deleted. To correct it, post another adjustment. The quick **Opening stock / adjustment** button on *Stock on hand* still exists for a single product.

14.4 Stock takes (counting stock)

There are two ways to bring the system in line with what is on the shelves.

A. A stock count (recommended for full or regular counts)

1. **Stock → Stock counts → Start a count** for a warehouse (or one bin). The system freezes what it expects at that moment. Leave **blind** on so counters do not see the expected figure.
2. Open the count and use **Count sheet**:
    - **PDF** to count on paper (SKU, barcode, product, unit, bin, and an empty *Counted* column; signature lines for the counter and the checker);
    - **CSV** to fill in Excel or on a tablet.
3. Enter the counts: scan with a handheld (Chapter 9), type them on the count page, or fill the CSV's `counted_qty` column and use **Import counted quantities (CSV)**. Rows left empty are **not counted** (not set to zero).
4. **Submit**, then a manager **approves and posts**. The differences become stock movements, with their value.

B. Update stock from a stock sheet (quick corrections)

1. **Stock → Stock on hand → Stock sheet (before a count)**: choose the warehouse and category, whether to show the system quantities (leave them out for a blind count), and PDF or CSV. This is the **current stock standing**.
2. Or, to update stock with a file: **Stock → Stock adjustments → Update stock from a CSV file**, choose the warehouse and reason, and **Stock sheet — current stock standing (CSV)**. Its `qty` column holds the system quantity; type the **new** quantity where it differs.
    - **Batches and expiry dates:** a product tracked by batch (most medicines) has **one row per batch**, with its `batch_no`, `expires_on` and quantity. Type the batch's new quantity; **add a row** (copy one and change `batch_no`, `expires_on`, `qty`) for a batch found that the system does not know; change `expires_on` to **correct a wrongly typed expiry date**. A new batch needs its expiry date.
    - The `location` (where the product is kept) and `bins` columns are for information: change a location on the product (or the products file), and move stock between bins on the Bins screen.
3. Import it. Rows whose quantity did not change are skipped; the others become **Set to** lines in a new draft adjustment (for a batch: an increase or decrease of the difference, worked out when you import). A corrected expiry date applies at once and is in the audit log.
4. Open the adjustment, check it, and post it (with approval above the limit).

Why two ways: a stock count freezes the expected figures and compares them (best for full counts while trading continues); the stock sheet route is quicker for a few corrections.

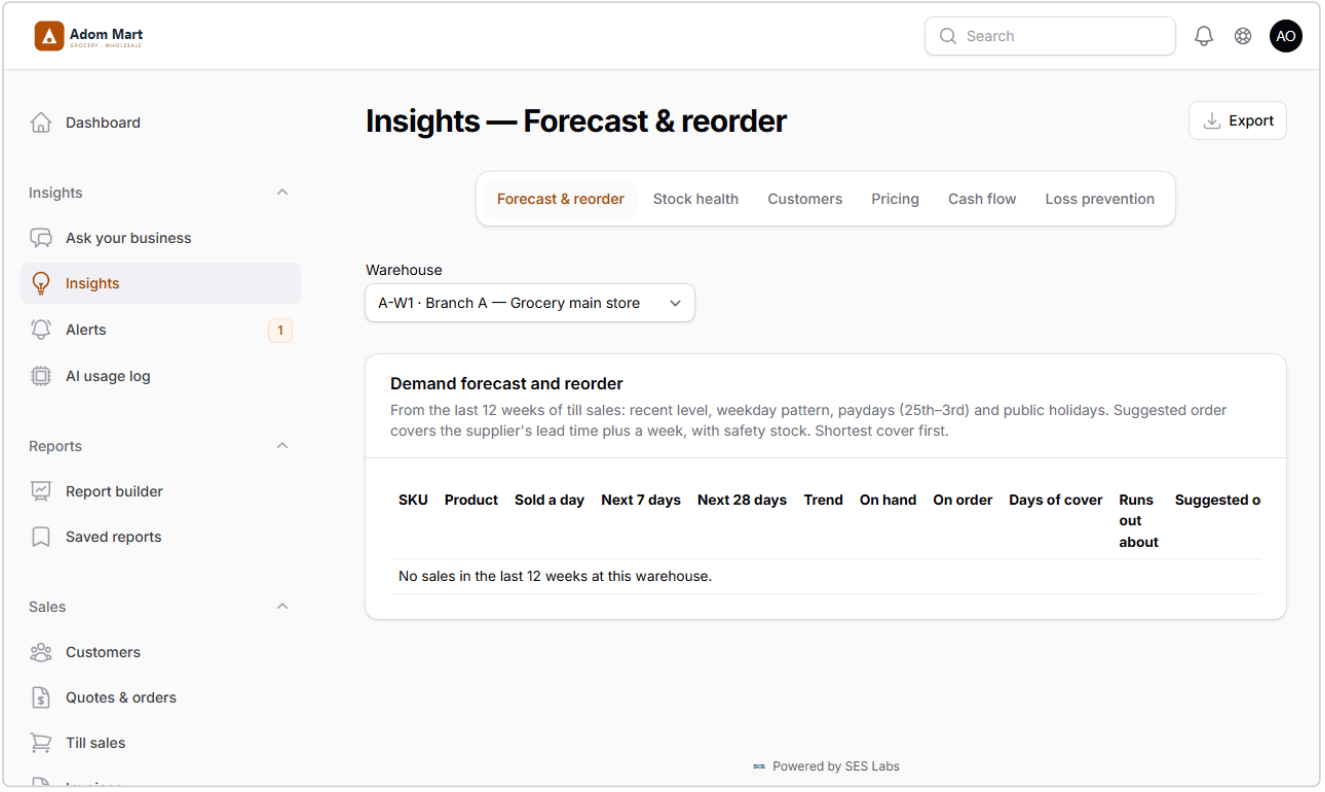
> **Tip:** count when the shop is closed, or count one area (bin or category) at a time. Sales during a count are still recorded normally.

15. Insights, alerts and the AI assistant

15.1 What works straight away (no AI needed)

The **Insights** menu works on every installation, offline too. It is ordinary arithmetic on your own sales and stock — nothing is sent anywhere and nothing needs installing.

Page	What it shows	Who sees it
Insights → Forecast & reorder	Expected sales of every product for the next 7 and 28 days, days of stock left, the day it runs out, and a suggested order quantity and min/max	Managers (*See insights*)
Insights → Stock health	Dead stock (no sale for 90 days), overstock, batches that will expire before they sell, and transfers between branches	Managers
Insights → Customers	Customer segments (champions, loyal, at risk…), regulars who have stopped coming, products bought together	Company-wide managers
Insights → Pricing	Products with a low margin or sold below cost, markdown suggestions, how much each promotion lifted sales	Company-wide managers
Insights → Cash flow	Money expected in and out, week by week, for 8 weeks	Company-wide managers
Insights → Loss prevention	For each cashier: refunds, refunds without a receipt, discounts, drawer over/short, kitchen voids, and a risk score	Owner, general, finance and branch managers
Insights → Alerts	A to-do list built every night from all of the above	Managers

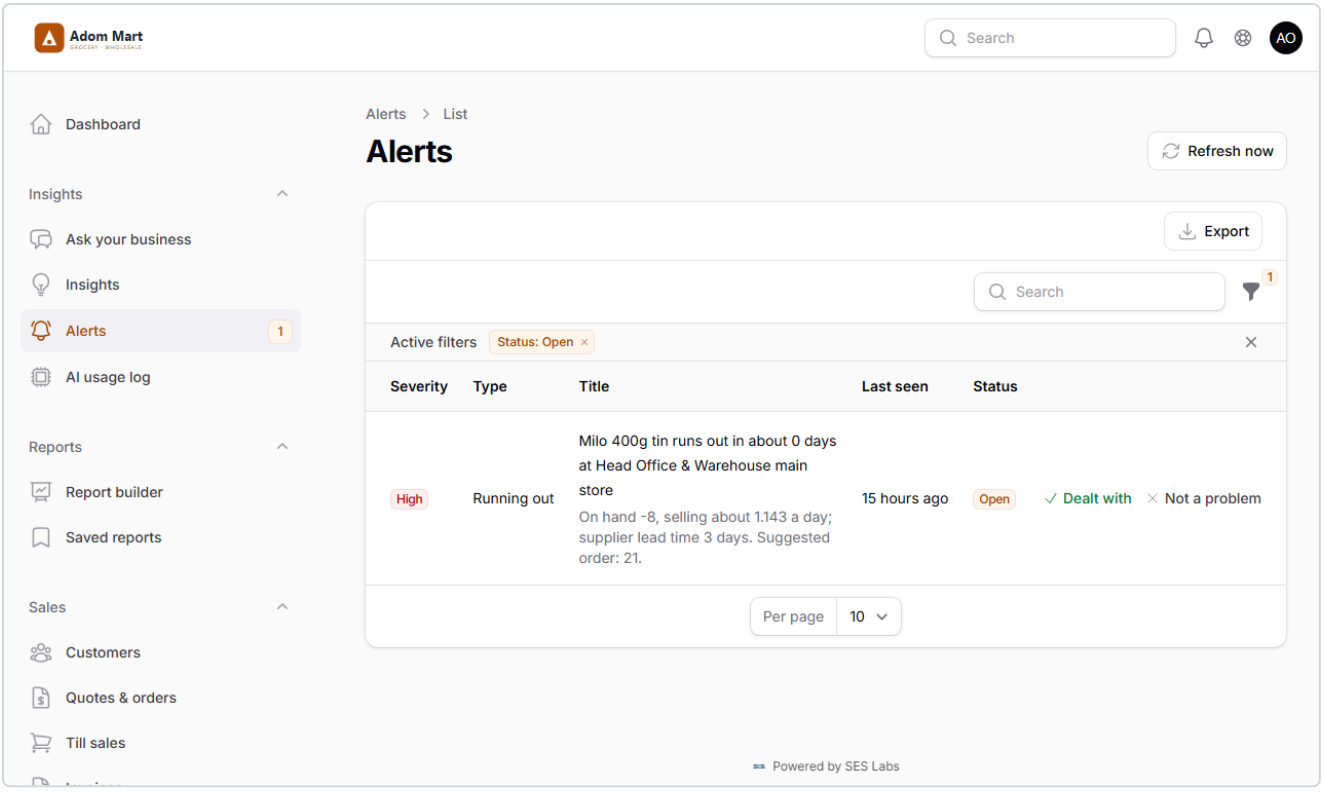


How the forecast works (in plain words): it looks at the last 12 weeks of till sales, gives the most recent weeks the most weight, and adjusts for the day of the week, for **paydays** (the 25th to the 3rd, when shops in Ghana are busier) and for **public holidays** (fixed holidays, Easter and Farmers' Day; Eid dates change each year and show up through your own history). The suggested order covers the supplier's **lead time** (set it on the supplier) plus a week, with a little safety stock.

> **Why:** you see trouble — a best-seller running out, yoghurt that will expire, a till with too many refunds — before it costs money.

> **Note:** The risk score is a reason to look (receipts, CCTV), never proof. Cashiers with fewer than 20 receipts in the period are listed but not scored.

Alerts

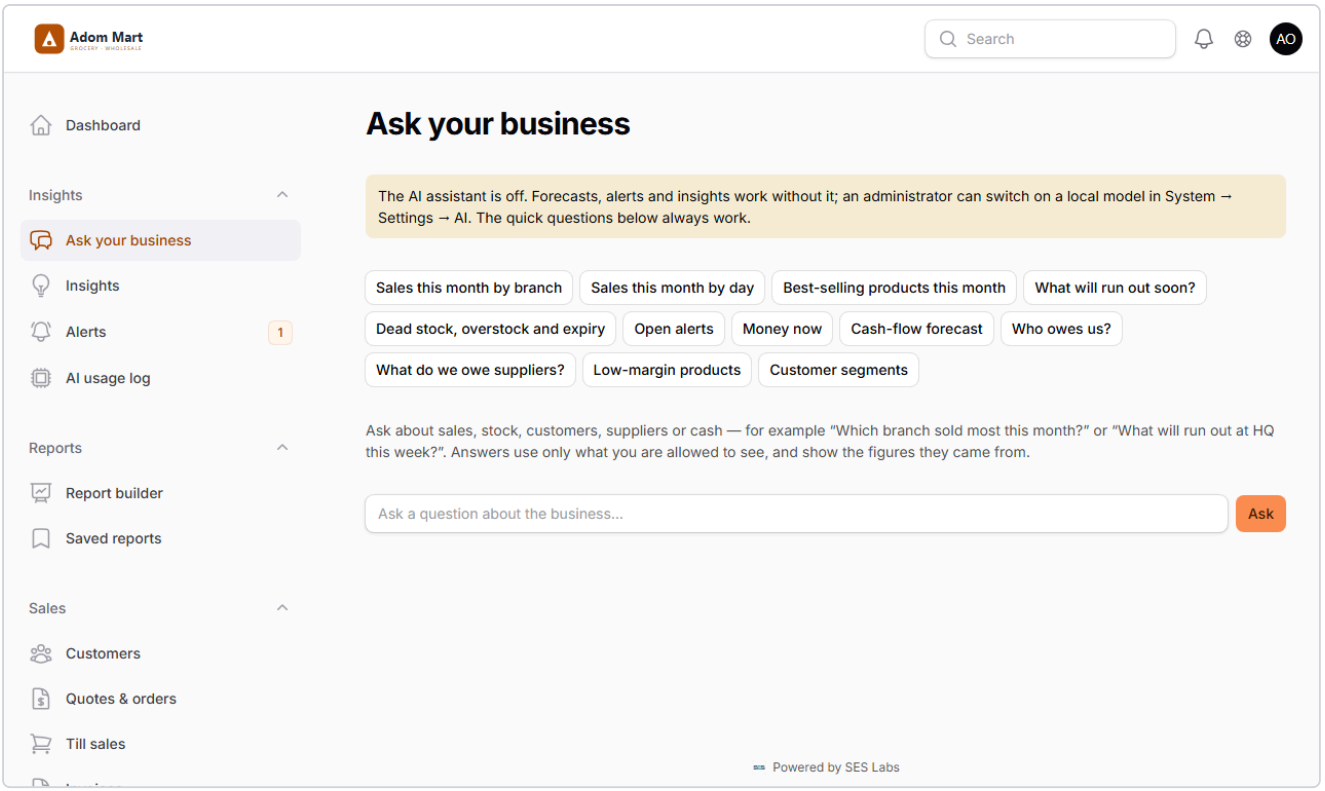


1. Every night at 04:30 the scheduler refreshes the alerts (it must be running — Chapter 4/5). To refresh now: **Insights → Alerts → Refresh now**.
2. For each alert choose **Dealt with** (say what you did) or **Not a problem** (it stays quiet for 30 days).
3. An alert closes by itself when its cause goes away (e.g. the stock arrived).

15.2 The AI assistant — off unless you switch it on

The owner chose **no cloud AI for now**, so the AI assistant is **off**. Three features need it:

- **Ask your business** (the copilot): questions in plain language, answered from your reports.
- **Scan an invoice** on Supplier bills: a photo of a supplier invoice fills in the bill form.
- **Write description (AI)** on a product: a draft text for the online shop.



Without AI, *Ask your business* still offers **quick questions** (sales by branch, what will run out, who owes us…) answered straight from the reports.

You can switch on a **local model**: it runs on your own server, so nothing leaves your network and there is no monthly bill.

Requirements for a local model

	Minimum	Better
Memory (RAM)	16 GB	32 GB
Disk space	15 GB free	30 GB
Graphics card	Not required (slower)	NVIDIA with 8 GB or more

> **Why:** the model is a large file held in memory while it answers. On a small server answers take 20–60 seconds; with a graphics card a few seconds.

Installing Ollama (the program that runs local models)

On Windows:

1. Download the installer from **https://ollama.com/download** and run it (Next → Install). It starts by itself and sits next to the clock.
2. Open **Command Prompt** and download the two models (this takes a while, about 5 GB each):

```
ollama pull qwen2.5:7b
ollama pull qwen2.5vl:7b
```

Why two: the first answers questions and writes text; the second reads photos of invoices.

3. Check it works: `ollama run qwen2.5:7b "Say hello"` — it should answer.

On Ubuntu:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:7b
ollama pull qwen2.5vl:7b
```

The installer sets Ollama to start with the server.

> **Note:** Ollama listens only on the server itself (`127.0.0.1:11434`). Keep it that way: do not open port 11434 in the firewall.

Switching the assistant on

1. **System** → **Settings** → **AI**.
2. **AI assistant:** *Local model (Ollama on your server)*.
3. Leave **Ollama address** as `http://127.0.0.1:11434` (Ollama on the same server). If Ollama runs on another computer in your office, put its address here.
4. The model names must match what you downloaded (`qwen2.5:7b` and `qwen2.5vl:7b`).
5. Save, then open **Insights** → **Ask your business** and ask "Which branch sold most this month?".

> **Check:** the answer appears with the table it came from. The question is listed in **Insights** → **AI usage log**.

If you later choose Claude (cloud AI)

Only with the owner's decision, because data then leaves your servers:

1. Get an API key from Anthropic (the owner's own account).
2. Put it in the server's `.env` as `ANTHROPIC_API_KEY=...`, then run `php artisan optimize`. *Why* `.env`*:* keys are never stored in the database or shown on screen.
3. **System** → **Settings** → **AI:** choose *Claude*, and set **Cloud AI budget per month (USD)**. With a budget of 0 Claude is never used; when the month's budget is spent it stops until the 1st.
4. Keep **Keep personal data on your server** on (recommended): questions about customers or staff are then answered by the local model only (or shown without AI wording if there is none). In every case names, phone numbers and e-mail addresses are replaced by placeholders such as *[Customer 1]* before anything is sent, and put back when the answer arrives.

If you choose another cloud model (some are free)

Instead of Claude you can use another cloud service. Google Gemini, Groq, OpenRouter (its *free* models) and Mistral have free plans that limit how many questions a minute or a day; OpenAI is paid. Data leaves your servers just as with Claude, so the owner decides.

1. **Get an API key** from the service, signed in with the business's own account:

    - Google Gemini: aistudio.google.com → **Get API key**;
    - Groq: console.groq.com → **API Keys**;
    - OpenRouter: openrouter.ai → **Keys** (choose models ending in *:free*);
    - Mistral: console.mistral.ai → **API Keys**.
2. Put it in the server's `.env` as `AI_CLOUD_API_KEY=...`, then run `php artisan optimize`. *Why* `.env`*:* keys are never stored in the database or shown on screen.
3. **System** → **Settings** → **AI:** choose **Another cloud model**, then the **service**. Leave the model names empty to use the service's usual model, or copy an exact name from the service's model list (services rename their models from time to time). For **Other OpenAI-compatible service**, also give its address ending in `/v1`.
4. Keep **Keep personal data on your server** on, as for Claude. Names, phones and e-mails are masked before anything is sent.

> **Check:** ask a question in **Insights** → **Ask your business**; **Insights** → **AI usage log** shows the service and model that answered.

> **Note:** the monthly cloud budget applies to Claude only. Free plans cost nothing; with a paid service (e.g. OpenAI) set a spending limit in that service's own account. Invoice scanning with these services reads photos (JPG, PNG), not PDFs.

15.3 Using the AI features safely

- **The AI never changes anything.** It reads reports; it cannot post, approve, order or pay. An invoice scan only fills in a form: you check each line against the paper and press **Create**, and the bill then follows the normal approval rules.
- **Answers show their figures.** Every copilot answer comes with the tables it used. If a figure looks wrong, the table shows where it came from.
- **People only see what they may see.** A branch manager's questions use only their branch; money and customer figures for the whole company need a company-wide role.
- **Everything is logged.** *Insights → AI usage log* lists every question, what was sent (as the model saw it, with names masked), the answer and the cost.
- Photos of invoices: the local model reads JPG and PNG photos (not PDFs — take a photo or screenshot). Claude can read PDFs.

15.4 Troubleshooting

Message	What to do
The AI assistant is off	Switch it on (15.2), or use the quick questions
The local AI model is not reachable	Start Ollama (Windows: open it from the Start menu; Ubuntu: `sudo systemctl start ollama`) and check the address in Settings → AI
The local AI model answered with an error: model not found	Download the model named in Settings (`ollama pull <name>`)
Answers are very slow	Normal without a graphics card; use a smaller model such as `qwen2.5:3b` (less accurate)

Message	What to do
This month's cloud AI budget is used up	Wait for the 1st, raise the budget, or switch to the local model
No alerts ever appear	Check the scheduler is running (Chapter 4/5) or press **Refresh now**

16. Messages, reports and connecting other systems

16.1 E-mail (any mail server)

The system sends e-mail for scheduled reports, invoices to customers (when you switch that on) and digital receipts. It uses **any SMTP mail server** you already have: Google Workspace or Gmail, Zoho Mail, Microsoft 365, or your web host's mail.

1. Get the SMTP details from your mail provider: server name, port, user name and password.
   - **Gmail / Google Workspace:** server `smtp.gmail.com`, port `587`; the password must be an **app password** (Google account → Security → 2-Step Verification → App passwords), not your normal password.
   - **Zoho:** `smtp.zoho.com`, port `587`.
   - **Your web host:** see its "e-mail accounts" page (often `mail.yourdomain.com`, port `587` or `465`).
2. Open the server's `.env` file (Chapter 4/5) and set:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=reports@yourcompany.com
MAIL_PASSWORD=<the app password>
MAIL_FROM_ADDRESS=reports@yourcompany.com
MAIL_FROM_NAME="Your Company"
```

3. In the `server` folder run `php artisan optimize`.
4. **System** → **Messaging** → **Send a test e-mail** to your own address.

> **Why:** the password lives only in `.env` on the server — never in the database or on screen. `MAIL_MAILER=log` (the default) writes e-mails to the log file instead of sending them, which is safe for testing.

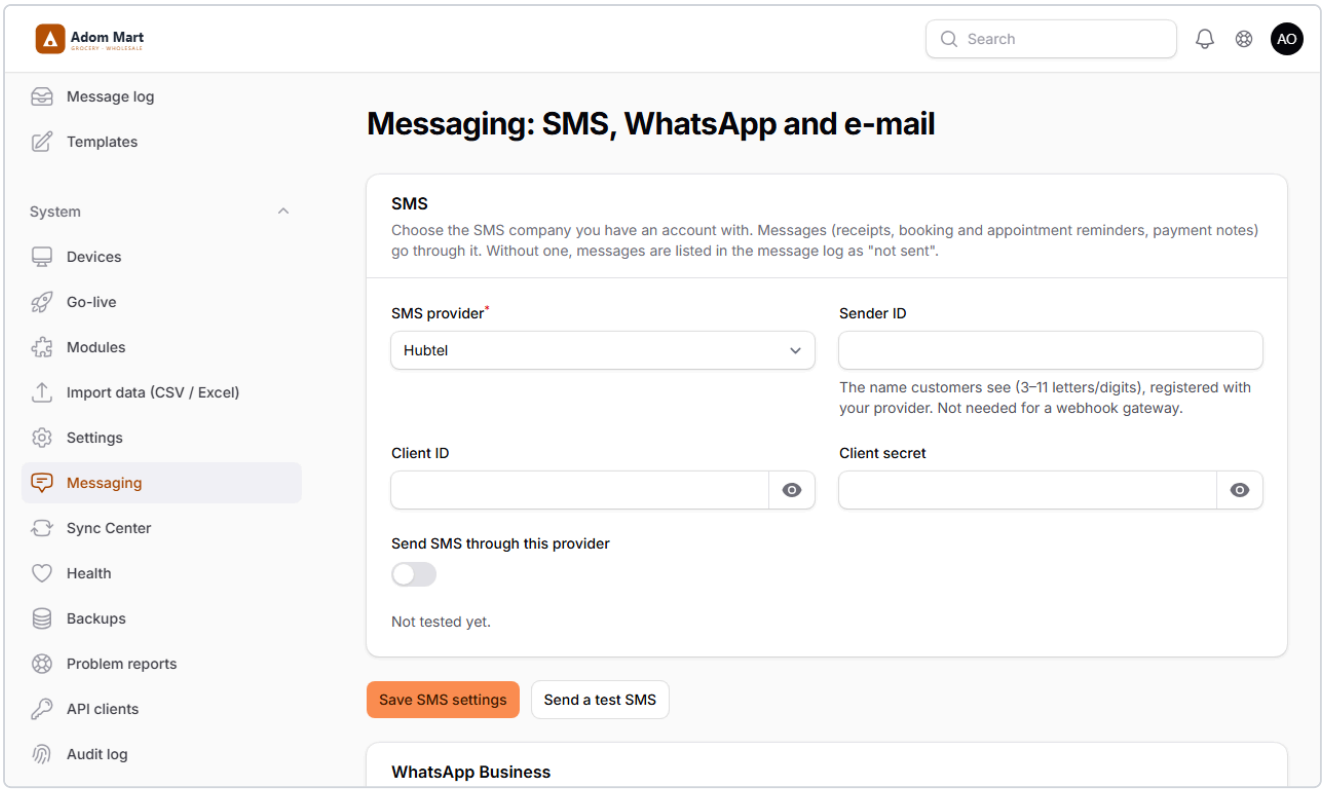
16.2 SMS — choose your provider

You can use **Hubtel**, **Arkesel**, **mNotify**, or **any other SMS company** through a small "webhook" bridge. Use the one you already have an account with.

1. Open an account with the provider (their website) and buy SMS credit.
2. Register a **sender ID** (the name customers see, 3–11 letters, e.g. `DemoShop`). In Ghana sender IDs must be approved (your provider handles it with the NCA); it can take a few days.
3. Copy the keys from the provider's dashboard:

Provider	What you need
Hubtel	Client ID and Client secret (Hubtel dashboard → Programmable SMS → API keys)

Provider	What you need
Arkesel	API key (Arkesel dashboard → SMS API)
mNotify	API key (mNotify dashboard → API)
Other (webhook)	The address of your gateway. It receives `{"to", "message", "reference"}` as JSON; an optional shared secret signs it (`X-Signature`).



4. **System** → **Messaging** → **SMS**: choose the provider, type the sender ID and keys, tick **Send SMS through this provider**, **Save**.
5. **Send a test SMS** to your own phone.

> **Note:** keys are stored encrypted with this server's key and never shown again — an empty box keeps the saved key. Each server that sends messages (normally head office) has its own settings.

16.3 WhatsApp Business (optional)

WhatsApp messages need a **WhatsApp Business account on Meta's Cloud API**; Meta charges per conversation. Businesses may only start a conversation with a **template approved by Meta**.

1. In Meta Business Manager create a WhatsApp Business app, add your business phone number and create a **permanent access token** (system user).
2. Create templates with the same wording as in **Messages** → **Templates** (WhatsApp lines); the placeholders become `{{1}}`, `{{2}}` … in order. Wait for approval.
3. **System** → **Messaging** → **WhatsApp Business**: Phone number ID and access token, tick **Send WhatsApp messages**, **Save**; **Send Meta's "hello_world" test**.

4. In **Messages** → **Templates**, put the approved template name on each WhatsApp line and switch it on.

When WhatsApp is on and a message has an active WhatsApp template, customers get WhatsApp instead of SMS.

16.4 Which messages are sent, and their wording

Messages → **Templates** lists every message per channel. Edit the text, use the placeholders shown, and switch each one **on or off**.

Message	Starts	Sent when
Receipt	On	The cashier presses **Send receipt** on the till (SMS, WhatsApp or e-mail)
Table booking / appointment confirmed and reminder	On	A booking or appointment is made, and before it starts
Invoice sent to the customer	**Off**	An invoice is issued (e-mail to the customer's address)
Payment received	**Off**	A customer pays on account
Scheduled report	On	A scheduled report runs (16.6)

Messages → **Message log** shows every message: to whom, the text, and whether it was **sent**, is **waiting** (the provider was down — it is retried automatically for a few hours), **failed**, or **not sent** because no provider is set up. **Try again** resends one.

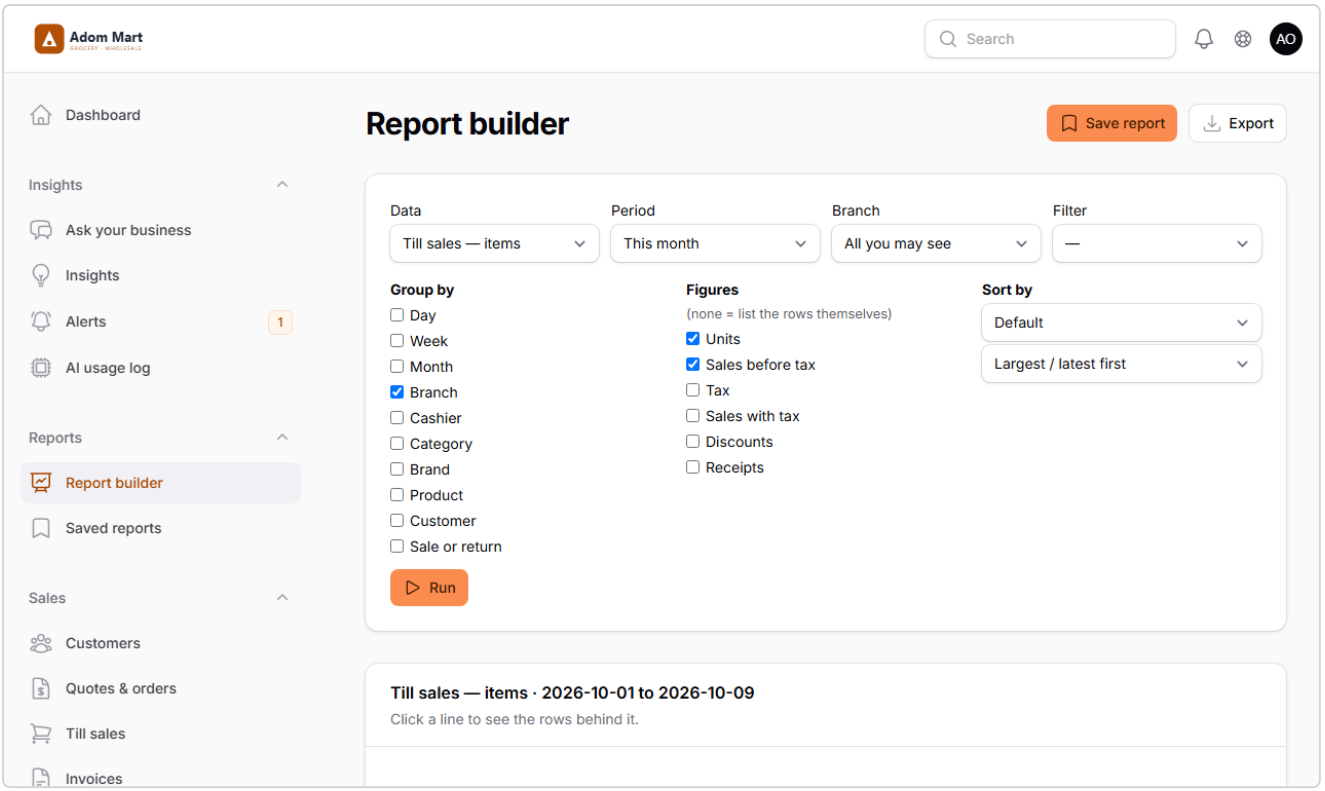
In the back office the bell (top right) tells approvers when something waits for them: purchase orders, supplier bills with price differences, stock adjustments, deliveries over the tolerance, stock counts, van settlements and expense claims.

16.5 Digital receipts at the till

After a sale, under the receipt: choose **SMS**, **WhatsApp** or **E-mail**, type the number or address, **Send receipt**. Only the channels switched on in Messages → Templates appear. A sale made offline can be sent once the till is back online.

16.6 Reports you build yourself

Reports → **Report builder**:



1. **Data**: till sales (items, receipts or payments), stock on hand, stock movements, customer invoices, supplier bills or the general ledger.
2. **Period** and **Branch**; optionally a **Filter** (e.g. Category contains "drinks").
3. **Group by** (branch, day, month, cashier, category, product, customer…) and **Figures** (units, sales, tax, discounts, receipts, value…). With no figures you get the rows themselves, each with an **Open** link to the document.
4. **Run**. Click any line to see the rows behind it (**Back to the summary** returns).
5. **Export** as Excel, CSV or PDF, or **Save report**.

Scheduling: in **Save report** choose *Every day / week / month*, the time, the format (PDF, Excel, CSV) and the recipients: e-mail addresses (the file is attached), WhatsApp numbers (a short note) or people in the system (bell). Reports are sent by head office every 15 minutes when due. **Reports → Saved reports** lists them with the next run and the last result.

> **Why:** a scheduled report runs *as the person who saved it*, so it never shows more than they are allowed to see.

16.7 Exporting to QuickBooks, Xero or Sage

If your accountant keeps the books in another program: **Finance → Export to QuickBooks / Xero / Sage**, choose the program and the period (normally last month), **Download the file**, and import it there:

Program	Import it under
QuickBooks Online	Settings → Import data → Journal entries
Xero	Accounting → Manual journals → Import

Program	Import it under
Sage 50 / Pastel	File → Import → Audit trail transactions

Before the first import, create the same account codes in the other program (or map them during import). The Xero file uses the tax rate "Tax Exempt" because tax is already in separate lines; rename it if your Xero uses another name.

16.8 Connecting other systems (the API)

For a developer connecting a website, a delivery partner or another program:

1. Optionally create a user just for that system with a narrow role (e.g. *Sales Rep*).
2. **System** → **API clients** → **New API client**: name, **scopes** (what it may do: read products, stock, customers, sales, invoices; create customers and orders; manage its webhooks), **Acts as** (that user), optionally **Only this branch**.
3. Copy the **key** shown once and give it to the developer securely. **New key** adds another (to rotate); **Revoke** stops every key at once.
4. The developer's documentation is at `https://<your server>/api/public/v1/openapi.json` (OpenAPI 3.1). Webhooks (product changed, sale, invoice, order confirmed or delivered) are signed with a secret shown once when they are added.

> **Why:** a key acts as a person, so it can never see more than that person may in the back office; everything it does is in the audit log under "API: ".

16.9 Importing from Excel

System → **Import data** now also takes **Excel (.xlsx)** files (the first worksheet). After choosing the file you see a **preview** of the first rows. If your columns have other names (e.g. "Item code" instead of `sku`), **Match columns** lets you say which column is which. Then **Check file** and **Import** as in Chapter 14.

> **Tip:** type dates in Excel as text like `2026-10-31` so they are read exactly.

17. Going live

This chapter takes a business from its old way of working (another till system, QuickBooks, Excel sheets or paper) to running on SES Labs, safely. Everything here is on one page in the back office: **System → Go-live** (owner and general manager).

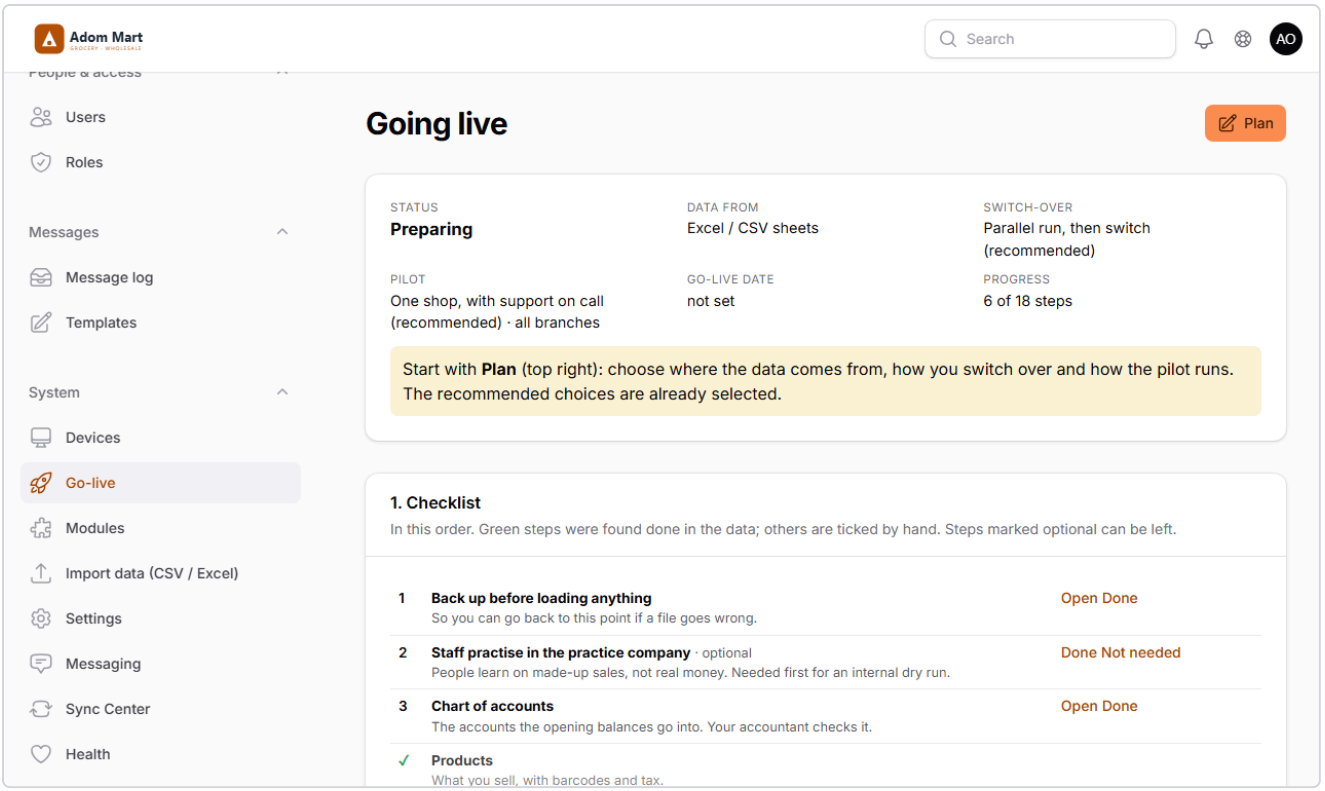
Why a plan: going live touches money, stock and every member of staff on the same day. A written plan, a checked set of figures and a way back make it a calm day instead of a risky one.

17.1 Make the plan

System → Go-live → Plan (top right). Choose:

1. **Where the data comes from** — tick every source you have:
    - **Excel / CSV sheets** — lists you keep yourself; use our templates.
    - **QuickBooks** — exports from QuickBooks Online or Desktop (lists and reports, saved as Excel or CSV).
    - **Another till (POS) system** — exports from Loyverse, Square, Lightspeed, Odoo and similar.
    - **Starting fresh** — nothing is brought over; you type in or import only products, prices and stock.
2. **How you switch over:**
    - **Parallel run, then switch (recommended)** — for one to two weeks both systems are used and each evening the day's totals are compared. **Why:** you see any difference while the old system still works.
    - **Switch fully on one day** — load the data on a quiet day (Saturday evening), use only the new system the next morning. Faster, but the go-live backup is your only way back.
    - **One till or branch first** — one till (or one branch) starts on the new system; the others follow after a week without problems.
3. **How the pilot runs:**
    - **One shop, with support on call (recommended)** — one real shop goes first. A **pilot report** is e-mailed every evening (17.6).
    - **Several shops at once** — two or three shops with different set-ups start together.
    - **Internal dry run first** — a whole simulated week in the **practice company** (17.3) before any real shop.
4. **Go-live date** — the opening balances are "as of" this date (normally the evening before the first day on the new system).
5. **Pilot branch(es)**, the **support e-mail address** (where problem reports and the pilot report go) and the **time of the pilot report** (default 19:00).

> **Check:** the top of the page shows your choices and the checklist below them.



17.2 Bring the data over

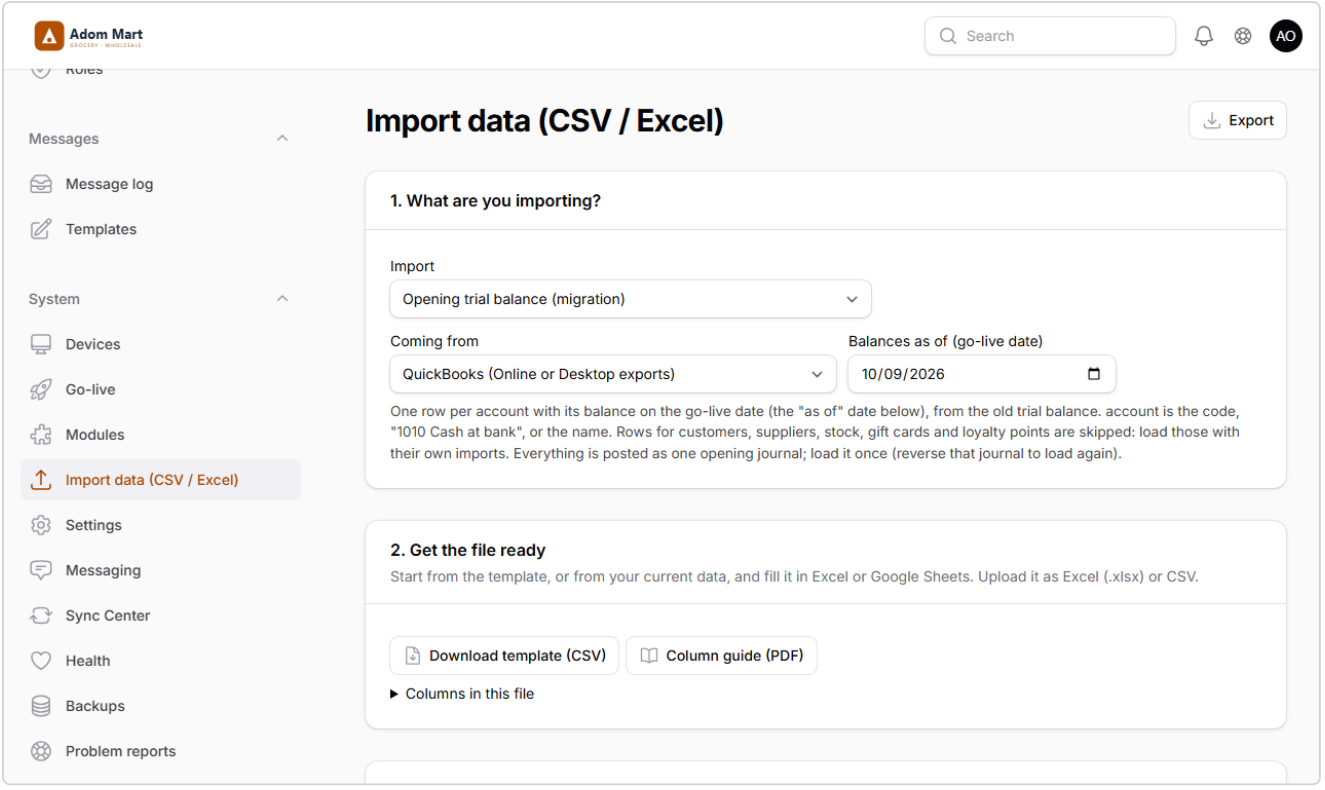
The **checklist** on the Go-live page lists each step in order. A step turns green by itself when the data is there; **Open** takes you to the right import, already set to your source. Do them in this order — later steps need the earlier ones.

Before you start: take a backup (**System → Backups → Back up now**). **Why:** if a file goes wrong, you can return to this point.

Step	From QuickBooks	From another till system	From Excel
Chart of accounts	*Account List* report	—	our template
Products	*Products and Services* list	the items export	our template
Prices	the same *Products and Services* file	the same items file	our template
Customers	*Customer Contact List*	the customers export	our template
Suppliers	*Vendor Contact List*	—	our template
Opening stock	*Products and Services* (Qty on hand) — or better, a fresh count	the stock / inventory export	our template

Step	From QuickBooks	From another till system	From Excel
Opening trial balance	*Trial Balance* report on the go-live date	—	our template
What customers owe	*Customer Balance Summary*	customer balances	our template
What you owe suppliers	*Vendor Balance Summary*	—	our template
Loyalty points	—	customers export with points	our template
Gift cards and store credit	—	gift card export	our template

For each file: **System → Import data**, choose the kind of import and **Coming from**. When you choose the file, the columns are matched for you (for example QuickBooks' *Product/Service Name* becomes *name*, *Sales Price / Rate* becomes *price*); check them, then **Check file**, fix what it reports, and **Import** (Chapter 14.2).



Good to know:

- **Codes from the old system are kept** as the products' SKUs, so the products, prices and stock files match; products added later in SyncLytics get system SKUs.
- **No SKU in the old system?** One is made from the name (*Malta Guinness 330ml* → `MALTA-GUINNESS-330ML`), the same way in the products, prices and stock files, so they match.

- **QuickBooks sub-items** (*Drinks:Malta 330ml*): the last part becomes the product name and the first its category.
- **Suppliers without a code** get one made from the name; customers are matched by code, phone number or exact name.
- **The opening trial balance skips** the lines for customers, suppliers, stock, gift cards and loyalty points: those come in through their own imports, with every customer's and supplier's detail. Whatever is left goes to *Opening balances (to clear)*.
- **Load the opening trial balance once.** To load it again, reverse the opening journal first (Finance → Journals).
- **A customer who is owed money** (a negative balance) gets it as **store credit** they can spend at the till.
- **Count the stock** rather than trusting the old system's quantities if you can: the opening stock is what everything else is measured against.

> **Warning:** only bring over customers who agreed to be on your records.

17.3 Let staff practise: the practice company

System → Go-live → Create the practice company. It copies your branches, tills, settings, products, prices and stock into a separate company with made-up customers. Choose the staff who get a practice login.

- In the back office, staff sign in at their e-mail address with **+practice** (*kofi+practice@yourshop.com*) and **their usual password** (and their usual two-step code).
- At a till, enrol a spare PC or tablet to the practice company (System → Devices, signed in as the practice owner); cashiers use **their usual username and PIN**.
- A purple **PRACTICE** bar shows on every screen and receipt. **Nothing leaves the practice company:** no GRA reporting, no SMS, WhatsApp or e-mail to customers, no updates to online stores or other systems. Its books never mix with yours.

> **Why:** people learn fastest by doing it, and mistakes there cost nothing.

Hand out the training material (17.7). When you want a clean start, **Archive the practice company** (its logins stop) and create a new one.

17.4 Check the figures

When everything is loaded: **System → Go-live → Check the figures**.

1. **Enter the old system's figures** on the go-live date: number of products, customers and suppliers; stock value at cost; what customers owe; what you owe suppliers; unspent gift cards; loyalty points. Take them from the old system's own reports (stock valuation, aged debtors, aged creditors).
2. Press **Post and check again**. The new system posts everything to the accounts and compares.

> **Check:** every difference shows ✓ and **Opening balances (to clear)** is **0.00**. If it is not zero, something from the old trial balance is missing or was loaded twice — your accountant compares the two trial balances.

> **Why:** a difference found now takes minutes to fix; found in a month, it takes days.

17.5 The switch-over

A parallel run (recommended)

1. **Start the parallel run** (Go-live page). From now, every sale is rung up in **both** systems.
2. Each evening, **Enter a day of the old system** for each pilot branch: its total sales, cash and number of receipts. The new system's figures for the same day are filled in for you.
3. A difference above 1.00 needs an **explanation** (a sale rung up in only one system, a price that differs). The checklist's *Parallel run is clean* step turns green after three days in a row that agree or are explained.

All on one day

Load everything on a quiet evening, check the figures (17.4), take the go-live backup and go live. Next morning only the new system is used.

One till or branch first

The first till or branch goes live as above. When it has run a week without problems, tick **First till ran a clean week** and move the others over the same way.

Going live

1. **Take the go-live backup** (Go-live page). **Why:** it is the exact point you can return to. It must be taken on the day you go live.
2. **Go live.** If a required step is still open, you are asked why you go live anyway; the answer is recorded.

From now on the new system holds the books. Keep the old system for **looking things up only** for at least three months.

17.6 During the pilot

- **Report a problem:** everyone has a **Report a problem** button (the life-buoy icon at the top of the back office; at the bottom of the till screen). It records what they wrote, the screen they were on and the system's recent errors, and e-mails support. Support and IT see the list under **System → Problem reports**, where they close each one with what was done.
- **The pilot report** is e-mailed every evening at the plan's time while the parallel run lasts and for 60 days after going live: sales per pilot branch, cash over/short, sales that reached the server late (a till was offline), the parallel-run difference, the problems reported today, and the health checks that are not green. **Show today's report** on the Go-live page shows it at any time.

- E-mail must work for both (Chapter 16.1). The support address can also be set in `.env` as `ERP_SUPPORT_EMAIL`.

17.7 Training material

Printable guides, in `docs/training/pdf/` of the release (rebuild them with `npm run training` in the `pos` folder):

File	For	What it covers
`teller-quick-card.pdf`	Tellers (people at a till)	One sheet to keep at the till: selling, payments, returns, offline, end of shift (with or without vaults)
`supervisor-guide.pdf`	Supervisors	Approvals, the review queue, lock-outs, end of day
`storekeeper-guide.pdf`	Storekeepers	Receiving, transfers, counts, adjustments
`accountant-guide.pdf`	Accountants	Opening balances, weekly checks, month-end
`manager-owner-guide.pdf`	Owners and managers	Access, daily checks, going live
`cashier-and-income-controller-guide.pdf`	Cashiers and income controllers	Vaults: issuing floats, receiving trays, handing over, blind counts, differences, bank deposits
`video-scripts.pdf`	Whoever records training videos	Nine short videos, what to show and what to say

Each guide ends with exercises to do in the practice company.

17.8 The rollback plan: going back to the old system

Decide in advance **who** may decide to go back (normally the owner), and keep the old system ready until the pilot is over.

When to go back: the tills cannot sell for more than an hour and support cannot fix it; or the figures are wrong in a way nobody can explain. A slow screen or a wrong price is fixed, not rolled back.

1. **Decide and record it: System → Go-live → Go back to the old system**, with the reason.
2. **Keep selling:** switch the tills back to the old system straight away. During a parallel run the old system is already up to date — that is the point of it.
3. **Keep what happened on the new system:** export today's sales (**Sales → Till sales → Export**) and any other documents created since going live, so they can be entered in the old system.
4. **Only if the new system's books must be put back** to the go-live point (for a second attempt later): the technician restores the **go-live backup** (shown on the Go-live page) as in Chapter 12.3. Everything entered after it is replaced — that is why step 3 comes first.
5. **Find the cause** with support (the problem reports and the pilot reports help), fix it, practise again in the practice company, and plan a new date.

Check before the go-live day: the people involved know this plan; the old system still opens; the go-live backup was copied off the server (System → Backups shows its copies).

17.9 The independent security test

Before the first customer's server is reachable from the internet, SyncLytics has an outside security firm try to break into a test copy of SES Labs: a *penetration test*. The full pack is in `docs/pentest/`, with the request for quote, the rules for the testers, how to build the test machine and how findings are handled.

> **Why:** your sales, stock, payroll and accounts are in SES Labs. An independent firm finds what the people who built the software are used to not seeing. A bank, an insurer or a large customer may also ask for the firm's report.

For the technician building the test machine: use a spare PC or a virtual machine on a network of its own, never a server with real data. After installing, run `php artisan erp:pentest-setup` as described in `docs/pentest/03-test-environment.md`. It fills the empty installation with made-up companies and one sign-in per role, and writes the sign-ins to a private file instead of the screen. It refuses to run where a company already exists.

For the owner: the pilot does not start while a *Critical* or *High* finding is open. Line 21 of the launch checklist is ticked when the firm's retest letter confirms they are fixed.

18. Reference

18.1 Security levels

A person's **security level** is the highest level of role they can ever be given.

Level	Typical people
L6 · Owner	Owner / director (everything)
L5 · Executive	General Manager, Finance Manager
L4 · Manager	Branch Manager, Accountant, HR Manager, Warehouse Manager, Pharmacist, IT Admin
L3 · Supervisor	Supervisor, Payroll Officer, Marketing
L2 · Staff	Cashier, Storekeeper, Waiter, Tailor, Repair Technician, Stylist / Therapist, Sales Rep
L1 · Restricted	Auditor (read-only), Kitchen Staff

IT Admin manages users, devices, modules and settings, but gets **no** access to sales, money or stock unless the owner also gives them a business role.

18.2 Roles provided

Owner · General Manager · Finance Manager · Accountant · Auditor · HR Manager · Payroll Officer · Purchasing Officer · Purchase Approver · Warehouse Manager · Storekeeper · Picker / Packer · Branch Manager · Supervisor · Teller (sells at a till; called Cashier before version 0.30) · Cashier (issues floats, receives the tellers' cash) · Income Controller (accounts: receives the day, decides differences, banks the cash) · Sales Rep · Van Sales Rep · Customer Service · Marketing · E-commerce Manager · Delivery Rider · Production Manager · Machine Operator · Quality Inspector · Pharmacist · Pharmacy Technician · Waiter / Bartender · Kitchen Staff · Repair Technician · Tailor · Stylist / Therapist · IT Admin.

You can change what a role allows under **People & access** → **Roles** (you can only add permissions you hold yourself).

18.3 Settings in `.env`

Key	Meaning
`APP_ENV`	`production` on a live system
`APP_DEBUG`	`false` on a live system (never `true`: it shows technical details)
`APP_URL`	The address people use to reach this server
`APP_KEY`	Encryption key (created once by `key:generate`). **Back it up, never change it.**
`DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Database connection (`127.0.0.1`, `5432`, `erp`, `erp`, your password)
`ERP_NODE_ROLE`	`hq` (head office or single server) or `branch` (branch server)

Key	Meaning
`ERP_NODE_ID`	Short unique id of this server (`HQ`, `B01` …), stamped on every record
`ERP_AUDIT_KEY`	Audit-log seal (created once by `erp:audit-key`). **Back it up, never change it.**
`REVERB_*`	Live updates (optional, 4.9 / 5.9)
`MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	Outgoing e-mail (Chapter 16.1). `MAIL_MAILER=log` only writes e-mails to the log
`BACKUP_PASSPHRASE`, `BACKUP_PG_BIN`, `BACKUP_TIME`, `BACKUP_KEEP_LOCAL`	Backups (Chapter 12.2). **Keep a written copy of the passphrase away from the server.**
`GRA_EVAT_URL`, `GRA_EVAT_KEY`, `GRA_EVAT_TIN`, `GRA_EVAT_TIMEOUT`	GRA E-VAT access from GRA (Chapter 11.5)
`BRAND_NAME`, `BRAND_VENDOR`, `BRAND_WEBSITE`	The product's name, the company that publishes and licenses it (SyncLytics unless a reseller supplies it), and its website (normally left as delivered)
`ERP_SUPPORT_EMAIL`	Where problem reports and the evening pilot report go when the go-live plan names no address (Chapter 17.6); empty = SyncLytics support (info@synclyticsenterprisesystems.com)
`BRAND_SUPPORT_PHONE`, `BRAND_SUPPORT_EMAIL`	The publisher's hotline and e-mail on the Licence page and in licence messages; empty = SyncLytics' own (normally left as delivered)
`SESSION_SECURE_COOKIE`	`true` once the site uses HTTPS (the Ubuntu script sets it)
`ANTHROPIC_API_KEY`	Only if you choose Claude (cloud AI) in Chapter 15. Leave empty for no cloud AI.
`AI_CLOUD_API_KEY`	Only if you choose *Another cloud model* (Google Gemini, Groq, OpenRouter, Mistral, OpenAI …) in Chapter 15.
`BROADCAST_CONNECTION`, `QUEUE_CONNECTION`, `CACHE_STORE`, `SESSION_DRIVER`	Leave as provided

After changing `.env`, run `php artisan optimize`.

18.4 Network ports

Port	Used by	Open to
443	The web server, Caddy (back office, tills, live updates)	Shop network, or the internet with an internet name
80	Sends visitors on to https; on a server without an internet name, also hands out its certificate (`/ca.crt`)	As 443
9001–9004	PHP workers behind the web server (Windows)	**This server only**

Port	Used by	Open to
8080	Live updates (Reverb)	**This server only** (reached through 443)
5432	PostgreSQL	**This server only** — never open it to the network
22	SSH (Ubuntu remote administration)	Your administrators only

18.5 Useful commands (in the `server` folder)

Command	What it does
`php artisan erp:install`	Create the company and owner (once)
`php artisan erp:demo-catalog --as=<email>`	Sample products and stock — **test installations only**
`php artisan migrate --seed --force`	Create or update the database structure (installation and updates)
`php artisan optimize` / `optimize:clear`	Apply `.env` changes / clear cached settings
`php artisan schedule:work`	The scheduler (normally runs as a task or cron, not by hand)
`php artisan erp:sync`	Sync a branch server with head office now
`php artisan erp:node:join <hq-url> <hq-id> <company-id>`	Pair a branch server with head office
`php artisan erp:stock-check` after a large import	Confirms stock levels still match the ledger
`php artisan erp:bill-memberships`	Raise membership invoices that are due (runs daily by itself)
`php artisan erp:backup` / `erp:backup --verify`	Encrypted backup now / check the newest one (both also run by themselves)
`php artisan erp:backup:key`	Create the backup passphrase (once)
`php artisan erp:backup:restore <file>`	Put a backup back (replaces the data) — Chapter 12.3
`php artisan erp:health`	All health checks (System → Health); fails when something needs fixing
`php artisan erp:evat-batch --now`	Send the GRA E-VAT till sales waiting (daily mode)
`php artisan erp:load-test`	Speed test of sales and reports on a test system (leaves no data)
`php artisan erp:pentest-setup --license=<file>`	Penetration test machine only: fill a **new, empty** installation with made-up companies and test sign-ins (Chapter 17.9)
`php artisan erp:reports`	Send scheduled reports that are due (runs every 15 minutes by itself)
`php artisan erp:pilot-report --show` / `--now`	Print / e-mail the pilot report now (sent every evening by itself during a pilot — Chapter 17.6)
`php artisan erp:insights`	Refresh forecasts and alerts now (runs daily at 04:30 by itself)
`php artisan license:status`	Licence state and limits

Command	What it does
`php artisan erp:audit-verify`, `erp:stock-check`, `erp:gl-check`	Health checks (Chapter 13)

18.6 Till keyboard shortcuts

Key	Action
F2	Search / scan
F3	Today's route (vans)
F4	Choose customer
F6	Tables (Restaurant)
F7	Repairs (Electronics)
F8	Return
F9	Alterations (Fashion)
F10	Appointments (Services)
F12	Pay

18.7 Go-live checklist

The checklist lives in the software: **System** → **Go-live** (Chapter 17) shows each step in order and ticks most of them by itself. Before the go-live day also confirm:

- ☐ Licence **Active** with the right modules and limits; two-step sign-in for every manager, recovery codes stored safely.
- ☐ Branch, warehouse and till codes correct (they cannot change later); tax groups checked by your accountant.
- ☐ Every user with the right role and scope; PINs set; IT Admin without business roles.
- ☐ Every till enrolled, printing, opening the drawer, and tested **offline**.
- ☐ Nightly backup running, copied off the server, and a **restore practised** (Chapter 12.3); **System** → **Health** all green.
- ☐ `.env`, the backup passphrase, the owner password and recovery codes stored safely, away from the server.
- ☐ Everyone knows the rollback plan (Chapter 17.8).

18.8 About this guide

This guide is generated from the files in `docs/manual/` and updated with every release. The software version and date are on the cover page.

19. Case study: a cloud server from start to finish

This chapter follows one real installation from start to finish: the first client's head office server, rented from **Contabo** and reached on the client's own internet name. It shows the order of the steps, what each screen asked for, and the problems met on the way. Use it beside Chapter 5, which explains each step in detail.

The client's real name, internet name and server address are replaced here by examples: `erp.clientdomain.com` and `203.0.113.10`.

19.1 What was decided first

Decision	Choice	Why
Where the server runs	Rented cloud server (setup D)	Two shops in different places; no server room; the tills keep selling if the internet drops.
One server or shared	**One server for this client**	The standard: own data, licence, backups and updates. A shared server (Chapter 5.0) is only for small clients starting out.
Provider and plan	Contabo **Cloud VPS 6**: 6 cores, 12 GB memory	12 GB lets the database keep the business in memory. The smaller plan saves very little and leaves no room to grow.
Disk	**100 GB NVMe** (not the 200 GB SSD at the same price)	The database works much faster on NVMe; 100 GB lasts two shops for years.
Region	**European Union** (Germany), free	Contabo serves Africa from Germany; its own test showed it as fast as the paid UK option from Ghana.
System	**Ubuntu 24.04**	What the install script is built and tested for.
Daily server copy	Contabo **Auto Backup**	A copy of the whole server kept off the server, on top of the system's own encrypted backups.
Term	24 months, paid up front	Cheapest monthly rate; matches the client's 24-month price lock.
Internet name	`erp.` in front of the client's own domain	Leaves the main name free for a website later.

19.2 Renting the server (Contabo)

1. On contabo.com: **VPS** → **Cloud VPS 6** → **Get Started**.
2. In the order page, check every section, because some defaults are not what you want:
    - **Term:** 12 or 24 months (cheaper than monthly).
    - **Region:** European Union.
    - **Storage:** change the default 200 GB SSD to **100 GB NVMe**.
    - **Image:** Ubuntu; open its version list and choose **24.04**.
    - **Auto Backup:** the default is *No Data Protection*: choose **Auto Backup**.
    - **Password:** choose a long one (20+ characters) and keep it in a password manager. Never send it by WhatsApp or e-mail.
3. Pay. Contabo e-mails the server's **IP address** when it is ready (minutes to a few hours).

Lesson: the order page's defaults (SSD, no backup) are the cheaper choices, not the right ones for a business server. Go through it line by line.

19.3 Pointing the internet name at the server (Namecheap)

1. Namecheap → **Domain List** → **Manage** next to the domain.
2. **Domain** tab: *Nameservers* must say **Namecheap BasicDNS**.
3. **Advanced DNS** tab → **Add New Record**: type **A Record**, host `erp`, value = the server's IP address, TTL Automatic → green tick.
4. Check it from any PC (it took a few minutes here):

   ```
   nslookup erp.clientdomain.com 8.8.8.8
   ```

   It must answer with the server's IP address.

Why before installing: the install script asks Let's Encrypt for the HTTPS certificate, and Let's Encrypt checks that the name leads to this server.

Lesson: a newly bought domain does not answer at all until a record is added. That is normal.

19.4 Getting the installation files

The installation uses a **release package made for this client** (`release:build`, run by SyncLytics): the code stamped with the client's name and signed with the SyncLytics key, plus the till app. The Ubuntu `install.sh` is copied beside it.

Lesson (fixed on 14 October 2026): the first package built for this client showed that the signed manifest was never copied to servers, so tamper protection was silently off everywhere. Since the fix, the package is checked before sending (signature valid, protected files unchanged).

19.5 Copying the files to the server (from Windows)

Windows 10 and 11 already have the tools. In PowerShell:

```
scp "<folder>\ses-labs-<client>-<build>.zip" "<folder>\installer\ubuntu\install.sh" root@203.0.113.10:/root/
```

- **First time:** "*The authenticity of host … can't be established*" — type `yes`. Your PC remembers the server and will warn if it ever changes. (To be certain, compare the fingerprint in Contabo's **VNC console** with `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`.)
- **The password does not show** as you type. To paste it, **right-click once** (Ctrl+V may paste nothing).
- **"Connection closed"** right after the password question happened once here. The second try worked. Causes: the password typed too slowly or wrongly, or the new server still finishing its first start.

Done when both files show **100%**.

19.6 Installing

1. Connect: `ssh root@203.0.113.10` (same password). The prompt becomes `root@vmi…:~#`.
2. **Do not** accept the "*New release … available, run do-release-upgrade*" offer: stay on Ubuntu 24.04, which the install script is tested on.
3. Set the server's clock to Ghana time (Contabo's servers start on German time), so the nightly backup and the logs use Accra times:

   ```
   timedatectl set-timezone Africa/Accra
   ```

4. Run the install script with the internet name:

   ```
   sudo bash /root/install.sh --zip /root/ses-labs-<client>-<build>.zip --domain erp.clientdo
   main.com --email info@synclyticsenterprisesystems.com
   ```

   It shows steps **1/10 to 10/10** and ends with *Done: the server answers at* *https://erp.clientdomain.com/admin*. Here the packages, libraries and database took about 5 minutes on a new Cloud VPS 6.
5. **Write down the backup passphrase** it shows, and keep it with the SyncLytics key backup. Backups cannot be restored without it.
6. Answer its questions to create the **company and the owner account** (Chapter 7).

What happened here: the script ended at step **9/10** without a message, so the firewall, the final check, the passphrase display and the company questions did not run. The cause was the SSH port lookup, which fails on Ubuntu 24.04 when SSH is started on demand. Fixed on 14 October 2026: the script now carries on, and any unexpected stop names the line. Everything before step 9 had worked, so the remaining steps were done by hand:

```
ufw allow 22/tcp && ufw allow 80/tcp && ufw allow 443/tcp && ufw --force enable && ufw status
curl -fsS https://erp.clientdomain.com/up > /dev/null && echo "Site OK"
grep '^BACKUP_PASSPHRASE=' /var/www/seslabs/server/.env
cd /var/www/seslabs/server && php artisan erp:install && chown -R www-data:www-data storage boot
strap/cache
```

The site answered over HTTPS straight away, with a Let's Encrypt certificate valid for three months and renewed automatically.

19.7 Licence and first sign-in

1. Open `https://erp.clientdomain.com/admin` and sign in as the owner. The padlock shows a valid certificate.
2. **Licence** page: copy the **installation code** (`SYNREQ-…`) and send it to SyncLytics. The code names the release build, which confirms the signed release is installed.
3. SyncLytics issues the key for exactly what the client bought. Here that was:

    - 2 shops, 4 tills and devices, 10 users;
    - Pharmacy, Grocery (to start the retail shop) and Retail & Wholesale (for when it is released);
    - accounts, and HR and payroll (added free for this client);
    - valid to the end of the set-up period. The full-year key follows when the first year's subscription is paid at go-live.

  Paste it on the Licence page → **Activate**. A key re-issued with more features simply replaces the earlier one.

4. **Set up your company** wizard, *Branches & modules* page. It opens with a **demo layout**: head office plus Grocery, Restaurant and Pharmacy branches, 2 tills each. Change it to the client's real shops **before** clicking *Finish setup*, or the licence limits refuse it. Here:

    - delete the demo branches the client does not have;
    - the pharmacy, code `PH`, **head office on**, module **Pharmacy** only, 2 tills;
    - the retail shop, code `RT`, module **Grocery / Supermarket** only, 2 tills;
    - leave *Retail & Wholesale* unticked (not released yet).

5. **Books:** a client that is **one registered company** running both shops switches off *System → Settings → Finance → Separate books for each industry* straight away, before anything is posted.

6. Company details and TIN under *Company → Legal entities*; users and roles (Chapter 7).

Lesson: read the wizard's last page line by line. Its demo branches look finished but are only an example.

19.8 After installing (the same day)

Check	Where	Expected
Server health	System → Health	Scheduler OK, backups OK, *PHP code cache: On*, HTTPS OK
A backup outside Contabo	System → Backups → add a second destination (Chapter 12.2)	Contabo's own backup is with the same provider; a second place protects against losing both
Firewall	Contabo panel and the server (`ufw status`)	Only SSH (22), 80 and 443 open; 443 for both TCP and UDP (HTTP/3)
Remote login	Your PC	Move from the password to an SSH key, and restrict port 22 to your office if possible
Tills	Each till device	Pair it (Chapter 9); it keeps selling if the internet drops. In the desktop till the server address is `https://erp.clientdomain.com`. Here the server's IP address was typed first, so the till could not connect; the settings were cleared (9.7) and the name entered. Newer till versions check the address before saving.
Speed	Outside trading hours: `php artisan erp:load-test --force` in `/var/www/seslabs/server`	No line marked *SLOW*

Speed from the shop

The back office felt slow at the shop. Measured: the server answered itself in 0.05 s, so the server was fine; the shop's line reached Google in about 50 ms with nothing lost, but the server in Europe in 150–230 ms with an occasional lost packet. Every click in the back office makes that trip. Two things help: **HTTP/3** (`sudo ufw allow 443/udp` ; the installer now does it), and a **cable instead of Wi-Fi** at the till PC. Selling is not affected: the till keeps its products and sales on the PC and sends them in the background.

To measure it yourself, on the shop PC in PowerShell:

```
1..3 | % { curl.exe -o NUL -s -w "first reply %{time_starttransfer}s  total %{time_total}s`n" ht
tps://erp.clientdomain.com/up }; ping -n 20 erp.clientdomain.com
```

Under 1 s per reply and no *Request timed out* is good.

19.9 Time and cost (this installation)

Item	Amount
Contabo Cloud VPS 6, 24 months	about $7.20 a month, paid up front
Auto Backup	$4 a month
Domain	the client's own (bought separately)
SyncLytics time	renting and DNS 30 minutes; copying and installing 30 minutes; licence and first set-up 1 hour